

Snapper Documentation

Comprehensive project guide

Generated on 2026-05-14

Table of Contents

[Snapper](#)

[AI Integration](#)

[API](#)

[Architecture](#)

[Backtesting](#)

[CLI](#)

[Configuration](#)

[Development](#)

[Messaging \(ZeroMQ\)](#)

[Observability](#)

[Operations runbook — multi-instance trade coordinator](#)

[Trading Strategies](#)

Snapper

Trading platform with market data collection, trade runtime, and backtester. Supports Kraken, Zonda, Walutomat exchanges and Polygon.io data.

Quick Steps

```
# Initialize database with seed data and build static assets
make migrate-dev run-static

# Start the server
make run-server
```

Open <http://localhost:8000/> and log in with the dev seed credentials — the default values are defined in `proprietary/data/seed/dev.toml` (the proprietary submodule). Override per-environment by editing the seed file before running `make migrate-dev`, or rotate after first login via `POST /api/auth/users/<username>/password`.

Features

- **Market data collection** — WebSocket and REST API from Kraken, Zonda, Walutomat, Polygon.io
- **Symbol correlation** — Underlying asset model linking instruments across exchanges (e.g., SPY, ESM6-CME, SPYX-USD-PERP all map to S&P 500). YAML-driven pattern matching, front-month rollover, contract ladder API
- **Trade runtime** — Facts-canonical live and paper trading with canonical Order and Execution facts, rebuildable Position and Balance projections, dispatched via a durable outbox
- **Strategies** — Framework for creating strategies based on RSI, MACD, cointegration, and TA-Lib indicators
- **ZeroMQ messaging** — Pub/sub architecture for market data and signals
- **Provenance and audit** — Every payload item carries `session_id` and `sequence_id` for gap detection. Control table (always-on) and telemetry table (toggleable) record all commands and operational events
- **Web dashboard** — FastAPI + React with real-time WebSocket
- **CLI** — Full system management via Typer CLI
- **Backtesting** — Strategy testing on historical data

Quick Start

Requirements

- Python 3.14+
- Poetry
- Node.js 25+ and pnpm (for frontend)
- TA-Lib (C library)

Installation

```
# Clone repository
git clone https://github.com/mateusz-klatt/snapper.git
cd snapper

# Install system dependencies (macOS)
brew install ta-lib

# Install Python dependencies
make setup

# Install frontend dependencies
make ui-setup

# Copy and configure environment variables
cp .env.example .env
```

Configuration

Edit .env file:

```
# Database (SQLite dev, PostgreSQL prod)
DB_URL=sqlite+aiosqlite:///./data/snapper.db

# Settings encryption in database
MASTER_PASSWORD=your_master_password

# HTTP Server
SERVER_HOST=127.0.0.1
SERVER_PORT=8000
SERVER_API_ONLY=false
SERVER_PROXY_HEADERS=true
SERVER_FORWARDED_ALLOW_IPS=127.0.0.1

# Telemetry recording (pings, heartbeats, GET reads)
TELEMETRY_RECORDING_ENABLED=false

# ZeroMQ broker
ZMQ_BROKER_XSUB=tcp://127.0.0.1:7500
ZMQ_BROKER_XPUB=tcp://127.0.0.1:7501
```

Running

```
# Initialize database and seed data
make migrate-dev

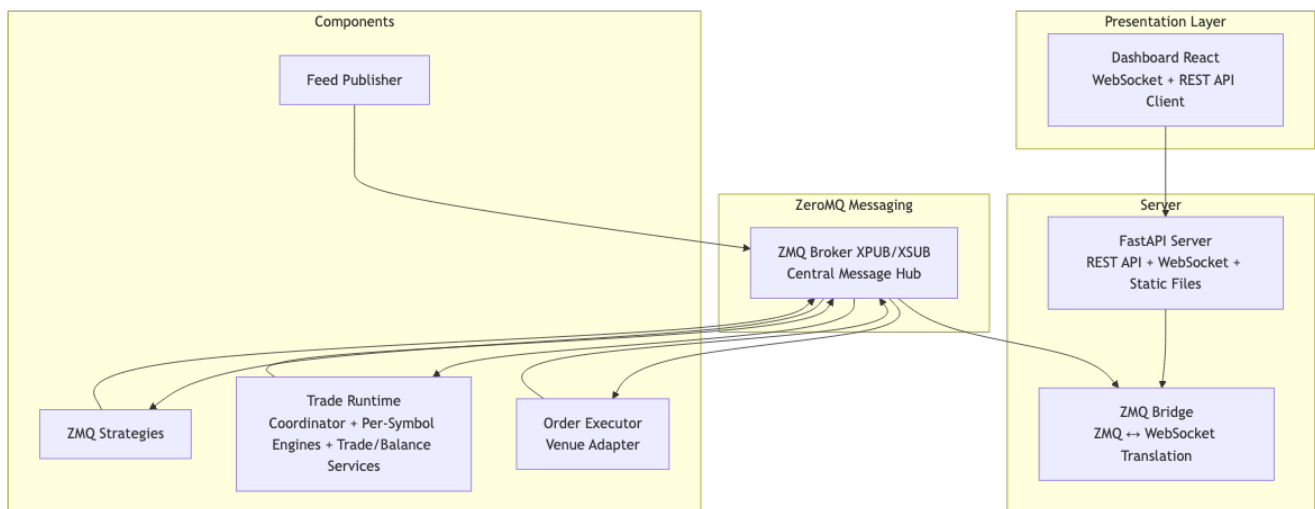
# Start server
snapper server
```

Dashboard available at <http://localhost:8000/>.

Trade Runtime Dispatch

The trade runtime always uses durable, outbox-driven dispatch: commands are persisted as TradeCommand rows and published by the outbox dispatcher. On the accepted/fill paths, executor VenueEvent persistence is fail-closed — a failed write raises before the corresponding orders.events.* publish.

System Overview



CLI Commands

Server and Infrastructure

```
snapper server          # Start FastAPI server
snapper broker          # Start ZMQ broker
snapper trade-zmq      # Trade runtime / coordinator (pass --instance-id +
--instance-count for N>=2, see docs/operations.md)
snapper executor       # Order executor
snapper feed           # Market data publisher
```

Database

```
snapper db-init          # Initialize schema
snapper db-upgrade       # Alembic migrations
snapper db-downgrade     # Rollback migrations
```

Users

```
snapper init-admin       # Create admin user
snapper list-users       # List users
snapper reset-password   # Reset password
```

Local AI delegate PAT (for @mateusz-klatt/snapper-mcp bridge)

After make dev-backend is running and the seed has provisioned the admin user (per dev.toml), mint a long-lived AI delegate PAT into a JSON file the bridge consumes via --config=PATH:

```
snapper dev-mint-pat      # writes data/dev-pat.json (mode 0600)
```

Wire your ~/.claude.json mcpServers entry once with the file path:

```
"snapper-local": {
  "command": "node",
  "args": [
    "/path/to/snapper-mcp/dist/index.js",
    "--config=/path/to/snapper/data/dev-pat.json"
  ]
}
```

After rm data/snapper.db && make dev-backend && snapper dev-mint-pat the bridge picks up the refreshed token automatically — no manual edits to ~/.claude.json per DB wipe. CLI flags + environment overrides:

--base-url URL	SNAPPER_DEV_BASE_URL	(default http://localhost:8000)
--admin-username USER	SNAPPER_DEV_ADMIN_USERNAME	(default admin)
--admin-password PASS	SNAPPER_DEV_ADMIN_PASSWORD	(default matches dev.toml)
--output PATH	SNAPPER_DEV_PAT_OUTPUT	(default data/dev-pat.json)
--label LABEL		(default "Local Dev MCP")

The script drives production REST endpoints (POST /api/auth/login + POST /api/ai-delegates) so the seeded delegate exercises the same code path the AI Integration UI uses. Tokens never appear in stderr — HTTP error bodies are passed through a JWT-shape redaction helper before printing.

Market Data

```
snapper update-kraken-symbols          # Sync Kraken symbols
snapper update-polygon-symbols         # Sync Polygon symbols
snapper update-underlyings             # Sync underlying asset mappings from YAML
snapper polygon-backfill-aggregates    # Backfill historical data
snapper archive --day 2024-01-15      # Export candle cache to CSV
snapper archive --from 2024-01-01 --to 2024-01-31 --exchange polygon
snapper archive --table ticks --day 2024-01-15 --exchange polygon
snapper archive --table candles-audit --day 2024-01-15 --closed-only --purge
```

Strategies

Creating your own strategy:

```
from snapper.strategies.base import BaseStrategy, StrategySignal, StrategyConfig
from snapper.strategies.decorators import register_strategy,
create_strategy_process
from snapper.messaging.schemas.data import CandleData

@register_strategy("MyStrategy")
@create_strategy_process(
    process_name="my_strategy",
    default_config={
        "name": "my_strategy",
        "inputs": ["market.kraken.BTC-USD.candles.1h"],
        "outputs": ["BTC-USD"],
        "exchange": "paper",
        "params": {"threshold": 0.5},
    },
)
class MyStrategy(BaseStrategy):
    def __init__(self, config: StrategyConfig) -> None:
        super().__init__(config)
        self.threshold = self.params.get("threshold", 0.5)

    async def on_candle(self, instrument: str, candle: CandleData) ->
StrategySignal | None:
        if some_condition:
            return StrategySignal(
                instrument=instrument,
                side="buy",
                strength=1.0,
                price=candle.close,
                reason="Condition met",
            )
        return None
```

Development

Quality Gates

```
make check-all    # Full quality gate (backend + frontend + tests + 100%  
coverage)  
make fix-all      # Automatic formatting and linting fixes
```

Individual Steps

```
make fmt          # Check formatting  
make lint         # Linting (ruff)  
make typecheck   # Type checking (mypy)  
make test        # Unit tests  
make cov         # Tests with coverage (100% required)
```

Frontend

```
make ui-dev       # Vite development server  
make ui-build     # Production build  
make ui-lint      # ESLint  
make ui-format    # Prettier
```

Docker

```
make docker-build-dev    # Build dev image  
make docker-build-prod   # Build production image  
make docker-run          # Run container  
make docker-stop         # Stop container
```

Or with docker compose:

```
docker compose up -d
```

Documentation

Detailed documentation in [docs/](#) directory:

- [Architecture](#) — System structure and layers
- [Configuration](#) — Environment variables and settings
- [CLI](#) — Full command documentation
- [Strategies](#) — Creating trading strategies

- [API](#) — REST API and WebSocket
- [Messaging](#) — ZeroMQ architecture
- [Development](#) — Developer guidelines

License

MIT License — see [LICENSE](#) file.

AI Integration

Snapper exposes a **Model Context Protocol (MCP)** endpoint at `/api/mcp` so Claude Desktop, Cursor, Windsurf, or a plain curl client can authenticate with a long-lived AI delegate access JWT and call a narrow set of trading + market-data tools. The surface is vendor-neutral — no Anthropic or OpenAI SDK is bundled in Snapper core.

This doc covers:

1. [Feature flag](#)
 2. [Creating an AI delegate](#)
 3. [Token model](#)
 4. [Client configuration examples](#)
 5. [Available tools](#)
 6. [Safety caps](#)
 7. [Kill switch + deactivation](#)
 8. [Rate limits](#)
 9. [Error catalog](#)
 10. [Security model](#)
-

Feature flag

The `/api/mcp` sub-app is always mounted and gated by the `ai_integration_enabled` database setting.

1. The flag **defaults to true** — a fresh install exposes the MCP endpoint and the AI Integration navigation entry with no manual setup. Operators who need to disable the feature flip the setting to false via the Settings UI (operator+) or directly:

```
curl -X POST http://localhost:8000/api/settings/ai_integration_enabled/set \
-H "Authorization: Bearer <operator-jwt>" \
-H "Content-Type: application/json" \
-d '{"session_id":"cli","sequence_id":1,"public_id":"$(uuidgen)","timestamp":"2026-04-20T00:00:00Z","payload":{"value":"false","category":"system"}}'
```

2. The frontend reads `GET /api/settings/features` on mount (no auth required) and surfaces the "AI Integration" navigation entry whenever the flag is on. The MCP endpoint returns 503 `feature_disabled` only when the flag is explicitly set to false.
 3. Whether the flag is on or off, the MCP endpoint requires every request to carry a valid `Authorization: Bearer <jwt>` header. Anonymous requests receive 401 `missing_bearer_token`.
-

Creating an AI delegate

An **AI delegate** is a dedicated AI_DELEGATE-role user an operator mints per MCP client. Delegates:

- Cannot log in via the web UI password form (the placeholder password hash is opaque to humans).
- Cannot see other delegates, operators, or wallets — they're scoped to the owning operator's visible wallet set.
- Carry per-delegate safety caps independent of the operating operator's caps.

Create one via POST /api/ai-delegates:

```
curl -X POST http://localhost:8000/api/ai-delegates \
-H "Authorization: Bearer <operator-jwt>" \
-H "Content-Type: application/json" \
-H "X-CSRF-Token: <csrf>" \
-d '{
  "session_id": "cli",
  "sequence_id": 1,
  "public_id": "'$(uuidgen)'",
  "timestamp": "2026-04-20T00:00:00Z",
  "payload": {
    "label": "Claude Desktop",
    "caps": {
      "max_open_orders": 3,
      "max_daily_notional_usd": 1000.0,
      "max_cancels_per_minute": 10
    }
  }
}'
```

The response is **one-shot**. Copy the tokens out of the HTTP session immediately — Snapper never re-serves them:

```
{
  "type": "delegate_created_response",
  "payload": {
    "delegate": {
      "public_id": "019da9e...",
      "username": "ai-clausedesktop-a1b2c3",
      "label": "clausedesktop",
      "created_by_user_public_id": "<operator-id>",
      "created_at": "2026-04-20T00:00:00Z",
      "is_active": true,
      "caps": { "max_open_orders": 3, "max_daily_notional_usd": 1000.0 }
    },
    "access_token": "<jwt-with-10y-exp>",
    "expires_in": 315360000
  }
}
```

Other endpoints on `/api/ai-delegates`:

- GET `/api/ai-delegates` — list the caller's delegates (no tokens re-served).
 - GET `/api/ai-delegates/{id}` — single delegate detail.
 - PATCH `/api/ai-delegates/{id}` — update caps (SCD2 close+insert). label/username are immutable post-mint.
 - POST `/api/ai-delegates/{id}/deactivate` — kill switch. Publishes `admin.user_deactivated` on the bus; every Snapper instance disconnects matching WebSocket sessions and evicts the token from every LRU within one bus round-trip.
-

Token model

Each AI delegate mints a single long-lived (~10-year) access JWT. The same token authenticates both the proxy MCP server and the optional push-wakeup watch monitor. Revocation works instantly: POST `/api/ai-delegates/{id}/deactivate` flips `users.is_active=False`, publishes `admin.user_deactivated` on the bus, and every Snapper instance evicts the delegate's `user_active_tokens` row from the verify-cache within one bus round-trip. The 10-year exp is a ceiling, not a commitment.

Client configuration examples

Claude Code plugin (recommended, since snapper-mcp v0.2.1)

In any Claude Code session:

```
/plugin marketplace add mateusz-klatt/snapper-mcp
/plugin install snapper-mcp@mateusz-klatt-snapper-mcp
/reload-plugins
```

Claude Code prompts for two required values plus one optional value at install time:

- **Snapper API URL** -- your backend's `/api/mcp` endpoint. The `@mateusz-klatt/snapper-mcp` bridge accepts the value with or without a trailing slash and normalizes it before connecting.
- **Access token** -- the `access_token` from the `delegate_created` response (or paste from the Settings -> AI Delegates config-snippet generator).
- **Refresh token** (*optional*) -- the `refresh_token` for rotating delegates; **leave blank** for long-lived PAT delegates.

Run `/mcp list` to confirm the snapper server is connected. The plugin pins the runtime to a specific `@mateusz-klatt/snapper-mcp` version (`v0.2.1 -> @0.2.0`, `v0.2.2 -> @0.2.2`, etc.) so future runtime publishes do not silently upgrade existing installs; `/plugin update snapper-mcp` opts in. Sensitive values land in the OS keychain (with `~/ .claude/credentials.json` fallback) -- never in `settings.json` or the plugin manifest.

Claude Desktop

Add to `~/Library/Application Support/Claude/claude_desktop_config.json`:

```
{
  "mcpServers": {
    "snapper": {
      "command": "npx",
      "args": ["-y", "@mateusz-klatt/snapper-mcp"],
      "env": {
        "SNAPPER_BASE_URL": "https://snapper.example.com/api/mcp",
        "SNAPPER_ACCESS_TOKEN": "<copied-from-create-response>"
      }
    }
  }
}
```

The `@mateusz-klatt/snapper-mcp` npm wrapper presents the access token as a Bearer header on every request. On 401 the bridge surfaces the auth failure to the MCP host once per session; recovery is via recreating the AI delegate in Snapper.

Cursor

In `~/ .cursor/config.json`:

```
{
  "mcpServers": {
    "snapper": {
      "url": "https://snapper.example.com/api/mcp/",
      "headers": {
        "Authorization": "Bearer <access-token>"
      }
    }
  }
}
```

Cursor currently requires manual token rotation when the access token expires (15 minutes by default). Use the refresh token via `POST /api/auth/refresh?return_tokens=true` with `Authorization: Bearer <refresh-jwt>` to mint a fresh pair.

Windsurf

`~/ .codeium/windsurf/mcp_config.json`:

```
{
  "mcpServers": {
    "snapper": {
      "serverUrl": "https://snapper.example.com/api/mcp/",
      "auth": { "type": "bearer", "token": "<access-token>" }
    }
  }
}
```

curl (diagnostics)

Quick reachability check:

```
# Feature flag
curl http://localhost:8000/api/settings/features

# Hello-world MCP initialise
curl -X POST http://localhost:8000/api/mcp/ \
  -H "Authorization: Bearer <access-token>" \
  -H "Content-Type: application/json" \
  -H "Accept: application/json, text/event-stream" \
  -d '{"jsonrpc":"2.0","id":1,"method":"initialize","params":
{"protocolVersion":"2025-03-26","capabilities":{},"clientInfo":
{"name":"curl","version":"8"}}}'
```

Available tools

Read-only tools surface the delegate's order book, position state, and venue market data; write tools (submit_manual_order, cancel_order, submit_ai_review_decision) gate on the caps enforcer + per-tool permissions.

- **list_instruments(exchange: str)** — returns sorted instrument symbols visible to the delegate's operator for the given exchange. Read-only; requires READ_MARKET_DATA permission (AI_DELEGATE role satisfies).
- **list_orders(wallet_public_id?, status?, exchange?, instrument?, limit=50, offset=0)** — paged read of the delegate's order history within accessible wallets. Requires READ_ORDERS. Surfaces order_not_found for inaccessible wallets (anti-enumeration).
- **get_order_status(command_public_id: str)** — single-order lookup keyed by the trade-command public id returned from submit_manual_order. Requires READ_ORDERS.
- **list_positions(wallet_public_id?, exchange?, instrument?)** — active positions across the caller's accessible wallets. Requires READ_POSITIONS.
- **get_position_cycle(cycle_public_id: str)** — full open→close audit trail for a position cycle. Requires READ_POSITIONS.

- **get_ohlcv(exchange, instrument, timeframe, since?, until?, limit=200)** — OHLCV candles for a venue + instrument. Range mode (both since + until ISO 8601 UTC) returns chronological candles in the closed window; latest-as-of mode (both omitted) returns the most recent limit candles in DESC order. Allowed timeframes: 1m, 5m, 15m, 1h, 4h, 1d. Requires READ_MARKET_DATA; market data is public so no wallet scope applies.
- **list_recent_signals(since, instrument?, strategy?, exchange?, wallet_public_id?, limit=50)** — strategy signals fired after a watermark, ordered by fired_at DESC. since is REQUIRED ISO 8601 UTC. Requires READ_SIGNALS; surfaces signal_not_found on wallet scope violation (anti-enumeration).
- **submit_manual_order(exchange, instrument, side, quantity, price?, mode?)** — enqueues a trade command under the delegate's user_public_id with source_surface='mcp' + the caps check from TradingCapsEnforcer.guard. Fails closed on any cap violation.
- **cancel_order(plan_public_id, idempotency_key)** — cancels an active execution plan via the same PlansCancelService REST goes through. Idempotent: same idempotency_key on retry returns the current plan state without re-executing.

The tool catalog is discoverable via the MCP tools/list JSON-RPC method:

```
curl -X POST http://localhost:8000/api/mcp/ \
-H "Authorization: Bearer <access-token>" \
-H "Content-Type: application/json" \
-H "Accept: application/json, text/event-stream" \
-d '{"jsonrpc":"2.0","id":2,"method":"tools/list","params":{}}'
```

Safety caps

Every delegate has its own user_trading_caps row. All fields are optional; null means "unbounded on this axis".

- max_order_quantity_per_instrument — JSON dict {instrument: max_qty} OR a scalar applied to every instrument.
- max_open_orders — all-time count of the delegate's in-flight commands (statuses in created/dispatched/acked/accepted/ partially_filled).
- max_daily_notional_usd — rolling 24h sum of submit_quantity * submit_price_usd across non-rejected commands (submit-time commitment basis; partial fills don't change accounting).
- max_cancels_per_minute — sliding-window cancel rate.

Caps are enforced at every trade-command insert site via the TradingCapsEnforcer.guard() surface. MCP submit_manual_order is the only tool that triggers them.

Update caps via PATCH /api/ai-delegates/{id} — the write is SCD2 close+insert, so cap history is auditable.

Kill switch + deactivation

POST /api/ai-delegates/{id}/deactivate runs the same code path as operator deactivation:

1. SCD2 close+insert on the users row flipping is_active=False.
2. TokenManager.revoke_user_sessions loads every user_active_tokens.jti for the delegate, flips revoked_at=NOW() in one SQL UPDATE, and seeds the in-memory JTI blacklist with a 10-second grace period.
3. UserService.deactivate_user publishes admin.user_deactivated on the bus (sole publisher).
4. Every Snapper instance's WebSocketAuthManager subscribes to the topic and closes matching WebSocket connections with code 4003.
5. Every Snapper instance's TokenManager subscribes to the topic and evicts matching verify-cache entries via invalidate_user_cache.

Latency: same-instance kill is immediate (JTI blacklist consulted before LRU in verify_token_with_db); cross-instance kill is bounded by one bus-message round-trip (sub-second on local ZMQ) instead of the 30-second LRU TTL.

In-flight MCP tool handlers are **not** force-cancelled. The guarantee is narrowly "the NEXT MCP request is rejected".

Rate limits

REST rate limits apply globally via RestCallTracker — observed via GET /api/metrics/rest-rate. MCP-specific rate limits live on top of the per-delegate max_cancels_per_minute cap. Bursts are served best- effort; sustained abuse over published exchange limits (Walutomat 20 req/s, Kraken 15 req/s, Polygon 5/min) surfaces as rate_limited warnings at 80/95% utilisation.

Error catalog

MCP responses use standard HTTP status codes + a Snapper-specific error_code in the JSON body. Clients should branch on error_code, not on status text.

Status	error_code	When it fires	Client action
503	feature_disabled		

Status	error_code	When it fires	Client action
		Flag off OR settings service not ready	Show "AI Integration disabled" banner
503	mcp_unavailable	Repository dep unavailable (lifespan not ready)	Retry with backoff
401	missing_bearer_token	No Authorization: Bearer ... header	Prompt user to authenticate
401	invalid_bearer_token	JWT signature/expiry/blacklist/inventory failure	Call POST /api/auth/refresh; fail → re-login
401	user_deactivated	Owner account deactivated	Prompt re-login; don't auto-refresh
401	Refresh token redeemed	Replay of a spent refresh JWT	Re-login
401	Account deactivated	Session cookie flow	Re-login
403	wallet_out_of_scope	Tool targets a wallet outside the caller's scope	Pick a wallet the caller still has a live grant on
403	operator_out_of_scope	Tool targets an operator not in the caller's JWT	Pick an operator from the caller's authenticated set

Delegate CRUD:

Status	Detail	When it fires
401	Requires populated user_public_id	Principal has blank user_public_id (legacy token rollout)
403	require_role(OPERATOR)	AI_DELEGATE or VIEWER trying to manage delegates
404	Delegate not found	Unknown ID OR cross-tenant (no existence leak)
409	Could not derive a unique username ...	Label slug collides 8+ times (pathological)
422	Operator '<id>' is not in ...	Caller picked operator_public_id outside their claim set
422	Caller has no primary operator ...	No explicit operator and no primary → binding is ambiguous

Security model

Token lifetime

- Access tokens live **15 minutes** (configurable via `auth_access_token_expire_minutes`).
- Refresh tokens live **7 days** (30 days with `remember_me=true`).
- Refresh rotation is **atomic**: one DB transaction revokes the old refresh JTI AND persists the new access+refresh pair. Replay of a spent refresh JWT fails with 401 Refresh token already redeemed — the transaction rolls back cleanly.

Inventory-based revocation

`user_active_tokens` is the ground truth for "is this JWT still valid?". Every mint persists a row; every verify consults it via a 30-second LRU. Deactivation flips `revoked_at=NOW()` in one UPDATE and fans out a bus event so every cross-instance LRU evicts the matching entry on receipt.

No secrets leaked at rest

Delegate password hashes are bcrypt-derived from 32 bytes of cryptographically random data the operator never sees. An attacker with DB read rights cannot brute-force the hash.

Single-publisher invariant

`UserService.deactivate_user` is the SOLE publisher of `admin.user_deactivated` across the entire process. `TokenManager` and `WebSocketAuthManager` are pure subscribers — they evict + close on receipt but never emit the event themselves. This holds for operator deactivation AND delegate deactivation (the route reuses `UserService.deactivate_user`).

MCP transport = Streamable HTTP

The MCP endpoint uses Model Context Protocol's Streamable HTTP transport. No cookies, no CSRF on MCP calls (bearer header bypass). Every tool call opens a sub-request inside the same bearer-authenticated HTTP connection.

Related reading

- [docs/architecture.md](#) — repository + SCD2 + bus layering.

- docs/operations.md — lifespan order, token cleanup loop, shard partitioning.
- docs/api.md — full REST surface.

API

Snapper provides a REST API and WebSocket interface for platform interaction. The API requires JWT authentication via HTTP-only cookies. Auth bootstrap endpoints under `/api/auth` are exempt, but other mutating requests (POST) require a valid X-CSRF-Token header.

Authentication

Authentication uses HTTP-only cookies. After login, the server sets `access_token`, `refresh_token`, and `csrf_token` cookies automatically.

Roles and Permissions

Role	Access
viewer	Read-only market data, orders, positions, strategies, system status
operator	Viewer permissions plus trade execution, process management
admin	Full access including user management, system configuration, wallet/credential management, scope grant management, operator impersonation

Multi-tenant permissions (ADMIN only): `read:wallet_credentials`, `manage:wallet_credentials`, `manage:scope_grants`, `impersonate:operator`.

POST `/api/auth/login`

Authenticate and create a session. Sets `access_token`, `refresh_token`, and `csrf_token` cookies on the response.

Request:

```
POST /api/auth/login
Content-Type: application/json

{
  "username": "admin",
  "password": "password123",
  "remember_me": false
}
```

Response (200):

```
{
  "message": "Login successful",
  "expires_in": 900,
  "user": {
    "username": "admin",
    "email": "admin@example.com",
    "role": "admin",
    "is_active": true,
    "created_at": "2026-01-10T08:00:00Z"
  }
}
```

Cookies set:

Cookie	HttpOnly	Secure	SameSite	Path	Max-Age	Description
access_token	Yes	Yes (prod)	Lax/Strict	/	session	JWT access token (15 min)
refresh_token	Yes	Yes (prod)	Lax/Strict	/api/auth	7 days	JWT refresh token
csrf_token	No	Yes (prod)	Lax/Strict	/	session	CSRF protection token

POST /api/auth/refresh

Refresh session tokens. The refresh_token cookie is sent automatically by the browser. Returns new tokens in cookies plus a WebSocket authentication token in the response body. Long-running clients that already hold a valid access token and only need a fresh ws_token should call [POST /api/auth/ws_token](#) instead — that route does not rotate the refresh-token pair.

Request:

```
POST /api/auth/refresh
```

Response (200):

```
{
  "message": "session refreshed",
  "ws_token": "eyJhbGciOi...",
  "ws_token_exp": "2026-01-18T12:30:00Z",
  "csrf_token": "abc123def456...",
  "user": {
    "username": "admin",
    "email": "admin@example.com",
    "role": "admin",
    "is_active": true,
  }
}
```

```
    "created_at": "2026-01-10T08:00:00Z"
  }
}
```

POST /api/auth/ws_token

Mint a one-shot WebSocket authentication token without rotating the caller's refresh-token pair. Authenticates via the access bearer (header or cookie) and returns a fresh ws_token bound to the access JWT's session. Per-source-IP rate-limited so a reconnect storm cannot exhaust the token store.

Use this route when a long-running client (e.g. a push-wakeup monitor) needs to mint successive ws_tokens on its own cadence without rotating the refresh-token pair shared with sibling processes.

Request:

```
POST /api/auth/ws_token
Authorization: Bearer <access_token>
```

Response (200):

```
{
  "type": "ws_token_response",
  "payload": {
    "type": "ws_token",
    "message": "ws_token issued",
    "ws_token": "eyJhbGciOi...",
    "ws_token_exp": "2026-01-18T12:30:00Z",
    "expires_in": 900
  }
}
```

expires_in mirrors ws_token_exp as relative seconds for clients that prefer relative-deadline math.

Errors:

- 401 Unauthorized — access bearer is absent, expired, or invalid.
- 429 Too Many Requests — per-source-IP minute budget exhausted.

POST /api/auth/logout

Logout and invalidate the current session. Clears all authentication cookies and blacklists tokens.

Request:

```
POST /api/auth/logout
```

Response (200):

```
{
  "message": "Logged out successfully"
}
```

GET /api/auth/me

Get the currently authenticated user's profile.

Request:

```
GET /api/auth/me
```

Response (200):

```
{
  "username": "admin",
  "email": "admin@example.com",
  "role": "admin",
  "is_active": true,
  "created_at": "2026-01-10T08:00:00Z",
  "operator_public_ids": ["019d6ca4-..."],
  "primary_operator_public_id": "019d6ca4-..."
}
```

The `operator_public_ids` and `primary_operator_public_id` fields are populated from `user_operator_memberships` (ADMIN receives every active operator; OPERATOR/VIEWER receive only their explicit memberships). These fields power the frontend `OperatorPicker` without a second round trip.

CSRF Protection

The `csrf_token` cookie is readable by JavaScript (not `HttpOnly`). For authenticated mutating requests outside `/api/auth/login`, `/api/auth/refresh`, and `/api/auth/logout`, include its value as a header:

```
X-CSRF-Token: <value from csrf_token cookie>
```

New CSRF tokens are issued on login and refresh. There is no separate endpoint for obtaining CSRF tokens.

REST Endpoints

REST endpoints return the same Data schemas used by WebSocket messages (OrderData, SignalData, ExecutionData, PositionData, CandleData from `messaging.schemas.data`). This means the wire format is identical whether data arrives via REST or the WebSocket feed.

REST = Bulk WS

REST responses are JSON arrays of payload items. Each item carries its own per-item provenance (`public_id`, `session_id`, `sequence_id`) — there is no wrapper-level `public_id` and no custom HTTP headers for provenance. The shape of each array element is identical to what the WebSocket feed delivers for the same data type.

Provenance on Reads vs. Mutations

- **GET reads** — No client provenance is expected. The server records the request in the telemetry table for observability but does not stamp provenance onto the response items beyond what was stored at write time.
- **Mutations (POST)** — All mutations use command-style POST with a `PayloadRequest` envelope carrying provenance (`public_id`, `session_id`, `sequence_id`, `timestamp`) and domain intent in payload. The server-side `ClientProvenanceMiddleware` extracts these fields, emits a structured info log, and runs a per-session `GapDetector` to warn on sequence gaps. Gaps are logged as warnings but never reject requests.

ClientProvenanceMiddleware

`ClientProvenanceMiddleware` is an ASGI middleware that processes every mutation request:

1. Intercepts the request body transparently (replays chunks to downstream handlers).
2. Extracts `session_id`, `sequence_id`, and `public_id` from the JSON body when present.
3. Emits a structured log with `client_session_id`, `client_sequence_id`, `client_public_id`, and the request path.
4. Runs a per-session `GapDetector` to detect sequence gaps from the client.
5. Records the mutation in the control table via a `finally` block, so the row is persisted whether the request succeeded or failed.

The control recording is non-blocking: any DB failure is logged and swallowed so the response already sent to the client is never invalidated. The control row includes the redacted request payload, HTTP method, path, outcome (ok, error, or exception), and server-side provenance (`session_id` and `sequence_id` from the middleware's own `SequenceTracker`).

All data endpoints support bitemporal querying via the optional `as_of` parameter (UTC datetime). When provided, the query returns data as it was known at that point in time. When omitted, the current time is used.

GET /api/health

Public health check endpoint. No authentication required.

status reflects the health of enabled long-running CORE processes: "healthy" when all are running, "error" if any are missing. In SERVER_API_ONLY mode the status is always "healthy" (no processes are started by design).

Response (200):

```
{
  "type": "health_check",
  "status": "healthy",
  "timestamp": "2026-01-18T12:00:00Z",
  "version": "0.1.0",
  "connections": {
    "type": "connection_stats",
    "active_connections": 5,
    "zmq_subscribers": 12,
    "subscriber_tasks": 12,
    "active_topics": 8,
    "active_clients": 3
  },
  "topics": {
    "type": "health_topics",
    "available": 7,
    "active": 3
  },
  "gap_detection": {
    "type": "gap_detection_stats",
    "bridge": {
      "type": "gap_stats",
      "gaps_detected": 0,
      "session_resets": 0,
      "duplicates": 0,
      "mid_stream_joins": 0,
      "rejected_unstamped": 0
    },
    "rest_clients": {}
  }
}
```

The `gap_detection` field provides observability into sequence gap detection across the ZMQ bridge and per-session REST client detectors.

GET /api/candles

Fetch OHLCV candlestick data. Requires `read:market_data` permission.

Request:

```
GET /api/candles?instrument=BTC-USD&exchange=kraken&timeframe=1h&limit=100
X-CSRF-Token: <csrf_token>
```

Parameters:

Parameter	Type	Required	Description
instrument	string	yes	Instrument symbol (e.g., BTC-USD)
exchange	string	yes	Exchange name (kraken, walumat, polygon)
timeframe	string	yes	Candle timeframe (e.g., 1m, 5m, 15m, 1h, 4h, 1d)
limit	int	no	Number of candles, max 1000 (default 100)
as_of	datetime	no	Point-in-time query, UTC (default: current time)

Returns 204 No Content if the instrument is not found.

Response (200):

```
[
  {
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "type": "candle",
    "timestamp": "2026-01-18T12:00:00Z",
    "instrument": "BTC-USD",
    "exchange": "kraken",
    "timeframe": "1h",
    "open_at": "2026-01-18T11:00:00Z",
    "open": 42000.0,
    "high": 42500.0,
    "low": 41800.0,
    "close": 42300.0,
    "volume": 1234.56,
    "vwap": 42150.0,
    "trades": 5678
  }
]
```

GET /api/orders

Fetch orders with optional filtering. Requires read:orders permission.

Request:

```
GET /api/orders?symbol=BTC-USD&exchange=kraken&limit=100&offset=0
X-CSRF-Token: <csrf_token>
```

Parameters:

Parameter	Type	Required	Description
symbol	string	no	Filter by instrument symbol
exchange	string	no	Filter by exchange (paper, kraken, walutomat)
limit	int	no	Number of orders, 1-1000 (default 100)
offset	int	no	Number of orders to skip (default 0)
as_of	datetime	no	Point-in-time query, UTC (default: current time)
operator_public_id	string	no	Scope to a single operator (403 if foreign)
wallet_public_id	string	no	Scope to a single wallet (403 if inaccessible)

Response (200):

```
[
  {
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "type": "order",
    "timestamp": "2026-01-18T12:00:00Z",
    "instrument": "BTC-USD",
    "exchange": "kraken",
    "client_order_id": "signal-a1b2c3d4",
    "exchange_order_id": "KRAKEN-456",
    "created_at": "2026-01-18T12:00:00Z",
    "updated_at": "2026-01-18T12:01:00Z",
    "side": "buy",
    "order_type": "limit",
    "price": 42000.0,
    "size": 0.1,
    "filled_size": 0.1,
    "average_price": 42000.0,
    "status": "filled",
    "time_in_force": "GTC",
    "mode": "live",
    "error": null
  }
]
```

POST /api/orders

Create a manual order via a manual_once execution plan. Requires create:orders permission.

Request:

```
POST /api/orders
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
```

```

"type": "create_order_command",
"session_id": "ui",
"sequence_id": 0,
"public_id": "<uuid7>",
"timestamp": "2026-04-10T12:00:00Z",
"payload": {
  "instrument": "BTC-USD",
  "instrument_public_id": "<uuid>",
  "exchange": "kraken",
  "mode": "live",
  "side": "buy",
  "order_type": "limit",
  "quantity": 0.5,
  "price": 50000.0,
  "wallet_public_id": "<uuid>",
  "idempotency_key": "<uuid7>"
}
}

```

Payload fields:

Field	Type	Required	Description
instrument	string	yes	Native symbol (e.g., BTC-USD)
instrument_public_id	string	yes	Instrument UUID
exchange	string	yes	Exchange name
mode	string	no	live (default) or paper
side	string	yes	buy or sell
order_type	string	yes	market, limit, stop, stop_limit
quantity	float	yes	Order size (must be > 0)
price	float	cond	Required for limit and stop_limit
stop_price	float	cond	Required for stop and stop_limit
wallet_public_id	string	yes	Target wallet UUID
operator_public_id	string	no	Operator identity
idempotency_key	string	no	Dedup key (409 on duplicate)

Response (200): ExecutionPlanResponse envelope with plan details.

Errors: 403 (disabled/forbidden), 409 (idempotency conflict), 422 (validation).

POST /api/orders/{plan_public_id}/cancel

Cancel an active execution plan by its plan public id. Requires cancel:orders permission and caller access to the plan's wallet.

The route transitions the plan to `cancel_requested`, hydrates the venue-assigned `exchange_order_id` from the active orders row if available, and inserts a cancel `TradeCommand` so the outbox dispatcher can publish `OrderCancelData` on the `orders.commands.{ex}.{instr}.cancel` topic. If the cancel `TradeCommand` insert fails the plan is transitioned to `failed` with `last_error`, and HTTP 500 is returned — `PlanExecutorService` will re-emit the cancel on the next startup (idempotent via `has_pending_cancel_command`).

Request:

```
POST /api/orders/<plan_public_id>/cancel
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
  "type": "cancel_order_command",
  "session_id": "ui",
  "sequence_id": 0,
  "public_id": "<uuid7>",
  "timestamp": "2026-04-10T12:01:00Z",
  "payload": {
    "reason": "changed mind"
  }
}
```

Response (200): `ExecutionPlanResponse` with status `cancel_requested`.

Errors: 403 (wallet not accessible), 404 (not found), 409 (already terminal or concurrent change), 500 (cancel command insert failed).

POST /api/orders/by-client-order-id/{client_order_id}/cancel

UI convenience route that cancels by the child order's `client_order_id` (the value displayed on the Orders table). Resolves the owning plan via the `trade_commands` table and delegates to the shared cancel flow described above.

Requires `cancel:orders` permission. Same response shape and error codes as the `plan-id` route, plus 404 when no plan-linked `trade_commands` row is found for the given `client_order_id`.

Request:

```
POST /api/orders/by-client-order-id/<client_order_id>/cancel
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
  "type": "cancel_order_command",
  "session_id": "ui",
  "sequence_id": 0,
```

```
{
  "public_id": "<uuid7>",
  "timestamp": "2026-04-11T12:00:00Z",
  "payload": {
    "reason": "cancelled from Orders table"
  }
}
```

Response (200): ExecutionPlanResponse with status cancel_requested.

Errors: 403 (wallet not accessible), 404 (no linked plan), 409 (already terminal), 500 (cancel command insert failed).

GET /api/instrument-capabilities

Fetch instrument order capability matrix. Requires read:market_data.

Parameters:

Parameter	Type	Required	Description
exchange	string	no	Filter by exchange
instrument_public_id	string	no	Filter by instrument
as_of	datetime	no	Point-in-time query

Response (200): InstrumentCapabilityListResponse with capability flags.

GET /api/venue-fee-schedules

Fetch venue fee schedules. Requires read:market_data.

Parameters:

Parameter	Type	Required	Description
exchange	string	no	Filter by exchange
as_of	datetime	no	Point-in-time query

Response (200): VenueFeeScheduleListResponse with fee tiers.

GET /api/signals

Fetch trading signals with optional filtering. Requires read:market_data permission.

Request:

```
GET /api/signals?instrument=BTC-USD&strategy=rsi_btc_1h&exchange=paper&hours=24&limit=100
X-CSRF-Token: <csrf_token>
```

Parameters:

Parameter	Type	Required	Description
instrument	string	no	Filter by instrument symbol
strategy	string	no	Filter by strategy name
exchange	string	no	Filter by exchange (paper, kraken, walutomat)
hours	int	no	Hours of history, max 168 (default 24)
limit	int	no	Number of signals, max 1000 (default 100)
as_of	datetime	no	Point-in-time query, UTC (default: current time)

Response (200):

```
[
  {
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "type": "signal",
    "timestamp": "2026-01-18T12:00:00Z",
    "instrument": "BTC-USD",
    "exchange": "paper",
    "side": "buy",
    "strength": 0.85,
    "reason": "RSI 28.5 <= 30",
    "strategy_name": "rsi_btc_1h",
    "price": 42000.0,
    "fired_at": "2026-01-18T12:00:00Z"
  }
]
```

GET /api/executions

Fetch order fills/executions. Requires read:orders permission.

Request:

```
GET /api/executions?limit=100
X-CSRF-Token: <csrf_token>
```

Parameters:

Parameter	Type	Required	Description
limit	int	no	Number of executions, max 1000 (default 100)

Parameter	Type	Required	Description
as_of	datetime	no	Point-in-time query, UTC (default: current time)

Response (200):

```
[
  {
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "type": "execution",
    "timestamp": "2026-01-18T12:01:00Z",
    "trade_id": "TTRAD-456",
    "exchange_order_id": "KRAKEN-456",
    "client_order_id": "signal-a1b2c3d4",
    "instrument": "BTC-USD",
    "exchange": "kraken",
    "side": "buy",
    "size": 0.1,
    "price": 42000.0,
    "fee": 0.001,
    "fee_asset": "USD",
    "status": "filled",
    "executed_at": "2026-01-18T12:00:59Z"
  }
]
```

GET /api/positions

Fetch current portfolio positions. Requires read:positions permission.

Request:

```
GET /api/positions
X-CSRF-Token: <csrf_token>
```

Parameters:

Parameter	Type	Required	Description
as_of	datetime	no	Point-in-time query, UTC (default: current time)

Response (200):

```
[
  {
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "type": "position",
    "timestamp": "2026-01-18T12:00:00Z",
    "instrument": "BTC-USD",
    "exchange": "kraken",
    "quantity": 0.5,
    "average_price": 41500.0,
    "unrealized_pnl": 250.0,
  }
]
```



```
    "realized_pnl": 100.0,  
    "mode": "live",  
    "position_cycle_public_id": "019e1a2b-4d5e-7f6a-8b9c-0d1e2f3a4b5c"  
  }  
]
```

The position_cycle_public_id field is null when no open position cycle exists for the position (e.g. flat positions or positions without cycle tracking).

POST /api/execution-plans

Create a bracket (SL/TP) execution plan on an open position cycle. Requires create:orders permission.

Request:

```
POST /api/execution-plans  
Content-Type: application/json  
X-CSRF-Token: <csrf_token>
```

Body (BracketCreateCommand envelope):

```
{  
  "type": "create_bracket_command",  
  "public_id": "019e1a2b-0000-7000-8000-000000000001",  
  "session_id": "ui-session-1",  
  "sequence_id": 1,  
  "timestamp": "2026-04-12T18:00:00Z",  
  "payload": {  
    "position_cycle_public_id": "019e1a2b-4d5e-7f6a-8b9c-0d1e2f3a4b5c",  
    "sl_price": 48000.0,  
    "tp_price": 55000.0  
  }  
}
```

At least one of sl_price or tp_price is required.

Response (200): ExecutionPlanResponse wrapping the new armed bracket plan.

Errors: 422 (invalid params, missing capability), 409 (cycle not open, duplicate), 403 (wallet inaccessible), 503 (executor unavailable).

POST /api/execution-plans/{plan_public_id}/cancel

Cancel a bracket execution plan. Armed brackets transition to cancelled directly. Active brackets transition to cancel_requested with cancel TradeCommands emitted. Requires cancel:orders permission.

Request:

```
POST /api/execution-plans/{plan_public_id}/cancel
Content-Type: application/json
X-CSRF-Token: <csrf_token>
```

Response (200): ExecutionPlanResponse wrapping the updated plan.

Errors: 404 (not found), 409 (already terminal or concurrent cancel).

GET /api/execution-plans/{plan_public_id}

Retrieve a single execution plan by public_id. Requires read:orders permission.

Response (200): ExecutionPlanResponse wrapping the plan.

GET /api/execution-plans/{plan_public_id}/decisions

List decision audit rows for an execution plan. Requires read:orders permission.

Response (200): List of decision rows with action, reason, and timestamp.

POST /api/trailing-stops

Create a trailing stop execution plan on an open position cycle. The trailing stop ratchets the stop price as the market moves favorably and triggers a reduce_only market close on breach.

Request body:

- position_cycle_public_id (string, required): Target position cycle.
- trailing_pct (float, required): Trailing distance as percentage ($0 < x < 100$).
- min_lock_pct (float, optional, default 0): Minimum profit % before trailing activates.
- idempotency_key (string, optional): Dedup key for retries.

Permission: create:orders. CSRF token required.

Response (200): ExecutionPlanResponse with plan_type: "trailing_stop", status: "armed".

Response (409): Cycle not open or duplicate trailing stop on same cycle.

Response (422): Invalid params, missing capability, or no open position.

POST /api/trailing-stops/{plan_public_id}/cancel

Cancel a trailing stop. Armed stops transition directly to cancelled. Active stops (with in-flight child orders) transition to cancel_requested.

Permission: cancel:orders. CSRF token required.

GET /api/trailing-stops/{plan_public_id}

Retrieve a trailing stop plan by public_id. Only returns plans with plan_type: "trailing_stop".

Permission: read:orders.

GET /api/trailing-stops/{plan_public_id}/decisions

List decision audit rows for a trailing stop plan.

Permission: read:orders.

GET /api/trailing-stops/by-cycle/{cycle_public_id}

Get live trailing stop state for a position cycle. Returns peak_price and current_stop from the evaluator's in-memory state.

Permission: read:orders.

Response (200): TrailingStopStateResponse with live state, or { "type": "message", "payload": "none" } if no active trailing stop.

Response (404): Position cycle not found.

GET /api/position-cycles/open

List all open position cycles with age information. Admin-only endpoint for diagnosing orphaned cycles (open cycles without a matching trading engine).

Query parameters:

- min_age_hours (float, optional, default 0): Only return cycles open longer than this.

Permission: manage:users (admin only).

Response (200): List of open cycles with cycle_public_id, shard_key, instrument_public_id, exchange, mode, wallet_public_id, direction, max_qty (per-cycle peak, not lifetime), opened_at, and age_hours.

POST /api/position-cycles/close-orphan

Close a specific orphaned position cycle. The operator should verify via GET /open that the cycle is genuinely orphaned before calling this.

Query parameters:

- cycle_public_id (string, required): Public ID of the cycle to close.

Permission: manage:users (admin only). CSRF token required.

Response (200): { "closed_count": 1, "closed_cycle_ids": ["<id>"] }

Response (404): No active open cycle found for the given public_id.

POST /api/position-cycles/sweep-orphans

Bulk-close all open position cycles older than min_age_hours. Default threshold is 72 hours (3 days). Review via GET /open?min_age_hours=72 before running.

Query parameters:

- min_age_hours (float, optional, default 72, minimum 1): Minimum age threshold.

Permission: manage:users (admin only). CSRF token required.

Response (200): { "closed_count": N, "closed_cycle_ids": [...] } }

Response (400): min_age_hours must be at least 1.

GET /api/exchanges

List distinct exchange names from active symbol aliases. Requires read:market_data permission.

Request:

```
GET /api/exchanges
X-CSRF-Token: <csrf_token>
```

Response (200):

```
["kraken", "polygon", "walutomat"]
```

GET /api/exchanges/{exchange}/instruments

List distinct native symbols available on a given exchange. Requires read:market_data permission.

Request:

```
GET /api/exchanges/kraken/instruments
X-CSRF-Token: <csrf_token>
```

Response (200):

```
["BTC-USD", "ETH-USD", "SOL-USD"]
```

GET /api/status

System-wide status including trader process, backtests, and active strategies. Requires read:system_status permission.

Request:

```
GET /api/status
X-CSRF-Token: <csrf_token>
```

Response (200):

```
{
  "trader": {
    "status": "running",
    "pid": null,
    "started_at": null,
    "command": null,
    "exit_code": null,
    "error": null
  },
  "backtests": {},
  "strategies": [
    {
      "strategy_name": "rsi_btc_1h",
      "status": "running",
      "details": {},
      "signals_generated": 42,
      "trades_executed": 5,
      "last_signal": "buy",
      "last_signal_time": "2026-01-18T11:45:00Z",
      "pnl": 150.25,
      "pid": 12345,
      "uptime": "2h 15m"
    }
  ]
}
```

GET /api/ws/stats

WebSocket and ZMQ bridge statistics. Requires read:system_status permission.

Request:

```
GET /api/ws/stats
X-CSRF-Token: <csrf_token>
```

Response (200):

```
{
  "websocket": {
    "active_connections": 5,
    "topic_subscribers": {
```

```

        "market.kraken.BTC-USD.candles.1h": 3,
        "signals.paper.BTC-USD.rsi_btc_1h": 2
    },
    "client_count": 5
},
"zmq_bridge": {
    "active_topics": 12,
    "subscriber_tasks": 12,
    "available_topics": ["admin", "market", "orders.commands",
"orders.events", "signals", "strategy.signals", "system.heartbeats."]
},
"connections": {
    "active_connections": 5,
    "zmq_subscribers": 12,
    "subscriber_tasks": 12,
    "active_topics": 8,
    "active_clients": 5
},
"topics": {
    "market.kraken.BTC-USD.candles.1h": {
        "active_subscribers": 3,
        "received": 1200,
        "forwarded": 1180,
        "throttled": 20,
        "dropped": 0,
        "timeout": 0,
        "errors": 0,
        "invalid_messages": 0,
        "last_message_ts": 1737208800.0,
        "throttle_ms": 100,
        "pattern": "market."
    }
},
"subscriptions": {
    "per_topic": {
        "market.kraken.BTC-USD.candles.1h": 3
    },
    "per_client": {
        "140234567890": ["market.kraken.BTC-USD.candles.1h"]
    }
},
"config": {
    "broker_xpub": "tcp://127.0.0.1:7501",
    "heartbeat_interval_ms": 15000
}
}

```

GET /api/zmq/health

ZMQ bridge health check. Requires read:system_status permission.

Request:

```

GET /api/zmq/health
X-CSRF-Token: <csrf_token>

```

Response (200):

```
{
  "status": "healthy",
  "timestamp": "2026-01-18T12:00:00Z",
  "components": {
    "zmq_context": "ok",
    "websocket_manager": "ok",
    "active_connections": 5
  },
  "config": {
    "available_topics": ["admin", "market", "orders.commands",
"orders.events", "signals", "strategy.signals", "system.heartbeats."]
  },
  "connections": {
    "active_connections": 5,
    "zmq_subscribers": 12,
    "subscriber_tasks": 12,
    "active_topics": 8,
    "active_clients": 3
  },
  "message_stats": {},
  "errors": []
}
```

Process Management

Process endpoints manage background services (feeds, strategies, executors, brokers). Most require manage:processes permission (operator/admin). The summary endpoint requires only read:system_status (viewer+).

GET /api/processes/available

List registered process templates that can be instantiated. Requires manage:processes permission.

Response (200):

```
{
  "processes": [
    {
      "name": "zmq_broker",
      "class_path":
"snapper.messaging.infrastructure.broker.ZmqBrokerProcess",
      "method": "start",
      "description": "ZeroMQ XPUB/XSUB message broker",
      "lifecycle": "long_running",
      "role": "core",
      "tags": ["infrastructure"],
      "parameters_schema": null
    }
  ]
}
```

```
],  
  "count": 1  
}
```

GET /api/processes/configured

List configured process instances with runtime state. Requires `manage:processes` permission.

Response (200):

```
{  
  "processes": [  
    {  
      "name": "zmq_broker",  
      "enabled": true,  
      "running": true,  
      "mode": "thread",  
      "class_path":  
"snapper.messaging.infrastructure.broker.ZmqBrokerProcess",  
      "method": "start",  
      "parameters": {},  
      "note": null,  
      "lifecycle": "long_running",  
      "role": "core",  
      "tags": ["infrastructure"],  
      "parameters_schema": null,  
      "is_one_shot": false,  
      "active_public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b"  
    }  
  ],  
  "count": 1  
}
```

GET /api/processes/summary

Lightweight process category counts for the overview dashboard. Requires `read:system_status` permission.

Response (200):

```
{  
  "feeds": { "running": 2, "total": 3 },  
  "strategies": { "running": 1, "total": 2 },  
  "executors": { "running": 1, "total": 1 },  
  "brokers": { "running": 1, "total": 1 }  
}
```


POST /api/processes

Create a new process configuration from a registered template. Requires manage:processes permission. Returns 201 on success.

Request:

```
POST /api/processes
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
  "name": "kraken_feed_btc",
  "template": "kraken_feed_publisher",
  "enabled": true,
  "mode": "thread",
  "parameters": { "symbols": ["BTC-USD"] },
  "note": "Kraken BTC feed"
}
```

Response (201):

```
{
  "status": "created",
  "process": {
    "name": "kraken_feed_btc",
    "template": "kraken_feed_publisher"
  }
}
```

GET /api/processes/schema/{name}

Get the configuration schema and defaults for a registered process template. Requires manage:processes permission.

Response (200):

```
{
  "name": "kraken_feed_publisher",
  "description": "Kraken market data feed publisher",
  "class_path":
    "snapper.messaging.publishers.kraken.KrakenMarketDataPublisher",
  "method": "start",
  "default_enabled": true,
  "default_mode": "thread",
  "default_parameters": {},
  "lifecycle": "long_running"
}
```

POST /api/processes/{name}/start

Start a configured process. Requires manage:processes permission.

Request:

```
POST /api/processes/zmq_broker/start
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
  "mode": "process"
}
```

All fields are optional. mode overrides the stored execution mode for this run only — it does not persist to Settings. Omitting it uses the stored value.

Response (200):

```
{
  "status": "success",
  "name": "zmq_broker",
  "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
  "message": "Process started"
}
```

POST /api/processes/{name}/stop

Stop a running process. Requires manage:processes permission.

Request:

```
POST /api/processes/zmq_broker/stop
X-CSRF-Token: <csrf_token>
```

Response (200):

```
{
  "status": "success",
  "name": "zmq_broker",
  "message": "Process stopped"
}
```

GET /api/processes/runs

List historical process runs. Requires manage:processes permission.

Parameters:

Parameter	Type	Required	Description
limit	int	no	Number of runs to return (default 50)
name	string	no	Filter by process name

Response (200):

```
{
  "runs": [
    {
      "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
      "process_name": "zmq_broker",
      "status": "succeeded",
      "role": "core",
      "lifecycle": "long_running",
      "parameters": {},
      "result": null,
      "error": null,
      "tags": ["infrastructure"],
      "started_at": "2026-01-18T10:00:00Z",
      "completed_at": "2026-01-18T18:00:00Z"
    }
  ],
  "count": 1
}
```

Strategies

GET /api/strategies

List configured strategy processes with lightweight status. Requires read:strategies permission (viewer+).

Response (200):

```
{
  "strategies": [
    {
      "name": "strategy_rsi_btc_1h",
      "running": true,
      "enabled": true,
      "mode": "process"
    }
  ],
  "count": 1
}
```

Settings

Settings endpoints require configure:system permission (admin only).

GET /api/settings

List all settings, optionally filtered by category.

Parameters:

Parameter	Type	Required	Description
category	string	no	Filter by setting category

Response (200):

```
[
  {
    "key": "polygon_api_key",
    "value": "IvLG...",
    "category": "api",
    "description": "Polygon.io market-data API key",
    "updated_at": "2026-01-15T10:00:00Z",
    "updated_by": "admin"
  }
]
```

Note: per-wallet trading credentials (kraken, walutomat, kraken_futures) are NOT exposed through the settings endpoints. They live in the wallet_credentials table and are currently managed via seed files (proprietary/data/seed/dev.toml / prod.toml — see [Configuration / Wallet Credentials](#)). REST endpoints for runtime wallet credential management land with the upcoming frontend work.

GET /api/settings/categories

List distinct setting category names.

Response (200):

```
{
  "categories": ["exchanges", "strategy", "system"]
}
```

POST /api/settings/{key}/set

Set (create or update) a setting value.

Request:

```
POST /api/settings/polygon_api_key/set
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
  "public_id": "<uuid7>",
  "session_id": "<client-session>",
  "sequence_id": 1,
  "timestamp": "2026-01-18T12:00:00Z",
  "payload": {
    "value": "new-api-key-value",
    "category": "api",
    "description": "Polygon.io market-data API key"
  }
}
```

Payload fields:

Field	Type	Required	Description
value	string	yes	Setting value
category	string	no	Setting category (default: system)
description	string	no	Human-readable description

Response (200):

```
{
  "type": "setting_response",
  "public_id": "<uuid7>",
  "session_id": "<server-session>",
  "sequence_id": 1,
  "timestamp": "2026-01-18T12:00:00Z",
  "payload": {
    "type": "setting_read",
    "key": "polygon_api_key",
    "value": "new-api-key-value",
    "category": "api",
    "description": "Polygon.io market-data API key",
    "updated_at": "2026-01-18T12:00:00Z",
    "updated_by": "admin"
  }
}
```

POST /api/settings/{key}/remove

Soft-delete a setting by key (sets known_to to current time).

Request:

```
POST /api/settings/polygon_api_key/remove
Content-Type: application/json
X-CSRF-Token: <csrf_token>
```

```
{
  "public_id": "<uuid7>",
  "session_id": "<client-session>",
  "sequence_id": 2,
  "timestamp": "2026-01-18T12:00:01Z",
  "payload": {}
}
```

Response (200):

```
{
  "type": "message",
  "public_id": "<uuid7>",
  "session_id": "<server-session>",
  "sequence_id": 2,
  "timestamp": "2026-01-18T12:00:01Z",
  "payload": "Setting 'polygon_api_key' deleted successfully"
}
```

Underlying Assets

GET /api/underlyings

List all underlying assets with instrument counts.

```
GET /api/underlyings?as_of=2026-06-20T12:00:00Z
```

Returns PayloadListResponse with UnderlyingAssetData items (ticker, name, asset_class, sector, instrument_count).

GET /api/underlyings/{ticker}/instruments

List instruments mapped to an underlying asset.

```
GET /api/underlyings/SPX/instruments?relationship_type=derivative
```

Query parameters:

- relationship_type (optional): Filter by exact, derivative, or proxy
- as_of (optional): Point-in-time query timestamp

GET /api/underlyings/{ticker}/front-month

Return the front-month (nearest non-expired) futures contract for an underlying.

```
GET /api/underlyings/SPX/front-month?exchange=kraken_equities&contract_family=ES
```

Query parameters:

- exchange (optional): Filter by exchange
- contract_family (optional): Filter by product root (e.g., ES vs MES)
- as_of (optional): Point-in-time query timestamp

Returns PayloadResponse with FrontMonthData (instrument_public_id, native_symbol, exchange, expiry_at, relationship_type, contract_family). Returns 404 if no active futures contracts exist.

GET /api/underlyings/{ticker}/contracts

List all futures contracts for an underlying asset.

```
GET /api/underlyings/SPX/contracts?include_expired=true&contract_family=ES
```

Query parameters:

- exchange (optional): Filter by exchange
- contract_family (optional): Filter by product root
- include_expired (optional, default false): Include expired contracts
- as_of (optional): Point-in-time query timestamp

Returns PayloadListResponse with ContractData items. Each item includes is_front_month (true for nearest non-expired within same contract family).

Multi-Tenant (Wallets, Operators, Scope Grants, Credentials)

ADMIN principals see the full catalogue; VIEWER and OPERATOR principals see only the subset covered by their operator memberships and active scope grants.

GET /api/wallets

List wallets accessible to the current principal. ADMIN sees all; OPERATOR/VIEWER sees only wallets covered by at least one active scope grant from their operator set.

GET /api/operators

List operators accessible to the current principal. ADMIN sees all; OPERATOR/VIEWER sees only operators in principal.operator_public_ids.

GET /api/scope-grants

List active scope grants on a given wallet. Non-ADMIN callers must have visibility into the target wallet; otherwise 403. Required query parameter: wallet_public_id.

POST /api/scope-grants

Create a new scope grant. Requires manage:scope_grants (ADMIN only). Returns 409 on overlap conflict. Body fields: operator_public_id, wallet_public_id, scope_kind (underlying or instrument), underlying_public_id or instrument_public_id, optional note.

POST /api/scope-grants/handover

Atomic SCD2 close + insert transfer of a scope grant to a different operator. Requires manage:scope_grants. Body: from_grant_public_id, to_operator_public_id, optional reason. Returns 404 if source grant missing, 400 on self-handover, 409 on cross-scope overlap.

POST /api/scope-grants/{grant_public_id}/revoke

Atomic SCD2 close (no replacement row) of an active scope grant + emits admin.scope_revoked on the bus for live AI_DELEGATE subscription revalidation. Requires manage:scope_grants (ADMIN only). Body: optional reason (flows to event payload, NOT to the closed row). Returns 404 if the grant does not exist or is already closed (double-revoke).

POST /api/wallets

Create a new wallet. Requires manage:wallet_credentials (ADMIN only). Returns 409 if (label, is_paper) active-unique index is violated. Body: label, optional description, is_paper (default false).

GET /api/wallets/{wallet_public_id}/credentials

List active credentials on a wallet as summaries (no encrypted payload on the wire). Requires read:wallet_credentials (ADMIN only).

POST /api/wallets/{wallet_public_id}/credentials

Create a new wallet credential. Requires manage:wallet_credentials. Plaintext credential_payload is Fernet-encrypted server-side before DB insert. Body: exchange, credential_type (api_key_secret, rsa_pem, oauth, paper), credential_payload (dict), optional label. Returns 409 if (wallet, exchange) already exists.

POST /api/wallets/{id}/credentials/{cid}/rotate

SCD2 close + insert rotation. Old credential row closed, new row inserted with updated encrypted payload. Requires manage:wallet_credentials. Returns 404 if credential not found. Validates credential_payload against the existing credential type before encrypting.

WebSocket

Connection and Authentication

The WebSocket endpoint is at /api/ws. Authentication is message-based, not query-parameter-based.

Connection flow:

1. Client connects to ws://host:port/api/ws (or wss:// over TLS)
2. Server accepts the connection and validates the origin header
3. Server sends auth_required message
4. Client sends authenticate message with a WebSocket token
5. Server validates the token and sends auth_ok
6. Server sends auth_complete with available topics for the user's role
7. Client can now subscribe to topics

Obtaining a WebSocket token:

Call POST /api/auth/refresh (requires a valid refresh_token cookie). The response includes ws_token and ws_token_exp fields.

Authentication Messages

Server sends after connection:

```
{
  "type": "auth_required",
  "timeout": 10,
  "timestamp": "2026-01-18T12:00:00Z"
}
```

Client sends to authenticate:

```
{
  "type": "authenticate",
  "ws_token": "eyJhbGciOi...",
  "timestamp": "2026-01-18T12:00:00Z"
}
```

Server sends on success:

```
{
  "type": "auth_ok",
  "exp": "2026-01-18T12:30:00Z",
  "timestamp": "2026-01-18T12:00:00Z"
}
```

Server sends session info:

```
{
  "type": "auth_complete",
  "available_topics": [
    "market",
    "orders.commands",
    "orders.events",
    "signals",
    "strategy.signals",
    "system.heartbeats."
  ],
  "user_role": "operator",
  "session_expires_at": "2026-01-18T12:15:00Z",
  "ws_token_exp": "2026-01-18T12:30:00Z",
  "timestamp": "2026-01-18T12:00:00Z"
}
```

available_topics contains allowed topic roots/pattern keys for the user's role. Concrete subscriptions may use those roots as prefixes or full topic strings such as market.kraken.BTC-USD.candles.1h or signals.paper.BTC-USD.rsi_btc_1h.

Server sends on failure:

```
{
  "type": "auth_failed",
  "reason": "Invalid token",
  "timestamp": "2026-01-18T12:00:00Z"
}
```

Reauthentication

Before the WebSocket token expires, the server sends a `reauth_required` message. Clients may also refresh proactively before that deadline. In both cases they obtain a new `ws_token` via `POST /api/auth/refresh` and send it as a `reauth` message.

Server warning:

```
{
  "type": "reauth_required",
  "deadline": "2026-01-18T12:29:00Z",
  "timestamp": "2026-01-18T12:28:00Z"
}
```

Client sends new token:

```
{
  "type": "reauth",
  "ws_token": "eyJhbGciOiI... (new token) ...",
  "timestamp": "2026-01-18T12:28:30Z"
}
```

Server confirms:

```
{
  "type": "reauth_ok",
  "exp": "2026-01-18T13:00:00Z",
  "timestamp": "2026-01-18T12:28:30Z"
}
```

If the client does not reauthenticate in time:

```
{
  "type": "auth_expired",
  "timestamp": "2026-01-18T12:30:00Z"
}
```

Subscription Management

Subscribe to topics:

```
{
  "type": "subscribe",
  "topics": ["market.kraken.BTC-USD.candles.1h", "signals.paper.BTC-USD.rsi_btc_1h"]
}
```

Server confirms subscription:

```
{
  "type": "subscription_success",
  "action": "subscribe",
  "status": "subscribed",
  "topics": ["market.kraken.BTC-USD.candles.1h", "signals.paper.BTC-USD.rsi_btc_1h"],
  "denied_topics": [],
  "active_subscriptions": ["market.kraken.BTC-USD.candles.1h", "signals.paper.BTC-USD.rsi_btc_1h"],
  "message": null,
  "timestamp": "2026-01-18T12:00:01Z"
}
```

Unsubscribe:

```
{
  "type": "unsubscribe",
  "topics": ["market.kraken.BTC-USD.candles.1h"]
}
```

List active subscriptions:

```
{
  "type": "get_subscriptions"
}
```

Response:

```
{
  "type": "subscriptions_list",
  "subscriptions": ["signals.paper.BTC-USD.rsi_btc_1h"],
  "available_topics": ["market", "orders.commands", "orders.events", "signals", "strategy.signals", "system.heartbeats."],
  "total_available": 6,
  "timestamp": "2026-01-18T12:00:02Z"
}
```

Ping/Pong

Client sends:

```
{
  "type": "ping"
}
```

Server responds:

```
{
  "type": "pong",
  "timestamp": "2026-01-18T12:00:00Z",
  "active_connections": 5
}
```

Server Data Messages

Data messages are flat JSON objects forwarded directly from the ZMQ bus. There is no "data" wrapper. Messages with malformed JSON are dropped by the bridge before forwarding and are counted in the `invalid_messages` metric for the topic.

All data messages carry `session_id` and `sequence_id` provenance fields (see REST section above). All WebSocket control messages (auth, subscribe, ping/pong, errors) inherit from `StrictDataSchema`, so they also carry the same provenance envelope. Clients can use these fields to detect gaps without server-side replay support.

Candle

```
{
  "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
  "type": "candle",
  "timestamp": "2026-01-18T11:00:00Z",
  "session_id": "019e0000-0000-7000-0000-000000000001",
  "sequence_id": 42,
  "instrument": "BTC-USD",
  "exchange": "kraken",
  "timeframe": "1h",
  "open_at": "2026-01-18T11:00:00Z",
  "open": 42000.0,
  "high": 42500.0,
  "low": 41800.0,
  "close": 42300.0,
  "volume": 1234.56,
  "vwap": 42150.0,
  "trades": 5678
}
```

Tick

```
{
  "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
  "type": "tick",
  "timestamp": "2026-01-18T12:00:00Z",
  "instrument": "BTC-USD",
  "exchange": "kraken",
  "volume": 1234.5,
  "bid": 42000.0,
  "ask": 42001.0,
  "last": 42000.5
}
```

Trade

```
{
  "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
  "type": "trade",
  "timestamp": "2026-01-18T12:00:00Z",
  "instrument": "BTC-USD",
  "exchange": "kraken",
  "executed_at": "2026-01-18T12:00:00Z",
  "price": 42000.5,
  "volume": 0.25,
  "side": "buy"
}
```

Signal

```
{
  "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
  "type": "signal",
  "timestamp": "2026-01-18T12:00:00Z",
  "session_id": "019e0000-0000-7000-0000-000000000001",
  "sequence_id": 7,
  "instrument": "BTC-USD",
  "exchange": "paper",
  "side": "buy",
  "strength": 0.85,
  "reason": "RSI 28.5 <= 30",
  "price": 42000.0,
  "strategy_name": "rsi_btc_1h",
  "fired_at": "2026-01-18T12:00:00Z"
}
```

Execution

```
{
  "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
  "type": "execution",
  "timestamp": "2026-01-18T12:01:00Z",
  "trade_id": "TTRAD-456",
  "exchange_order_id": "KRAKEN-456",
  "client_order_id": "signal-a1b2c3d4",
  "instrument": "BTC-USD",
  "exchange": "kraken",
  "side": "buy",
  "size": 0.1,
  "price": 42000.0,
  "fee": 0.001,
  "fee_asset": "USD",
  "status": "filled",
  "executed_at": "2026-01-18T12:00:59Z"
}
```

Order

```
{
  "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
  "type": "order",
  "timestamp": "2026-01-18T12:00:00Z",
  "exchange_order_id": "KRAKEN-456",
  "client_order_id": "signal-a1b2c3d4",
  "instrument": "BTC-USD",
  "exchange": "kraken",
  "side": "buy",
  "status": "submitted",
  "order_type": "limit",
  "size": 0.1,
  "filled_size": 0.0,
  "price": 42000.0,
  "average_price": null,
  "reason": null,
  "time_in_force": "GTC",
  "error": null,
  "created_at": "2026-01-18T12:00:00Z",
  "updated_at": null
}
```

Heartbeat

```
{
  "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
  "type": "heartbeat",
  "timestamp": "2026-01-18T12:00:00Z",
  "session_id": "019e1a2b-0000-7000-8000-000000000001",
  "sequence_id": 42,
  "component": "zmq_broker",
  "sequence": 42,
  "status": "healthy",
  "lag_ms": 5,
  "meta": {}
}
```

Error

```
{
  "type": "error",
  "message": "Topic 'invalid.topic' does not exist",
  "timestamp": "2026-01-18T12:00:00Z"
}
```

Available Topic Patterns

Topics use dot-separated hierarchical names. Subscribe using the full topic string or a prefix to match multiple topics.

Market Data

- `market.{exchange}.{instrument}.candles.{timeframe}` -- OHLCV candles
- `market.{exchange}.{instrument}.ticks` -- Price ticks
- `market.{exchange}.{instrument}.trades` -- Individual trades
- `market.paper.{source_exchange}.{instrument}.candles.{timeframe}` -- Paper candles sourced from a real exchange
- `market.paper.{source_exchange}.{instrument}.ticks` -- Paper ticks sourced from a real exchange

Signals

- `signals.{exchange}.{instrument}.live` -- Live trading signals
- `signals.paper.{instrument}.{strategy_name}` -- Paper trading signals

Order Events

- `orders.events.{exchange}.{instrument}.submitted` -- Order submitted
- `orders.events.{exchange}.{instrument}.accepted` -- Order accepted
- `orders.events.{exchange}.{instrument}.rejected` -- Order rejected
- `orders.events.{exchange}.{instrument}.executed` -- Order fill
- `orders.events.{exchange}.{instrument}.cancelled` -- Order cancelled
- `orders.events.{exchange}.{instrument}.expired` -- Order expired
- `orders.events.{exchange}.{instrument}.replaced` -- Order replaced

Order Commands

- `orders.commands.{exchange}.{instrument}.submit` -- Submit order
- `orders.commands.{exchange}.{instrument}.cancel` -- Cancel order
- `orders.commands.{exchange}.{instrument}.replace` -- Replace order

System

- `system.heartbeats.{component}.{name}[.{source}]` -- Component heartbeats
- `admin.{resource}` -- Administrative events (admin only)

Error Handling

HTTP Status	Description
401	Missing or invalid authentication
403	Insufficient permissions or invalid CSRF token
404	Resource not found

HTTP Status	Description
422	Request validation error
429	Rate limit exceeded (Retry-After header included)
500	Internal server error

Usage Example (Python)

```
import httpx

BASE_URL = "http://localhost:8000/api"

async def main():
    async with httpx.AsyncClient() as client:
        login_resp = await client.post(
            f"{BASE_URL}/auth/login",
            json={"username": "admin", "password": "password123"},
        )
        csrf_token = client.cookies.get("csrf_token")

        refresh_resp = await client.post(f"{BASE_URL}/auth/refresh")
        ws_token = refresh_resp.json()["ws_token"]

        candles_resp = await client.get(
            f"{BASE_URL}/candles",
            params={
                "instrument": "BTC-USD",
                "exchange": "kraken",
                "timeframe": "1h",
            },
            headers={"X-CSRF-Token": csrf_token},
        )
        candles = candles_resp.json()
    print(candles)
```

Usage Example (JavaScript)

This example is intentionally minimal. The shipped frontend client reuses cached WS tickets, schedules proactive reauthentication before expiry, and sends heartbeat pings every 5 seconds.

```
async function connect() {
    const refreshResp = await fetch("/api/auth/refresh", {
        method: "POST",
        credentials: "include",
    });
    const { ws_token } = await refreshResp.json();

    const protocol = location.protocol === "https:" ? "wss:" : "ws:";
```

```

const ws = new WebSocket(`${protocol}/${location.host}/api/ws`);

ws.onopen = () => {
  console.log("Connected, waiting for auth_required...");
};

ws.onmessage = (event) => {
  const msg = JSON.parse(event.data);

  if (msg.type === "auth_required") {
    ws.send(JSON.stringify({
      type: "authenticate",
      ws_token: ws_token,
    }));
  }

  if (msg.type === "auth_complete") {
    console.log("Authenticated. Topics:", msg.available_topics);
    ws.send(JSON.stringify({
      type: "subscribe",
      topics: ["market.kraken.BTC-USD.candles.1h"],
    }));
  }

  if (msg.type === "candle" || msg.type === "tick") {
    console.log("Data:", msg);
  }

  if (msg.type === "reauth_required") {
    refreshAndReauth(ws);
  }
};

setInterval(() => {
  if (ws.readyState === WebSocket.OPEN) {
    ws.send(JSON.stringify({ type: "ping" }));
  }
}, 30000);
}

```

Backtests

Backtest endpoints manage strategy backtesting runs. Read endpoints require `read:backtests` permission (viewer+). Mutation endpoints require `manage:backtests` (operator+). Reads are wallet-scoped when the caller has an active wallet selected; admins without a wallet selection see all runs.

POST /api/backtests

Create and launch a new backtest run. Requires an active wallet selection.

Request body:

```
{
  "strategy_class": "sma_cross",
  "instrument_public_id": "BTC-USD",
  "exchange": "kraken",
  "timeframe": "1h",
  "start_date": "2026-01-01T00:00:00Z",
  "end_date": "2026-06-01T00:00:00Z",
  "initial_cash": 10000.0,
  "strategy_params": {"fast": 10, "slow": 30}
}
```

Response: BacktestRunResponse with the created run details.

GET /api/backtests

List backtest runs with optional filters.

Query parameters:

Parameter	Type	Description
strategy	string	Filter by strategy name
status	string	Filter by status (pending, running, completed, failed, cancelled)
limit	int	Page size (1-100, default 20)
offset	int	Page offset (default 0)
as_of	datetime	Temporal query timestamp

GET /api/backtests/{run_id}

Get backtest run detail by public ID.

POST /api/backtests/{run_id}/cancel

Cancel a pending or running backtest. Sets status to cancel_requested via SCD2 close-and-insert. Returns 409 if the run is already in a terminal state.

POST /api/backtests/{run_id}/rerun

Create a new backtest run with the same configuration as the original.

GET /api/backtests/{run_id}/trades

Get paginated trades for a completed backtest run.

GET /api/backtests/{run_id}/signals

Get paginated signals for a completed backtest run.

GET /api/backtests/{run_id}/events

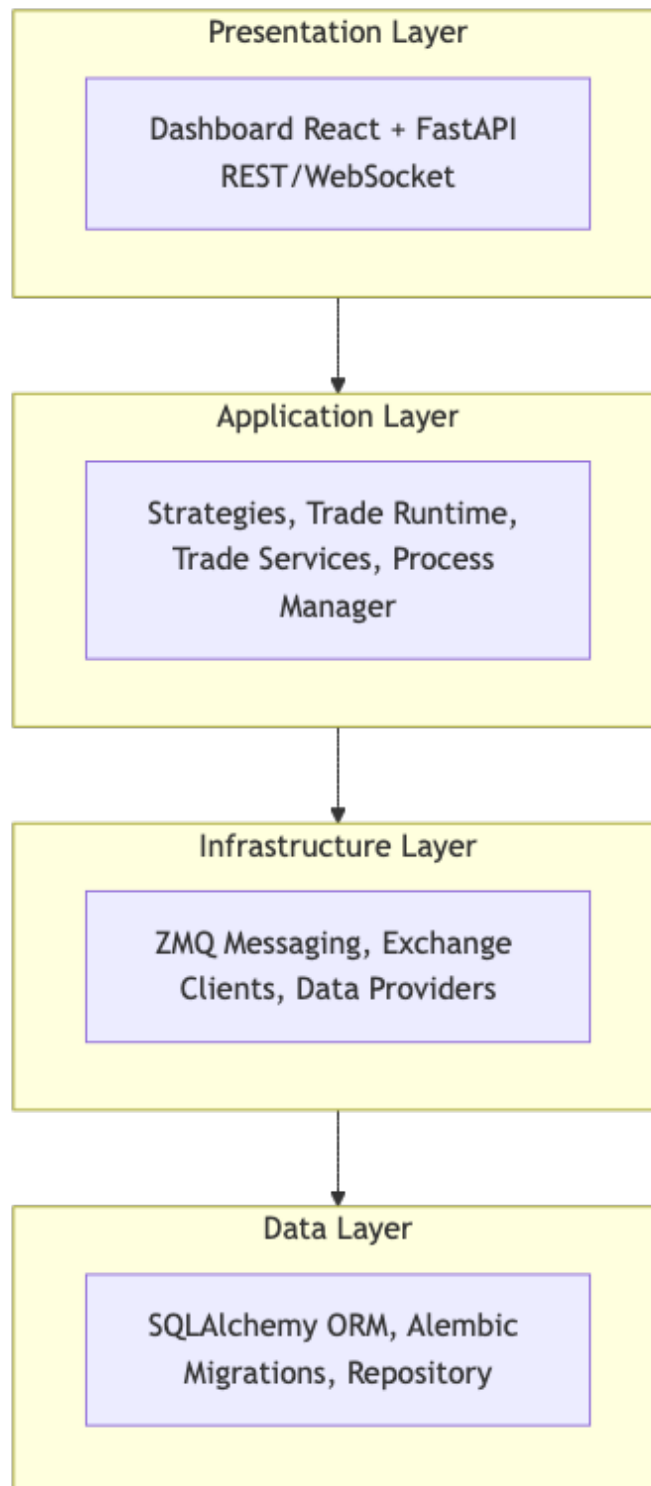
Get lifecycle events for a backtest run (started, failed, etc.).

Architecture

Snapper is a trading platform built with a layered architecture using asynchronous processing and ZeroMQ messaging.

The trading path is facts-canonical: Order and Execution are the canonical business facts, while Position, Balance, and equity are derived projections that can be rebuilt from facts plus checkpoints.

System Layers



Components

Core (src/snapper/core/)

Fundamental types and aliases used throughout the application:

- TradeSide — Trade direction (buy, sell)
- OrderType — Order type (market, limit, stop, stop_limit)
- OrderStatus — Order status in lifecycle
- ExecutionMode — Execution mode (live, paper)
- OrderExchange — Order-capable exchanges (paper, kraken, walutomat)

Config (src/snapper/config/)

Two-layer configuration:

1. Bootstrap Settings (bootstrap.py)

Settings from environment variables required before database connection:

- Database URL
- Master password for encryption
- HTTP and ZMQ server endpoints

2. App Settings (app.py)

Settings from database (encrypted):

- Exchange API keys
- Trading parameters
- Authentication configuration

Data (src/snapper/data/)

Persistence layer with SQLAlchemy:

- **ORM Models** (models.py):
 - Instrument — Financial instruments (natural key: symbol_public_id + exchange). Instrument.public_id is the stable identity used by fact tables (orders, executions, positions, signals, candles). The versioned business attributes symbol_public_id and exchange are resolved via ensure_instrument()
 - Candle — OHLCV candles
 - Trade — Transactions
 - Order — Canonical order lifecycle facts (includes mode: "live" or "paper")
 - Execution — Canonical confirmed fills
 - Position — Derived position projection (includes mode: "live" or "paper")
 - TradeCommand — Durable trade intent written by the engine before execution

- VenueEvent — Durable venue observations and acknowledgements persisted by executors
- TradeProjectionCheckpoint — Materialized position/balance snapshot for fast recovery
- Signal — Signal events
- User — System users
- Setting — Settings (encrypted)
- Symbol — Symbol identity with versioned attributes (native_symbol, base, quote, asset_type) via SCD Type 2. Archive symbols for filesystem paths are derived from the anchor row (first version per public_id)
- SymbolAlias — Exchange-specific symbol aliases (one row per native/exchange/channel)
- SymbolExchangeCapability — Exchange-specific symbol capabilities
- ProcessRun — Background process execution records
- InstrumentSpec — Instrument trading specifications (tick_size, margins, expiry_at, instrument_kind)
- UnderlyingAsset — Canonical asset identity (e.g., S&P 500, Gold) linking instruments across exchanges
- InstrumentUnderlyingMapping — Temporal link from instrument to underlying (exact/derivative/proxy)
- MarketSnapshot — Real-time market data snapshots
- UserLoginEvent — Authentication event log
- Control — Always-on audit for commands, auth events, subscribe/unsubscribe, REST mutations. Includes redacted payload, outcome, and client causation linkage (client_session_id, client_public_id)
- Telemetry — Toggleable high-volume table for pings, heartbeats, pongs, and GET read requests. Recording gated by TELEMETRY_RECORDING_ENABLED setting

All ORM models carry provenance via TemporalMixin:

- session_id (str, required) — producer session identity
- sequence_id (int, required) — per-table monotonic counter for gap detection

StrictDataSchema (Pydantic base) requires these fields at construction. Producers must obtain values from a SequenceTracker before creating the event, ensuring every payload is complete and identifiable from birth.

All ORM models use a dual-key identity pattern:

- id (INTEGER, internal PK) — row/version identifier for SCD2 close operations: SELECT → lock → close by id → INSERT new version. Never exposed outside the repository layer.
- public_id (UUID7 string) — logical entity identifier for domain joins and external references. Stable across SCD2 versions.

- timestamp (DateTime, system time / known_from)
- known_to (DateTime NOT NULL, default KNOWN_TO_MAX = 9999-12-31T23:59:59 UTC, active rows have known_to == KNOWN_TO_MAX; query pattern: WHERE timestamp <= :t AND known_to > :t)

No hard foreign keys between temporal entities. All domain joins use child.instrument_public_id == Instrument.public_id (or Execution.order_public_id == Order.public_id) with temporal filters. This enables future archival of closed SCD2 rows without breaking referential integrity.

- **SQLAlchemyRepository** (repository.py) — Async CRUD for SQLite/PostgreSQL
- **DatabaseRepository** (repository.py) — Sync access for scripts, archiver, and background updaters
- **CandleCacheArchiver** (archiver.py) — Exports active candle data to polygon-compatible CSV cache files per exchange/archive_symbol
- **EventArchiver** (archiver.py) — Exports append-only event rows (Tick, Trade, Signal, Execution, Telemetry, Control) to per-day CSV archive files with full temporal metadata, merge/dedup, and optional purge
- **CandleAuditArchiver** (archiver.py) — Exports all candle SCD2 versions (closed + active) with full temporal metadata, grouped by open_at date
- **StateArchiver** (archiver.py) — Generic archiver for state-SCD2 tables (Order, Position, Instrument, Setting, Symbol, etc.) with closed_only filtering and Symbol anchor row protection
- **Archive symbol resolution** (archive_symbols.py) — Stable filesystem-safe symbol naming derived from Symbol anchor rows, with seniority-based collision handling

Strategies (src/snapper/strategies/)

Trading strategies framework:

- **BaseStrategy** (base.py) — Base class for all strategies
- **StrategySignal** — Trading signal structure
- **StrategyConfig** — Strategy configuration

Built-in strategies:

- **RSIReversion** (rsi.py) — Mean reversion on RSI
- **MACDCrossover** (macd.py) — MACD crossover
- **CointegrationPairs** (cointegration.py) — Pairs trading

Indicators (src/snapper/indicators/)

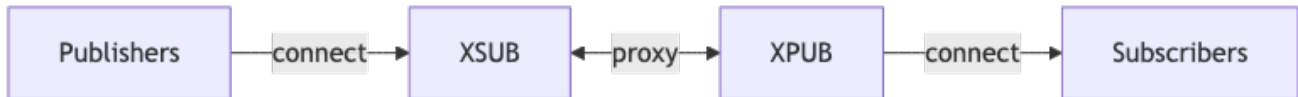
Technical indicators:

- **rsi** — Relative Strength Index (Python implementation)

- `macd` — Moving Average Convergence Divergence
- `ta_lib_adapter` — TA-Lib library adapter

Messaging (`src/snapper/messaging/`)

ZeroMQ pub/sub architecture:



Components:

- **Broker** (`infrastructure/broker.py`) — XPUB/XSUB proxy
- **Publishers** (`publishers/`) — Market data publication
- **Executors** (`executors/`) — Per-wallet order execution on exchanges. One executor process per (exchange, wallet) pair is spawned at boot from active `wallet_credentials` rows; each instance filters incoming commands by `wallet_public_id` and loads its credentials via `CredentialResolver` during startup.
- **Schemas** (`schemas/`) — Pydantic message models
- **Topics** (`topics/`) — ZMQ topic definitions

Message types (in `messaging.schemas.data`):

- `TickData` — Price tick
- `CandleData` — OHLCV candle
- `SignalData` — Trading signal
- `TradeData` — Trade execution
- `OrderData` — Order status
- `ExecutionData` — Order execution
- `OrderRequestData` — Order request
- `HeartbeatData` — Component heartbeat

Every Data class carries `public_id: str (UUID7)`, `type: Literal[...]`, and `timestamp: datetime`.

Venue reconciliation (executors)

Executors can periodically poll exchange REST APIs to reconcile local pending-order state against the exchange's authoritative view. This detects two failure modes that WebSocket streams alone miss: **fill gaps** (exchange has more filled quantity than the executor observed) and **disappeared orders** (pending orders absent from the open-orders snapshot).

Always-on — runs alongside the executor's order handler + heartbeat tasks (no feature flag; the defensive polling is cheap enough that gating it provided no value).

_reconcile_fill_gap (messaging/executors/base.py:1139-1211) emits a corrective ExecutionUpdate covering the observed cum-qty delta, stamping the synthetic execution with `exec_id=recon-<exchange_oid>-<monotonic_ns>` so downstream de-dup stays correct.

_reconcile_disappeared_order (messaging/executors/base.py:1074-1137) calls `get_order()` on orders missing from the open-orders snapshot to determine their actual terminal status (CLOSED / CANCELED / EXPIRED), emits any residual fill gap first, then publishes the terminal ExecutionUpdate.

Known limitation — market orders without price

If a market order's exchange snapshot reports `price=None`, the fill-gap path logs an ERROR ("no price on market order, skipping corrective fill") and skips the corrective emission. The executor does not subscribe to ticks and cannot approximate the fill price without degrading VWAP accuracy silently. Recovery is lifecycle-dependent: if the order stays open and a later snapshot populates price, the next reconciliation iteration emits the corrective fill; if the order reaches a terminal state first, the skipped gap persists until reconciled manually against the exchange's fill history. Operators seeing this ERROR log should not patch Snapper to approximate; the skip is an intentional trade-off (predictable behaviour + observable gap) over silent approximation drift.

Application (src/snapper/application/)

Business logic:

- **Engine** (engine/) — `TraderCoordinator` is the multi-symbol trade runtime coordinator; `TradingEngineService` is the per-instrument engine that turns signals into `TradeCommand` rows and order requests
- **Trade** (trade/) — Trade-domain services: `trade_service.py` (in-memory command and position read model), `balance_service.py` (cash/equity/exposure projection), `outbox.py` (outbox-driven publishing), `reconciler.py` (stale-command scan and reconciliation failure feedback)
- **Process Manager** (process_manager/) — Process management
- **Services** (services/) — Application services
- **Updaters** (updaters/) — Data updates (symbols, historical)
- **Risk** (risk/) — Risk management
- **Portfolio** (portfolio/) — Portfolio management

Infrastructure (src/snapper/infrastructure/)

External integrations:

- **Exchanges** (exchanges/) — Exchange clients (Kraken, Walutomat)
- **Market Data** (market_data/) — WebSocket feeds
- **Symbols** (symbols/) — Symbol mapping
- **Security** (security/) — Settings encryption
- **Logging** (logging/) — Logging configuration

Server (src/snapper/server/)

FastAPI application:

- **App Factory** (app.py) — create_app() with lifespan management
- **REST API** — HTTP endpoints
- **WebSocket** — Real-time streaming via ZMQ bridge
- **Static Files** — Frontend dashboard
- **ClientProvenanceMiddleware** (provenance_middleware.py) — Extracts client provenance from mutation requests, runs per-session gap detection, records mutations to control table and GET reads to telemetry table

API (src/snapper/api/)

Shared API schemas and WebSocket auth helpers (not route definitions):

- **Schemas** (schemas/) — Pydantic request/response models (health, process, settings).

Schema class hierarchy:

```
text
BaseModel
├── PartialBody          - extra="ignore", strict=True (GapEnvelope,
WsTokenPayload)
├── StrictBody           - extra="forbid", strict=True (request bodies,
nested DTOs)
│   ├── StrictDataSchema - + provenance (all event payloads, REST/WS/ZMQ)
│   │   ├── PayloadRequest[T]
│   │   ├── PayloadResponse[T]
│   │   └── PayloadListResponse[T]
├── ExchangeResponse     - extra="allow", strict=True (exchange API
parsing)
├── ExchangeRequest      - extra="forbid", strict=True (outgoing exchange
requests)
└── BaseSettings         - BootstrapSettingsLoader (env var config)
```

- **Auth** (auth/) — WebSocket token service and schemas

Route modules live closer to their domains: `server/app.py` (assembly and data endpoints), `server/process_routes.py`, `server/strategy_routes.py`, `config/settings_routes.py`, and `auth/routes.py`.

REST endpoints return the same Data schemas used by WebSocket messaging (`messaging.schemas.data`): `OrderData`, `SignalData`, `ExecutionData`, `PositionData`, `CandleData`. Old REST-specific schema modules were removed.

Auth (`src/snapper/auth/`)

Authentication system:

- JWT tokens with access/refresh via HTTP-only cookies
- CSRF protection
- Role-based access (viewer, operator, admin)
- WebSocket authentication

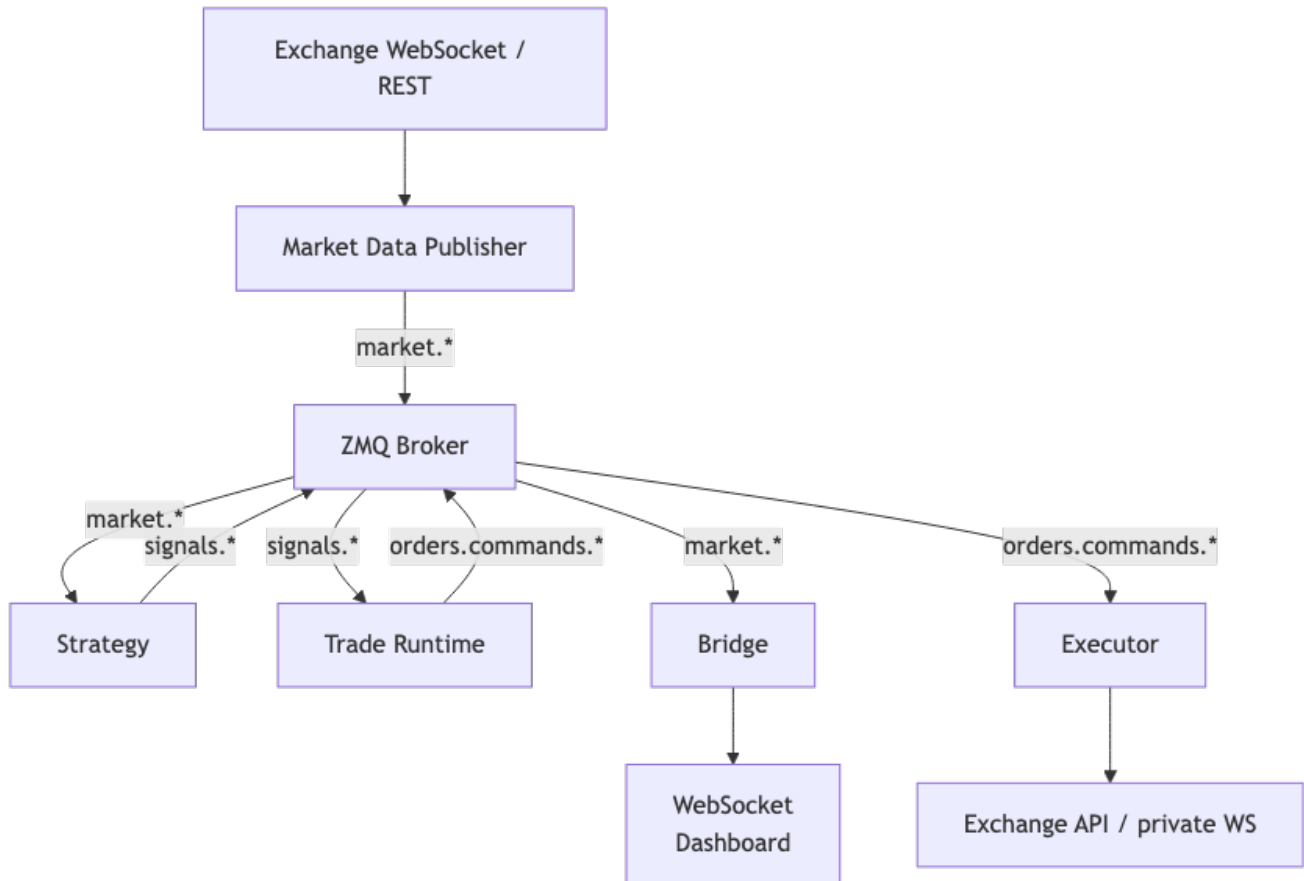
CLI (`src/snapper/cli/`)

Command-line interface (Typer):

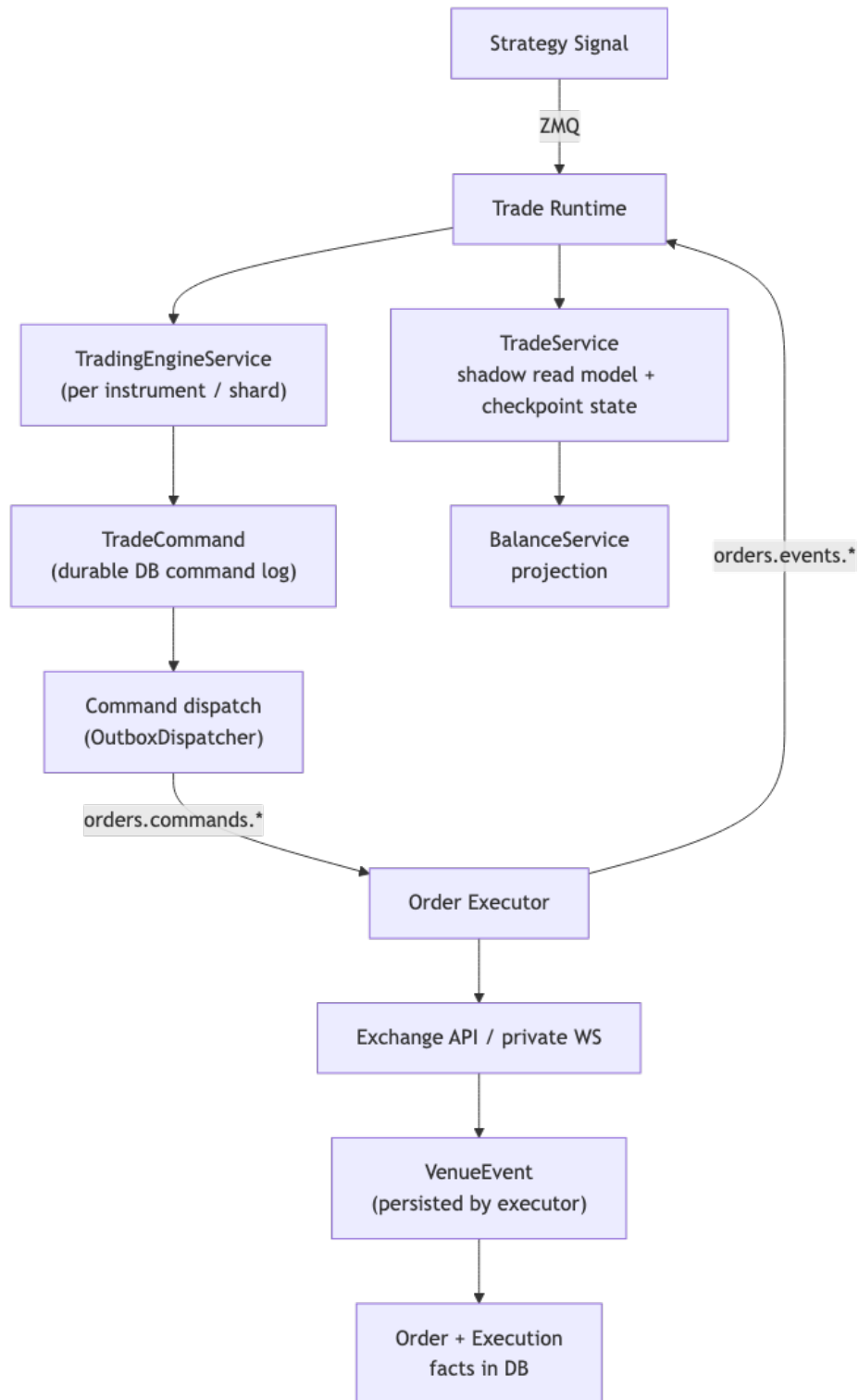
- Server and infrastructure management
- Database operations
- User management
- Market data updates

Data Flow

Market Data and Signal Flow



Order Flow



Database

SQLite for development, PostgreSQL for production.

Schema

```
-- Market data (joined to instruments via instrument_public_id)
instruments          -- Financial instruments (logical key: symbol_public_id +
exchange)
candles              -- OHLCV data
ticks                -- Real-time price snapshots
trades               -- Transaction history
market_snapshots     -- Real-time market data (SCD2 per instrument, one active
row each)

-- Trading (joined to instruments via instrument_public_id)
orders               -- Order history (mode: live/paper)
executions           -- Order executions (joined to orders via order_public_id)
positions            -- Portfolio positions (mode: live/paper)
signals              -- Strategy signals

-- Trade runtime (durable command path)
trade_commands        -- Durable trade intent (engine writes before
execution)
venue_events          -- Durable venue observations and acknowledgements
from executors
trade_projection_checkpoints -- Materialized position/balance snapshots

-- Symbol management
symbols              -- Symbol identity + versioned attributes (SCD2)
symbol_aliases        -- Exchange-specific symbol mappings
symbol_exchange_capabilities -- Exchange-specific capabilities

-- Instrument specifications
instrument_specs      -- Trading specifications (tick_size, lot_size, limits)

-- System
users                -- User accounts
settings              -- Settings (encrypted)
user_login_events     -- Authentication event log
process_runs          -- Background process execution records
control               -- Audit log (commands, auth events, mutations)
telemetry             -- High-volume operational events (toggleable)
```

Migrations

Alembic manages migrations:

```
snapper db-upgrade    # Apply migrations
snapper db-downgrade  # Rollback
```


Trade Runtime

The trade runtime always uses the durable (outbox-driven) dispatch path.

TradingEngineService writes TradeCommand rows to the database; OutboxDispatcher polls the table and publishes undispatched commands to ZMQ. Executor VenueEvent writes are fail-closed — a failed persist raises before the executor acknowledges the venue event.

The TraderCoordinator class acts as the trade runtime coordinator and integrates TradeService (command lifecycle) and BalanceService (balance tracking). It spawns the outbox dispatcher (when a SQLAlchemyRepository is wired) plus per-exchange reconciliation loops that feed the circuit breaker. Canonical Order and Execution rows are persisted on the exchange/executor path.

Multiple TraderCoordinator processes can run against the same DB + broker with deterministic SHA-256 shard partitioning via snapper.core.partitioning.ShardOwnership. Each coordinator owns $\sim 1/N$ of the shard_key set; signals, venue events, recovery, outbox dispatch, and reconciliation all filter by ownership so no two instances process the same shard. Default --instance-count 1 is byte-identical to single-coordinator behavior. See [docs/operations.md](#) for the systemd template recipe and scale-up / scale-down / crash-recovery procedures.

Bitemporal Model

All entity tables inherit TemporalMixin which provides id, public_id, timestamp (bus-time / known_from), and known_to columns.

Key concepts:

- **SCD Type 2** — No in-place UPDATE or DELETE. Mutations use a close+insert pattern: the current row is closed (known_to set to now) and a new row is inserted with the updated values and known_to = KNOWN_TO_MAX.
- **KNOWN_TO_MAX** — Sentinel value (9999-12-31T23:59:59 UTC) marking the active version of a row.
- **Point-in-time queries** — REST endpoints accept an as_of query parameter to retrieve the state of data at a specific moment.
- **Helpers:**
 - where_active(model, at) — SQLAlchemy filter for temporal queries. at: datetime is required — no silent default-to-now
 - where_active_now(model) — convenience for operations that genuinely mean "current state" (auth login, heartbeat). Not a shortcut for skipping as_of threading
 - close_and_insert() / close_and_insert_sync() — Repository helpers that atomically close the current row and insert a new version

Batch atomicity guarantees

True SCD2 batch writes (upsert_candles, upsert_market_snapshots) and single-row close+insert methods (revise_*, close_and_insert) commit as all-or-nothing transactions. The pattern is:

```
async with self.session() as s:           # opens a transaction
    for row in rows:
        # SELECT ... FOR UPDATE the active version
        # UPDATE close (set known_to = bus_time)
        # s.add(new row)
    await s.commit()                       # single commit
```

If any row in the loop raises (IntegrityError, serialization failure, connection drop, await cancellation), the transaction aborts and **no** close/insert pair from this batch lands in the DB. The batch must be retried in full; there is no partial-progress mode.

upsert_candles batch-loads existing versions for unique (instrument_public_id, timeframe, open_at) keys before closing and inserting rows. If the incoming batch contains duplicate candle natural keys, those rows keep the sequential close+insert path because an earlier row in that group can create the version a later row must close. Lookup batches are chunked to stay below SQLite parameter limits.

Append-only batch writes do NOT use the SCD2 close-insert pattern and split into two shapes:

- upsert_trades delegates to the dialect-aware _upsert_batch. On native dialects (SQLite / Postgres) it uses INSERT ... ON CONFLICT DO NOTHING in a single s.execute — atomic per-batch, and duplicates are silently skipped. On the fallback row-by-row path it wraps each row in a begin_nested() SAVEPOINT; successful rows stay in the outer transaction and are committed at the final await s.commit(), while rows that hit IntegrityError are skipped. This trades all-or-nothing for conflict tolerance.
- upsert_ticks is plain session.add_all(rows) + single await s.commit(). Atomic per-batch on every dialect — it is not routed through _upsert_batch and has no duplicate-skipping behaviour.

Serialization on the same natural key: on **PostgreSQL**, row-level SELECT ... FOR UPDATE serializes concurrent transactions via MVCC row locks — a second session waiting on a locked active row observes the first session's close+insert only after the first commits; both batches complete without error and produce a clean SCD2 chain.

On **SQLite** (aiosqlite), SELECT ... FOR UPDATE is a no-op and the default BEGIN DEFERRED isolation does not acquire a write lock at SELECT time — two concurrent tasks can both read the same active row before either writes. The first writer to COMMIT wins; the second writer's INSERT then collides on the public_id UNIQUE index and raises sqlalchemy.exc.IntegrityError. The final DB state is still a valid SCD2 chain (exactly one

active row, carried-forward public_id preserved, no orphan inserts) — one writer is simply reported the conflict via exception rather than implicitly serialized. Callers on SQLite that race on the same natural key must either retry the failed batch or rely on the single-publisher-per-exchange invariant to avoid the contention entirely.

At-scale implication: under heavy multi-publisher contention on the same natural key, batches serialize strictly. If a long-running batch holds the lock for many milliseconds per row, throughput falls linearly in contention. The current single-publisher-per-exchange shape makes this a non-issue; revisit if multi-publisher contention on the same natural key becomes a real workload.

Non-atomic fallback (_upsert_batch row-by-row path): the dialect-agnostic fallback at repository.py:2540-2550 wraps each row in a begin_nested() SAVEPOINT and continues on IntegrityError. Successful rows are NOT committed immediately — they are held in the outer transaction and finalised only by the final await s.commit(). This trades all-or-nothing for conflict tolerance — the caller sees inserted = N_successful, not all-or-nothing. Used only for append-only paths where a duplicate key is a genuine no-op. Never used for SCD2 close-and-insert.

Processes

The system manages processes through Process Manager:

- **Core** — Broker, Feed, Bridge (required — startup is aborted if any enabled long-running CORE process fails to start)
- **Strategy** — Trading strategies
- **Task** — One-time tasks
- **Backtest** — Backtesting processes

Processes can be:

- long_running — Run continuously (services)
- one_shot — Execute once (tasks)

Deployment Modes

By default, the FastAPI lifespan starts all enabled processes (all-in-one dev mode). Set SERVER_API_ONLY=true to skip process autostart -- the ZMQ-WebSocket bridge still starts, so the frontend receives live data from a separately-running engine. Useful when broker/strategies/executors run on different hosts.

Multi-instance deployment (AI-review fanout dedup): multi-worker uvicorn is a supported production topology. The shared ZMQ XPUB-XSUB broker delivers every internal bus event to every FastAPI worker's `AiReviewService.start_bus_listener`; each subscribed topic has its own dedup story so connected clients see exactly one external WS frame per source event regardless of worker count:

- `bus.delegate_offline` is idempotent at the DB layer — the handler runs a per-row `atomic_dispatch_fanout_with_audit CAS UPDATE`, so only one worker wins per affected review row even when every worker receives the bus event.
- `bus.ai_review_decision` is process-local — the handler resolves an `asyncio.Future` from the per-worker registry populated by the worker that ran `create_review` for the strategy await loop. Other workers stash the event in a bounded pending-resolution cache (1024-entry hard cap, 30s TTL); cache memory across the cluster is bounded by $N \times 1024$ entries even at peak.
- `bus.caps_violation_after_ai_approve` is gated by deterministic partitioning. `handle_caps_violation_bus_message` short-circuits on `ShardOwnership.owns(msg.review_public_id)`; exactly one worker (the SHA-256 owner of the review row) re-publishes the external `ai_reviews.*.caps_violation` frame. Other workers observe the bus event, log a debug skip, and return `False`. `instance_count == 1` (default deployment) makes `ShardOwnership.owns` return `True` unconditionally so behaviour is byte-identical to single-worker.

The same partitioning primitive backs the Trade Runtime shard ownership; AI-review reuses it via the `set_shard_ownership(ShardOwnership(coordinator_instance_id, coordinator_instance_count))` injection seam wired by the FastAPI lifespan after the publisher seam and before the bus listener starts.

Operational scaling: raise `SNAPPER_COORDINATOR_INSTANCE_COUNT` to `N` and start `N` FastAPI workers each with a distinct `SNAPPER_COORDINATOR_INSTANCE_ID` in `[0, N)` (env vars only — snapper server does not expose per-worker CLI flags; the `--instance-id` / `--instance-count` CLI flags exist on snapper trade-zmq for the Trade Runtime coordinator partitioning, not on the FastAPI server). Each worker subscribes to every bus topic; only the SHA-256 owner runs the caps-violation external fanout.

Execution Plans

The Execution Plans framework provides a unified control plane above the existing TradeCommand spine. All manual and algorithmic trading actions become `ExecutionPlan` instances evaluated by pluggable `PlanEvaluator` classes running inside `PlanExecutorService`.

Tables: execution_plans (plan state + lifecycle), execution_plan_checkpoints (high-churn evaluator state), execution_plan_decisions (tiered decision log), instrument_order_capabilities (capability matrix), venue_fee_schedules (fee tiers), position_cycles (one open→close lifetime per shard, brackets attach by position_cycle_public_id).

Plan types: manual_once (shipped), bracket (shipped — attaches to a position_cycles row), trailing_stop (shipped — stateful ratcheting stop with checkpoint persistence, separate /api/trailing-stops route module), peg, scheduler.

Manual order create flow: POST /api/orders creates a manual_once plan (pending), stamps child_client_order_id, native_instrument, and venue_order_type into plan.params, then inserts a create TradeCommand for the outbox dispatcher and transitions the plan to active. The OutboxDispatcher inside TraderCoordinator publishes the command as OrderRequestData on the orders.commands.{ex}.{instr}.submit topic.

Fill propagation: PlanExecutorService subscribes to orders.events. and market. on the broker XPUB and consumes every fill. Cumulative ExecutionData.size is compared against plan.filled_quantity with a small epsilon so duplicate or stale frames are dropped. Partial fills update filled_quantity and keep the plan active; full fills transition the plan to completed with completed_at stamped via SCD2.

Cancel flow: POST /api/orders/{plan_public_id}/cancel (plan id path) or POST /api/orders/by-client-order-id/{client_order_id}/cancel (UI convenience route) resolves the plan, verifies wallet scope, transitions the plan to cancel_requested, hydrates exchange_order_id from the active orders row, and inserts a cancel TradeCommand. On insert failure the plan is marked failed and the route returns HTTP 500; PlanExecutorService._recover_plans then re-emits the cancel on the next service restart (deduplicated via has_pending_cancel_command). The outbox dispatcher branches on command_type and re-hydrates exchange_order_id one more time at publish time so a late venue ACK is picked up, then publishes OrderCancelData on the orders.commands.{ex}.{instr}.cancel topic.

Listen loop resilience: per-message exceptions (parse failures, handler errors) are caught inside the loop so a single bad frame cannot silently stop the service. Transient recv_multipart errors retry with a 100 ms backoff. asyncio.CancelledError still unwinds the loop for graceful shutdown. After _setup_subscriber() the service sleeps 500 ms for ZMQ XPUB/XSUB slow-joiner stabilization before recovery to avoid losing terminal events triggered by stranded-cancel re-emits.

Position cycles: the position_cycles table represents one flat→non-flat→flat lifetime per shard via the standard TemporalMixin SCD2 envelope. A partial unique index uq_pc_shard_open_active enforces at most one open cycle per shard_key at any instant. Five repository methods cover the lifecycle: insert_position_cycle, close_position_cycle (SCD2 close-and-insert), get_open_position_cycle (lookup by shard_key because paper shards embed strategy_tag), flip_position_cycle (TOCTOU-hardened atomic close+open

with `shard_key` match assertion), and `update_position_cycle_max_qty` (monotonic — silent no-op when the new peak does not strictly exceed the existing one). The repository methods are called from the trader (not from `TradeService`, which stays sync and pure); `ShardState` carries trader-owned cache fields `active_cycle_public_id` and `active_cycle_max_qty`.

The trader's `_sync_fill_to_trade_service` samples `old_qty` before `apply_venue_event` and `new_qty` after, then delegates to `_sync_position_cycle_on_fill`. That helper classifies the transition via `TradeService._detect_cycle_transition` (returns `Literal["open", "close", "flip", "scale_up"] | None` with a `1e-12` epsilon to match the existing zero-snap) and issues the matching repository call. Four guards apply: degraded-identity fail-closed when `engine.wallet_public_id` is falsy, idempotency on open via a `get_open_position_cycle` pre-check, async `instrument_public_id` resolution via `get_instrument_public_id_by_symbol`, and a symmetric DB fallback on close/flip/scale_up paths so a degraded restart cannot reuse a stale cycle row from a prior life of the shard. When a flip's instrument resolution fails, the trader degrades to close-only — the old cycle is closed cleanly, the new leg is left uncovered with an audit-gap warning, and the cache is cleared so subsequent fail-soft paths handle themselves correctly.

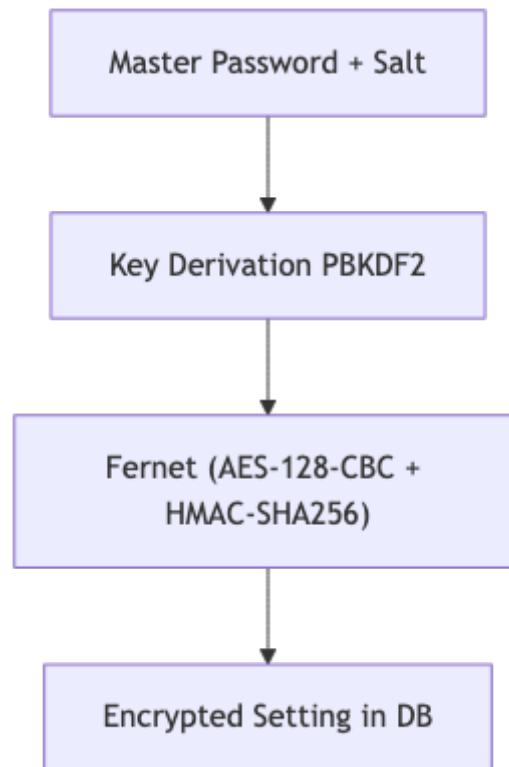
`_recover_engine_state` runs `_reconcile_position_cycles` as a fourth recovery phase, after engine state is rebuilt and before the trading loop starts. It iterates `self.engines` (engines, not `TradeService`, because full replay rebuilds `engine.position_qty` but not the projection) and handles four cases: recovered flat with a stale open row → close the row; recovered non-flat with a matching-direction open row → hydrate the cache and bump `max_qty` if downtime scaled-up beyond the stored peak; recovered non-flat with an opposite-direction row → atomic flip via `flip_position_cycle` (or degrade to close-only on unresolved instrument); recovered non-flat with no row → bootstrap a synthetic cycle. Brackets attach to `position_cycle_public_id`, not to an order, so a flat reopen does not inherit stale stop levels. Orphan cycles (open rows without a matching engine) can be detected and closed via admin endpoints at `/api/position-cycles/`.

max_qty is per-cycle, not lifetime. Each `position_cycles` row tracks the peak absolute quantity reached within its own `opened_at` → `closed_at` window. A new cycle on the same shard starts a fresh accumulator at `abs(opening_qty)`; it does not inherit or max against prior cycles' peaks. This matches the bracket-evaluator contract: a bracket attaching to cycle C1 is sized against C1's peak, not against all-time peak. A future UI surfacing `max_qty` must scope to the individual cycle row; summing or max-ing across cycles would conflate independent trading episodes.

Security

Settings Encryption

Sensitive data (API keys) are encrypted in the database:



Backtesting

The backtesting subsystem runs strategy simulations against historical candle data. DirectDbEngine performs synchronous candle reads from DB.

Architecture:

- **BacktestConfig** (Pydantic): Strategy, instruments, date range, fill model
- **DirectDbEngine**: Candle-driven simulation loop with time-batched processing
- **ResultCollector**: In-memory artifact buffer (signals, trades, equity)
- **BacktestMetrics**: Pure-math metrics computation (Sharpe, Sortino, CAGR, etc.)
- **BacktestRunnerProcess**: ONE_SHOT RegisterableProcess orchestrating lifecycle
- **BacktestRepository**: Separate repo sharing session_factory, SCD2 status transitions

Lifecycle: pending -> running -> completed | failed | cancelled

Boot reconciliation: Orphaned runs (pending/running/cancel_requested) are swept to failed at server startup via `reconcile_stale_runs()`. This runs only on the normal startup path (skipped in `SERVER_API_ONLY` mode).

Storage: 6 bitemporal tables (BacktestRun, BacktestEvent, BacktestSignal, BacktestTrade, BacktestEquityPoint, BacktestResult) all with TemporalMixin.

Authentication

- JWT tokens via HTTP-only cookies (access: 15min, refresh: 7 days)
- CSRF tokens for mutating requests
- WebSocket tokens (one-time)
- Bcrypt password hashing

Kraken FCM market data (TradFi)

Kraken's FCM platform hosts index-futures, energy, metals, and yield contracts (collective name: "TradFi" in this project). Unlike crypto spot + perpetuals, FCM has **no public order API** — Snapper treats these instruments as observation-only.

Data sources

- REST instrument metadata: `iapi.kraken.com/api/internal/markets/all/futures-contracts`. Reverse-engineered from `pro.kraken.com` (requires Origin: `https://pro.kraken.com` + Referer: `https://pro.kraken.com/` headers). Powers the `kraken_equities_symbol_updater` process.
- REST historical candles: `iapi.kraken.com/api/internal/markets/{ws_symbol}/ticker/history`. Same Origin/Referer requirement. Drives `KrakenEquitiesExchangeClient.get_ohlcv()` and the `kraken_equities_aggregates_backfill` process. Intervals {1, 5, 15, 30, 60, 1440} minutes accepted.
- WebSocket ticks + trades: `wss://ws-equities.kraken.com` (Kraken Spot WS v2 protocol with `asset_class: futures_contract`). Feed is delayed ~10 minutes per FCM policy. The outer envelope's `delayed: true` flag propagates onto every `TickData.is_delayed=True` so ZMQ subscribers can gate accordingly.

Capability model

Each TradFi symbol is seeded with a `SymbolExchangeCapability(exchange="kraken_equities", can_trade=False, can_market_data=True)` row. Every order-entry REST route (`/api/orders`, `/api/execution-plans`, `/api/trailing-stops`) calls `snapper.server._capability_guard.require_tradable` before emitting a `TradeCommand` — market-data-only submits fail with HTTP 422

error_code=instrument_market_data_only. The AST meta-test at tests/meta/test_order_entry_capability_guard.py enforces that every enumerated submit handler imports and calls the guard.

Frontend UX

GET /api/exchanges/{exchange}/instruments/detail exposes capability-aware rows (InstrumentDetailData — symbol + can_trade + can_market_data + instrument_kind + expiry_at). MarketData.tsx renders a "Market-data only" badge next to the page title when the currently-selected instrument is observation-only and marks every can_trade=false row in the instrument dropdown with the same pill. NewOrderModal.tsx consumes the same endpoint, shows an amber inline notice, suffixes option labels with — market-data only, and disables the submit button. Submit failures branch on APIError.details.error_code via errorMessages.ts.

Cross-asset strategy pattern (informational)

Strategies may subscribe TradFi candles via the normal market.kraken_equities.{symbol}.candles.{timeframe} topic and emit signals whose instrument targets a can_trade=True execution instrument (crypto spot, xStocks). See src/snapper/strategies/examples/tradfi_observe_crypto_execute.py for an illustrative EMA-crossover implementation + activation instructions in the module docstring. Cross-asset execution at the backtest-engine level is a deferred follow-up (engine changes required: batch_processor needs to use signal.instrument/signal.exchange for target attribution, plus multi-feed domain_time alignment).

Operational runbook

See docs/operations.md "Kraken Equities (TradFi) market data" for enable-the-feed steps (SQL DB Setting toggles), backfill invocation, quarterly rotation cadence, and troubleshooting (iapi reachability checks, WS disconnect loops).

Backtesting

Snapper ships two backtest execution engines, both producing the same artifacts (signals, trades, equity points) so a strategy author can pick either path with confidence.

Execution modes

The `execution_mode` field on `BacktestConfig` selects which engine the runner instantiates:

- `direct_db` — `DirectDbEngine` reads candles directly from the repository and feeds them into the strategy in-process. Lowest overhead; recommended for fast iteration.
- `zmq_replay` — `ZmqReplayEngine` spins up a private XPUB/XSUB broker on OS-assigned local ports, attaches the strategy as a SUB, and publishes candles via a `ReplayPublisher`. Exercises the full live message-bus path, so any production wiring bug surfaces in the backtest.

Both engines call the same `process_time_batch` helper for fill simulation, signal recording, and equity sampling, so artifact parity is structural rather than aspirational.

Choosing between modes

Concern	<code>direct_db</code>	<code>zmq_replay</code>
Speed	fastest (no IPC)	slower (per-candle ZMQ hop)
Fidelity	bypasses message bus	mirrors live wire path
Concurrency	unlimited	at most one active run (DB-enforced)
Cancel SLA	bounded by <code>cancel_poll_ms</code>	bounded by <code>cancel_poll_ms</code>

At-most-one-running invariant

Migration `0001_init` includes a partial unique index `uq_bt_single_running` on `backtest_runs(status)` filtered to `status='running'` AND `known_to=KNOWN_TO_MAX`. This is a declarative DB guarantee — a second runner trying to transition `pending` → `running` receives `IntegrityError` at `COMMIT`, the runner catches it via `is_single_running_conflict` and transitions to `failed` with a clear error message. No engine, no broker, no ports consumed for the loser.

The constraint is dialect-aware (PostgreSQL via `asyncpg` + SQLite via `aiosqlite`); see `snapper.data.backtest_conflict` for detection details.

Cancellation

CancelProbe polls `backtest_runs.status` between time batches (Direct-DB) or per processed candle (ZMQ replay). Throttled by `config.cancel_poll_ms` (default 500 ms), each DB read bounded by `probe_timeout_s` (default 1.0 s). A stuck DB logs a warning and the next probe re-tries — cancel detection degrades gracefully under SQLite lock contention rather than hanging behind the driver's 30 s lock timeout.

End-to-end cancel SLA target: ≤ 5 s from the API call `POST /api/backtests/{run_id}/cancel` (which sets status to `cancel_requested`) to terminal status `cancelled`.

start_date warmup gating

Candles before `config.start_date` feed strategy indicators (so SMA/MACD warm up properly) but do **not** generate fills, signals, or equity points. The `process_time_batch` helper enforces this with a single `if batch[0].open_at < config.start_date: return guard` before fill simulation.

Both engines honour the gate identically; the parity test `tests/integration/test_backtest_hardening.py::test_warmup_gating_parity_between_engines` asserts field-for-field equality.

ZMQ replay implementation notes

The replay path adds a few primitives the live system does not need:

- **Echo-ack handshake:** the publisher sends a sentinel `WARMUP_PUBLIC_ID` candle on every expected topic each round; the strategy mixin's `_listen_loop` ACKs each topic into `state.acked_topics` and only sets `state.subscriber_ready` when the set equals `state.expected_topics`. Up to `WARMUP_MAX_RETRIES=5` rounds of `WARMUP_READY_TIMEOUT_S=1.0` s each; failure raises `BacktestReadinessTimeoutError`.
- **Drain coordinator:** counts published vs processed candles. After the publisher streams the last candle it calls `mark_done_publishing()` and waits on `drained.wait()` bounded by `DRAIN_TIMEOUT_S=10.0` s. Failure raises `BacktestDrainTimeoutError` with both counters in the message so an operator can tell publisher- stall from strategy-death.
- **Bounded cleanup:** the engine cancels publisher and listen tasks then `asyncio.wait(timeout=2.0)` — tasks that refuse to terminate are logged as leaks rather than silently kept alive.

- **Shielded DB writes:** the runner wraps every terminal-status DB write (cancelled, failed, fail-event insert) in `asyncio.shield` so a second cancel arriving mid-write does not interrupt the write coroutine.

Advanced metrics, live progress, comparison

The backtest surface ships three interlocking capabilities:

Advanced metrics

Eight additional metric columns on `backtest_results` — five promoted from the `extra_metrics` JSON blob (`sortino_ratio`, `cagr`, `calmar_ratio`, `expectancy`, `avg_trade_pnl`) plus three net-new metrics computed in `snapper.application.backtest.metrics`:

- **max_drawdown_duration_seconds** — longest peak-to-recovery window; unrecovered runs use the final suffix from last peak to end-of-curve. Degenerate input (fewer than two points, flat curve, strictly monotone) returns `None` + a `metric_warning` event.
- **exposure_ratio** — fraction of total run duration during which the portfolio held a non-zero position; leading-edge interval attribution. Zero-trade runs with non-zero duration return `0.0` (not degenerate).
- **turnover_ratio** — total notional traded divided by mean equity. Zero-trade returns `0.0`; non-positive mean equity returns `None` + warning.

Read-side fallback at `GET /api/backtests/{id}` collapses legacy `extra_metrics` JSON into the typed slots using `explicit is not None` coalescing (never Python truthiness — preserves legitimate `0.0` values). The response strips the five promoted names from the emitted `extra_metrics` so mixed-vintage rows never surface the same metric twice.

Live progress (4-segment WS topic family)

Runner emits `BacktestProgressData` events on `backtest.{wallet_public_id}. {run_public_id}. {event}` where `event` $\in$ `{started, progress, milestone, completed, failed, cancelled}`. The existing ZMQ→WS bridge forwards the envelope to every subscribed client.

- `BacktestProgressEvent` is the canonical Literal source of truth in `snapper.messaging.schemas.data`; topic validator, emitter, and frontend handlers all derive from it.
- `BacktestProgressEmitter` throttles progress at 250 ms, emits milestone events exactly once per 25/50/75 pct bucket (bypassing the throttle so milestones + progress at the same step do not collide), and fires started / terminal events once. Degrades gracefully when `total_candles=None` by disabling milestones and pinning `progress_pct=0.0`.

- Per-subscription RBAC: admins subscribe to any wallet prefix; non-admin roles subscribe only to prefixes matching their active_wallet_public_id. Foreign-wallet or bare backtest. subscriptions are denied at the handler before the bridge is touched.
- BacktestProgressData.milestone carries a @model_validator enforcing the cross-field invariant milestone is not None iff event == 'milestone' so a malformed payload cannot reach the frontend chip logic.

Config hash for auto-pair

Every new run persists a config_hash via compute_fingerprint(config, for_pairing=True) — SHA-256 over the 10 pairing-stable fields (strategy_class, instruments, start_date, end_date, initial_balance, strategy_params, timeframe, fill_model, slippage_bps, commission_bps), plus target_execution_exchange when it is set on the config (see "Cross-asset execution" → "Fingerprint + pairing" below — legacy default-None runs keep their exact prior hash). execution_mode, snapshot_as_of, warmup_bars, and buffer_size are explicitly excluded so Direct-DB and ZMQ replay runs on the same config share a hash.

GET /api/backtests?config_hash={hash} returns sibling runs for the auto-pair UI; the repository query is wallet-scoped and indexed on (wallet_public_id, config_hash, timestamp).

Comparison

POST /api/backtests/compare creates (or returns existing) a comparison row; GET /api/backtests/compare/{id} returns the comparison metadata plus the diff **recomputed from current artifact rows** so metric-schema changes never stale a persisted diff. The diff surfaces four shapes:

- metrics_diff — per-name {run_a, run_b, delta, pct} with explicit is not None precedence.
- equity_overlay — full-outer-joined equity samples on point_time with nullable per-leg fields.
- trades_diff — multiset matching on (instrument, executed_at, side, quantized_qty, quantized_price) at 1e-8 precision so IEEE drift collapses but real ticks stay distinct. Common entries carry pnl_a/pnl_b/pnl_delta.
- signals_diff — multiset on (instrument, signal_time, signal_type).

Route ordering: compare endpoints are registered **before** /{run_id} so /compare is not captured as id="compare". Pair is normalised to lexical (min, max) before insert.

Duplicate submit is idempotent via SELECT-then-INSERT with an IntegrityError rollback+re-SELECT race recovery path.

No-active-wallet fail-closed

Every backtest read endpoint (list, detail, trades, signals, events, equity, compare) returns **400 no active wallet selected** when `principal.active_wallet_public_id` is `None`. WS subscribes return `topic_denied`. The picker triggers a `selectWalletAndRefresh` on change that mints a new JWT carrying the chosen wallet claim before swapping the client scope, so REST and WS both authorise against the same wallet after the picker moves.

Cross-asset execution

Cross-asset strategies observe market data on one venue / instrument and emit signals that execute on a *different* venue / instrument (for example `TradFiObserveCryptoExecute` observes MNQU6-CME candles on `kraken_equities` and emits BUY/SELL signals on BTC-USD / `kraken`). The backtest engine supports this through a single config field:

```
BacktestConfig(  
    strategy_class="TradFiObserveCryptoExecute",  
    instruments={  
        "kraken_equities": ["<MNQU6-CME-public-id>"],  
        "kraken": ["<BTC-USD-public-id>"],  
    },  
    target_execution_exchange="kraken",  
    start_date=..., end_date=...,  
    wallet_public_id=...,  
    initial_balance=10_000.0,  
    strategy_params={"fast_period": 12, "slow_period": 26, "min_candles": 30},  
)
```

Attribution semantics

When `target_execution_exchange` is not `None`, the batch processor substitutes at simulated-fill time:

- `target_exchange = str(config.target_execution_exchange)` — carries the *venue label* only. It is NOT validated against the feed populating `latest_closes[target_instrument]`; the caller is responsible for configuring instruments so the target symbol's feed matches the target venue.
- `target_instrument = signal.instrument` — comes directly from the strategy's `StrategySignal`, unchanged since pre-cross-asset.
- `target_close = latest_closes[target_instrument]` — the *price* at which the simulated fill executes. The source feed's close price is no longer used on cross-asset runs.

When `target_execution_exchange` is `None` (default), the fill is attributed to `event.exchange + signal.instrument`. In legacy single-feed strategies `signal.instrument == event.instrument` by convention (verified across every in-tree emitter at `rsi.py`,

macd.py, cointegration.py), so the recorded exchange / instrument / price are byte-identical with the pre-v1.2 path — every existing single-feed backtest keeps its exact fingerprint and result rows.

Missing-target-close policy

If signal.instrument has no entry in latest_closes (typically during the warmup overlap window where the strategy fires before the target feed has primed), the engine:

1. Skips simulate_market_fill + the trade row (no fill attempted).
2. Increments collector.cross_asset_blocked_fills by one, logged at DEBUG with reason='missing_target_close'.
3. Still records the signal row with price = float(signal.price) (source-close fallback) so downstream analytics see the strategy's intent.

At run end the counter surfaces through

BacktestResultInsertRow.extra_metrics["cross_asset_blocked_fills"] *only when positive*.

Single-feed runs keep extra_metrics == {} byte-identical with pre-cross-asset behaviour.

Scope limits

Three explicit non-goals in the current implementation:

- **Same-symbol multi-venue** — Portfolio.positions is keyed by instrument only, so running BTC-USD on Kraken Spot + Kraken Futures simultaneously in one backtest is NOT supported. Cross-asset strategies in scope use distinct symbols across feeds.
- **Multi-target cross-asset** — a single target_execution_exchange per run means one strategy cannot emit signals for different target venues inside the same backtest.
- **Public REST create path** — POST /api/backtests accepts an optional target_execution_exchange field on BacktestCreateBody. When set, simulated fills are attributed to that order-capable venue while candles still feed from exchange. When unset, the run stays single-exchange and byte-identical to legacy behaviour. The same field round-trips through DB persistence + the rerun endpoint, and surfaces on BacktestRunData for frontend display. Allowed target values: paper / kraken / kraken_futures / walutomat.

Fingerprint + pairing

compute_fingerprint includes target_execution_exchange in the payload **only when non-None**, so legacy (default-None) runs keep their exact pre-cross-asset hash in both the default and for_pairing=True paths. This preserves dedup cache validity and the auto-pair UI grouping logic at backtest_routes.py:348. Explicit cross-asset runs (field set to a concrete venue like "kraken") generate distinct fingerprints so they never collide with single-venue baselines.

CLI

Snapper provides a command-line interface built on the Typer framework. All commands are available through the snapper command.

Usage

```
snapper [COMMAND] [OPTIONS]
```

Help for a specific command:

```
snapper [COMMAND] --help
```

Server and Infrastructure

server

Starts the FastAPI server with the web dashboard.

```
snapper server [OPTIONS]
```

Options:

Option	Type	Default	Description
--host	string	from env	Listen address
--port	int	from env	Server port
--reload	bool	from env	Auto-reload for development

Environment:

Set `SERVER_API_ONLY=true` to skip process autostart while keeping the ZMQ-WebSocket bridge alive (useful when the engine runs as a separate process). Multi-worker uvicorn (multiple FastAPI processes sharing a broker) is supported — see the AI-review fanout dedup contract in docs/architecture.md (Deployment Modes); set `SNAPPER_COORDINATOR_INSTANCE_ID` + `SNAPPER_COORDINATOR_INSTANCE_COUNT` per worker (env vars only — snapper server does not expose per-worker CLI flags; the --

instance-id / --instance-count flags in the table below are on snapper trade-zmq) to enable the shared partitioning that prevents duplicate caps-violation WS frames under $N > 1$.

Examples:

```
# Start with default settings (all-in-one dev mode)
snapper server

# Start on all interfaces
snapper server --host 0.0.0.0 --port 8000

# Development mode with auto-reload
snapper server --reload

# API-only mode (engine runs separately)
SERVER_API_ONLY=true snapper server
```

broker

Starts the ZeroMQ XPUB/XSUB broker for message routing.

```
snapper broker [OPTIONS]
```

Options:

Option	Type	Default	Description
--xsub	string	from env	XSUB endpoint (publishers)
--xpub	string	from env	XPUB endpoint (subscribers)

Example:

```
snapper broker --xsub tcp://127.0.0.1:7500 --xpub tcp://127.0.0.1:7501
```

trade-zmq

Starts the central trade runtime coordinator.

The process still runs as the trade-zmq command, but after the trade runtime redesign it also hosts TradeService, BalanceService, the outbox dispatcher, and the reconciliation loops. It writes TradeCommand rows, consumes orders.events.* to keep projections in sync, and persists checkpoints for recovery.

```
snapper trade-zmq [OPTIONS]
```

Options:

Option	Type	Default	Description
--signal-topics	string	signals.	Signal topics to subscribe
--instance-id	int	0 (or \$SNAPPER_COORDINATOR_INSTANCE_ID)	Multi-instance partitioning: zero-based id for this coordinator in a multi-instance deployment. CLI flag > env var > default.
--instance-count	int	1 (or \$SNAPPER_COORDINATOR_INSTANCE_COUNT)	Multi-instance partitioning: total coordinator instance count across the deployment. Must match across all coordinators.

Example:

```
snapper trade-zmq --signal-topics "signals.paper.BTC-USD.rsi_btc_1h,signals.kraken.BTC-USD.live"
```

Multi-instance deployment:

```
# Coordinator 0 of 2
snapper trade-zmq --instance-id 0 --instance-count 2

# Coordinator 1 of 2 (separate process, same DB + broker)
snapper trade-zmq --instance-id 1 --instance-count 2
```

Each instance owns $\sim 1/N$ of the shard_keys via deterministic SHA-256 hashing. Default --instance-count 1 is byte-identical to single-coordinator behavior (every shard owned by the single coordinator). See [docs/operations.md](#) for systemd template unit + scale-up/down/crash-recovery runbooks.

executor

Starts the order execution service for a selected exchange.

```
snapper executor [OPTIONS]
```

Options:

Option	Type	Default	Description
-e, --exchange	string	kraken	Exchange (kraken, walutomat)

Examples:

```
# Kraken executor
snapper executor -e kraken

# Walutomat executor
snapper executor --exchange walutomat
```

feed

Starts the market data publisher via WebSocket.

```
snapper feed [OPTIONS]
```

Options:

Option	Type	Default	Description
--symbols	string	BTC-USD	Comma-separated symbols

Example:

```
snapper feed --symbols "BTC-USD,ETH-USD,SOL-USD"
```

zmq-logger

Monitors and logs ZMQ message traffic.

```
snapper zmq-logger [OPTIONS]
```

Options:

Option	Type	Default	Description
--payload/--no-payload	bool	false	Log full payload
--file/--no-file	bool	true	Write to audit file
--audit-file	string	data/zmq_audit.jsonl	Audit file path
--max-length	int	200	Max payload preview length

Example:

```
snapper zmq-logger --payload --audit-file logs/zmq.jsonl
```

Database

db-init

Initializes the database schema.

```
snapper db-init
```

db-upgrade

Runs Alembic migrations to a specified revision.

```
snapper db-upgrade [OPTIONS]
```

Options:

Option	Type	Default	Description
--revision	string	head	Target revision

Examples:

```
# Migrate to latest version
snapper db-upgrade

# Migrate to specific revision
snapper db-upgrade --revision abc123
```

db-downgrade

Rolls back Alembic migrations.

```
snapper db-downgrade [OPTIONS]
```

Options:

Option	Type	Default	Description
--revision	string	-1	Steps to roll back

Examples:

```
# Roll back one migration
snapper db-downgrade

# Roll back to specific revision
snapper db-downgrade --revision base
```

db-seed

Seeds the database with environment-specific data from TOML profiles. Automatically run by make migrate-dev and make migrate-prod.

```
snapper db-seed [OPTIONS]
```

Options:

Option	Type	Default	Description
--profile	string	dev	Seed profile name (e.g. dev, prod)

Users

init-admin

Creates the initial administrator user.

```
snapper init-admin [OPTIONS]
```

Options:

Option	Type	Default	Description
--username	string	admin	Username
--password	string	AdminSnapper2026!	Password

Example:

```
snapper init-admin --username admin --password SuperSecurePass123!
```

list-users

Displays a list of all system users.

```
snapper list-users
```

reset-password

Resets a user's password.

```
snapper reset-password USERNAME [OPTIONS]
```

Arguments:

- USERNAME — Username (required)

Options:

Option	Type	Default	Description
--new-password	string	(prompt)	New password

Example:

```
snapper reset-password john --new-password NewSecurePass123!
```

Symbol Synchronization

update-kraken-symbols

Synchronizes symbol mappings from the Kraken API.

```
snapper update-kraken-symbols [OPTIONS]
```

Options:

Option	Type	Default	Description
-f, --force	bool	false	Force update

update-walutomat-symbols

Synchronizes symbol mappings from the Walutomat API.

```
snapper update-walutomat-symbols [OPTIONS]
```

Options:

Option	Type	Default	Description
-f, --force	bool	false	Force update

update-polygon-symbols

Synchronizes symbol mappings from the Polygon.io API.

```
snapper update-polygon-symbols [OPTIONS]
```

Options:

Option	Type	Default	Description
-f, --force	bool	false	Force update
--insert-new	bool	false	Insert new symbols

Note: This operation may take 10-15 minutes (44k+ symbols).

update-underlyings

Syncs underlying asset definitions from YAML to the database. Matches active instruments to underlyings via pattern rules, upserts mappings, and applies YAML-fallback instrument_type/expiry_override to InstrumentSpec when API-sourced values are NULL.

```
snapper update-underlyings [OPTIONS]
```

Option	Type	Default	Description
--force	bool	false	Bypass safety guard for stale cleanup

Market Snapshots

update-kraken-market-snapshot

Updates Kraken market snapshots with current prices.

```
snapper update-kraken-market-snapshot
```

update-walutomat-market-snapshot

Updates Walutomat market snapshots with current prices.

```
snapper update-walutomat-market-snapshot
```

Data Backfill

polygon-backfill-aggregates

Fetches historical aggregated data from the Polygon.io API.

```
snapper polygon-backfill-aggregates [OPTIONS]
```

Options:

Option	Type	Default	Description
-s, --symbol	string[]	from settings	Symbols to backfill
--all	bool	false	All mapped symbols
-m, --multiplier	int	1	Timeframe multiplier
-t, --timespan	string	minute	Timespan (minute, hour, day)
-d, --days	int	from settings	Days back
--resume/--no-resume	bool	true	Resume from last timestamp
--csv/--no-csv	bool	true	Save to CSV.gz files

Examples:

```
# Backfill for specific symbols
snapper polygon-backfill-aggregates -s AAPL -s MSFT --days 365

# Backfill all mapped symbols
snapper polygon-backfill-aggregates --all --timespan day

# Hourly backfill with resume
snapper polygon-backfill-aggregates -s SPY -t hour -m 1 --resume
```

polygon-backfill-grouped

Fetches grouped daily data from the Polygon.io API.


```
snapper polygon-backfill-grouped [OPTIONS]
```

Data Archive

archive

Export data to CSV archive files. Supports candle cache projection and append-only event tables (ticks, trades, signals, executions, telemetry, control). Merges with existing files and deduplicates rows.

```
snapper archive [OPTIONS]
```

Options:

Option	Type	Default	Description
--table	TEXT	candles	Table to archive (candles, candles-audit, ticks, trades, signals, executions, telemetry, control)
--exchange	TEXT	None	Exchange filter (e.g. polygon, kraken)
--symbol	TEXT	None	Native symbol filter (e.g. BTC-USD), resolved to stable archive_symbol
--timeframe	TEXT	1m	Candle timeframe (1m, 5m, 1h, 1d) — candles only
--day	DATE	None	Single day to archive (YYYY-MM-DD)
--from	DATE	None	Start of date range (YYYY-MM-DD)
--to	DATE	None	End of date range (YYYY-MM-DD)
--dry-run	FLAG	False	Report counts without writing files
--purge	FLAG	False	Delete exported rows from DB after writing (event tables and candles-audit with --closed-only)
--closed-only	FLAG	False	Only closed SCD2 versions (candles-audit only)
--output-dir	PATH	data	Base output directory

Output structure:

```
Candles: {output_dir}/{exchange}/cache/{timespan}/{archive_symbol}/{year}/{date}.csv
Events:  {output_dir}/archive/{table}/{exchange}/{archive_symbol}/{year}/{date}.csv
Flat:    {output_dir}/archive/{table}/{year}/{date}.csv (telemetry, control)
```

Examples:

```
snapper archive --day 2024-03-15 --exchange polygon --symbol BTC-USD
snapper archive --from 2024-01-01 --to 2024-03-31 --dry-run
snapper archive --day 2024-03-15 --timeframe 1d
snapper archive --table ticks --day 2024-03-15 --exchange polygon
snapper archive --table trades --from 2024-01-01 --to 2024-03-31 --purge
snapper archive --table control --day 2024-03-15
snapper archive --table candles-audit --day 2024-03-15 --timeframe 1m
snapper archive --table candles-audit --day 2024-03-15 --closed-only --purge
snapper archive --table orders --day 2024-03-15
snapper archive --table symbols --day 2024-03-15 --closed-only --purge
snapper archive --table instruments --day 2024-03-15
```

Notes:

- --symbol resolves the current native_symbol to the stable archive_symbol via the Symbol anchor row. After a symbol rename, the CLI argument and the output directory may differ.
- --day or --from/--to is required.
- Daily timespan (1d) produces monthly CSV files to match Polygon layout.
- --purge is only supported for event tables, not candle cache export.
- Event archive CSV files include full temporal metadata (public_id, timestamp, known_to, session_id, sequence_id).
- Merge/dedup for events uses (public_id, timestamp, known_to) as key.
- Executions are partitioned by exchange/archive_symbol via order -> instrument.
- candles-audit exports all SCD2 versions (closed + active) grouped by open_at date. --purge requires --closed-only to protect active rows.

restore

Restore archived CSV data back into the database. Reads CSV files exported by snapper archive and inserts rows, deduplicating against existing data by (public_id, timestamp, known_to).

```
snapper restore [OPTIONS]
```

Options:

Option	Type	Default	Description
--table	TEXT	(required)	Table name to restore into
--file	PATH	None	Single CSV file to restore
--dir	PATH	None	Directory to scan recursively for CSV files
--source	TEXT	audit	Restore mode (audit for full history)

Examples:

```
snapper restore --table ticks --file data/archive/ticks/polygon/BTC-USD/2024/2024-01-01.csv
snapper restore --table settings --dir data/archive/settings/
snapper restore --table candles --dir data/archive/candles/polygon/BTC-USD/
```

Notes:

- At least one of --file or --dir is required.
- Rows already present in DB (matching public_id + timestamp + known_to) are skipped.
- Audit restore inserts all temporal columns exactly as exported.

One-time cache directory migration

Renames legacy Polygon cache directories from API ticker format (X_BTCUSD) to archive_symbol format (BTC-USD).

```
python -m scripts.migrate_polygon_cache_dirs [--dry-run] [--cache-root PATH]
```

Encryption Management

settings-rotate-encryption

Rotates encryption keys for settings in the database.

```
snapper settings-rotate-encryption [OPTIONS]
```

Options:

Option	Type	Default	Description
--new-password	string	(required)	New master password

Option	Type	Default	Description
--old-password	string	from env	Current master password (optional, reads from env/bootstrap)
--dry-run	bool	false	Show changes without applying

Example:

```
# Simulate rotation
snapper settings-rotate-encryption --new-password NewSecurePass --dry-run

# Actual rotation
snapper settings-rotate-encryption --new-password NewSecurePass
```

Example Workflows

New Installation Initialization

```
# 1. Initialize database
snapper db-init

# 2. Create administrator
snapper init-admin --username admin --password SecretPassword123!

# 3. Synchronize symbols
snapper update-kraken-symbols
snapper update-polygon-symbols

# 4. Start server
snapper server
```

Full Trading System Startup

```
# Terminal 1: ZMQ Broker
snapper broker

# Terminal 2: Data feed
snapper feed --symbols "BTC-USD,ETH-USD"

# Terminal 3: Executor
snapper executor -e kraken

# Terminal 4: Trade runtime
snapper trade-zmq

# Terminal 5: Server with dashboard
SERVER_API_ONLY=true snapper server
```

Historical Data Backfill

```
# Backfill last 30 days for SPY
snapper polygon-backfill-aggregates -s SPY --days 30 --timespan minute

# Backfill daily data for all symbols
snapper polygon-backfill-aggregates --all --timespan day --days 365
```

Backtesting

backtest-run

Run a backtest synchronously. Creates a DB record, executes the engine in-process, and persists results (signals, trades, equity, metrics).

```
snapper backtest-run \
  --strategy sma_cross \
  --instrument BTC-USD \
  --exchange kraken \
  --start 2026-01-01 \
  --end 2026-06-01 \
  --timeframe 1h \
  --initial-cash 10000 \
  --params '{"fast": 10, "slow": 30}'
```

Options:

Option	Type	Default	Description
--strategy	str	required	Registered strategy class name
--instrument	str	required	Instrument public ID
--exchange	str	required	Exchange name
--start	str	required	Start date (ISO format)
--end	str	required	End date (ISO format)
--timeframe	str	1h	Candle timeframe
--initial-cash	float	10000	Starting cash balance
--wallet	str	cli	Wallet public ID
--params	str	{}	Strategy params as JSON

backtest-list

List backtest runs with optional filters.

```
snapper backtest-list --status completed --limit 10
```

backtest-cancel

Cancel a pending or running backtest run.

```
snapper backtest-cancel <run-public-id>
```

backtest-rerun

Re-run a backtest with the same configuration (including strategy params).

```
snapper backtest-rerun <run-public-id>
```

Configuration

Snapper's configuration system is based on two layers: environment variables (bootstrap) and database settings.

Environment Variables

Settings loaded from .env file or environment variables.

Database

Variable	Default	Description
DB_URL	sqlite+aiosqlite:///./data/snapper.db	SQLAlchemy connection URL

Database URL examples:

```
# SQLite (development default)
DB_URL=sqlite+aiosqlite:///./data/snapper.db

# PostgreSQL (production)
DB_URL=postgresql+asyncpg://user:password@localhost:5432/snapper
```

Encryption

Variable	Default	Description
MASTER_PASSWORD	snapper_default_master_password_v1	Password for encrypting settings in database (salt derived automatically)

Important: Change default values in production environment.

HTTP Server

Variable	Default	Description
SERVER_HOST	127.0.0.1	Listen address
SERVER_PORT	8000	Server port
SERVER_RELOAD	false	Auto-reload for development

Variable	Default	Description
SERVER_PROXY_HEADERS	true	Enable proxy header parsing in uvicorn
SERVER_FORWARDED_ALLOW_IPS	127.0.0.1	Trusted proxy IPs/CIDRs for forwarded headers
SERVER_API_ONLY	false	Skip process autostart; serve API + WS bridge only (separate-engine boots). Multi-worker uvicorn (multiple FastAPI processes sharing a broker) is supported — see docs/architecture.md Deployment Modes for the AI-review fanout dedup contract under N>1. Set SNAPPER_COORDINATOR_INSTANCE_ID + SNAPPER_COORDINATOR_INSTANCE_COUNT per worker to enable the shared partitioning.
TELEMETRY_RECORDING_ENABLED	false	Record pings, heartbeats, and GET reads to telemetry table. High volume — enable for debugging only

ZeroMQ

Variable	Default	Description
ZMQ_BROKER_XSUB	tcp://127.0.0.1:7500	XSUB endpoint (publishers connect here)
ZMQ_BROKER_XPUB	tcp://127.0.0.1:7501	XPUB endpoint (subscribers connect here)

Database Settings

Sensitive data is stored encrypted in the settings table. Wallet-scoped exchange credentials (api keys, secrets, PEM material) live in a separate wallet_credentials table — see [Wallet Credentials](#) below. Shared market-data provider keys stay in settings.

Market Data API Keys

Key	Description
polygon_api_key	Polygon.io API key (shared market data — not wallet-scoped)

Wallet Credentials

Per-exchange trading credentials live in the `wallet_credentials` table and are loaded by `CredentialResolver` during per-wallet executor startup. Each row carries:

Column	Description
<code>wallet_public_id</code>	UUID of the owning wallet ((label, is_paper) unique in wallets table)
<code>exchange</code>	Exchange identifier (kraken, kraken_futures, walutomat, paper)
<code>credential_type</code>	Envelope shape: <code>api_key_secret</code> , <code>rsa_pem</code> , <code>oauth</code> , or <code>paper</code>
<code>encrypted_payload</code>	Fernet-encrypted JSON envelope (shape depends on <code>credential_type</code>)
<code>label</code>	Human-readable description of the credential row

Envelope shapes by `credential_type`:

- `api_key_secret` — `{"api_key": "...", "api_secret": "..."} (Kraken, Kraken Futures)`
- `rsa_pem` — `{"api_key": "...", "private_key_pem": "..."} (Walutomat)`
- `paper` — `{"initial_balance": "10000.0"} (paper wallets)`

Seed profiles (`dev.toml` / `prod.toml`) are the current mechanism for populating `wallet_credentials`. A runtime credential management UI (list / add / rotate / delete per-wallet credentials, with automatic executor restart on rotation) is on the frontend roadmap. Until the UI ships, rotation requires editing the seed file and running the seed command against a clean database (seed is idempotent — it skips wallets that already exist), or direct SQL surgery on the encrypted payload.

Seed file structure:

```
[[wallets]]
label = "default"
is_paper = false

[[wallets.credentials]]
exchange = "kraken"
credential_type = "api_key_secret"
api_key = "your-kraken-api-key"
api_secret = "your-kraken-api-secret"

[[wallets.credentials]]
exchange = "walutomat"
credential_type = "rsa_pem"
api_key = "your-walutomat-api-key"
private_key_pem_base64 = "your-pem-base64-encoded"
```

Two wallets named `default` — one with `is_paper=true` and one with `is_paper=false` — coexist via the (label, is_paper) unique index. The seed loader encrypts every payload with the master-password Fernet key before insert.

Trading Parameters

Key	Default	Description
instruments	{"kraken": ["BTC-USD", ...], ...}	Instruments per exchange (dict)
timeframes	["1m"]	Candle timeframes (list)
risk_max_leverage	1.0	Maximum leverage
risk_r_per_trade	0.005	Risk per trade (0.5%)

The trade runtime always uses the durable (outbox-driven) dispatch path. The engine writes TradeCommand rows to the database; the OutboxDispatcher picks them up and publishes to ZMQ. Executor-side VenueEvent writes are fail-closed — a failed persist raises before the executor acknowledges the venue event.

Publisher Micro-Batch

Key	Default	Description
write_buffer_flush_ms	50	Flush age threshold in ms (cached at start)
write_buffer_candle_max_rows	100	Candle batch size trigger
write_buffer_tick_max_rows	500	Tick batch size trigger
write_buffer_trade_max_rows	500	Trade batch size trigger

These settings are cached when the publisher starts and require a process restart to take effect.

Authentication

Key	Default	Description
auth_secret_key	change-me-in-production-use-openssl-rand-hex-32	JWT signing secret
csrf_secret_key	change-me-in-production-csrf-key	CSRF token signing secret
auth_algorithm	HS256	JWT signing algorithm
auth_access_token_expire_minutes	15	Access token lifetime
auth_refresh_token_expire_days	7	Refresh token lifetime

Key	Default	Description
auth_refresh_token_expire_days_extended	30	Extended refresh token lifetime
ws_token_ttl_seconds	900	WebSocket token lifetime
csrf_token_expire_minutes	60	CSRF token lifetime
session_secure	false	Require HTTPS for session cookies
session_same_site	lax	Session cookie SameSite mode
session_domain	""	Session cookie domain override
ui_origin	""	Additional allowed UI origins for CORS and WS origin validation

.env.example File

```

# Database connection
DB_URL=sqlite+aiosqlite:///./data/snapper.db

# Encryption settings for sensitive data (API keys, secrets)
# IMPORTANT: Change this default in production!
MASTER_PASSWORD=snapper_default_master_password_v1

# Server settings
SERVER_HOST=0.0.0.0
SERVER_PORT=8000
SERVER_RELOAD=false
SERVER_API_ONLY=false
SERVER_PROXY_HEADERS=true
SERVER_FORWARDED_ALLOW_IPS=127.0.0.1,172.17.0.1

# ZMQ broker settings
ZMQ_BROKER_XSUB=tcp://127.0.0.1:7500
ZMQ_BROKER_XPUB=tcp://127.0.0.1:7501

```

Accessing Configuration in Code

Bootstrap settings (without database)

```
from snapper.config.settings import get_settings

settings = get_settings()
db_url = settings.db_url
host = settings.server_host
```

With database access

```
from snapper.config.settings import get_settings_with_service

settings = get_settings_with_service(settings_service)
polygon_key = settings.polygon_api_key # Decrypted from database
```

Wallet-scoped exchange credentials do NOT appear on AppSettings. They are loaded by CredentialResolver during per-wallet executor startup:

```
from snapper.config.credentials import CredentialResolver

resolver = CredentialResolver(repository)
envelope = await resolver.get_credentials(
    exchange="kraken",
    wallet_public_id="01975a8b-3c7d-7000-8000-0000000000a1",
)
# envelope == {"api_key": "...", "api_secret": "..."}
```

Managing Settings via API

List Settings

```
GET /api/settings
GET /api/settings?category=exchange
```

List Categories

```
GET /api/settings/categories
```

Set Setting

```
POST /api/settings/{key}/set
X-CSRF-Token: <csrf_token>
Content-Type: application/json
```

```
{
  "public_id": "<uuid7>",
  "session_id": "<client-session>",
  "sequence_id": 1,
  "timestamp": "2026-01-18T12:00:00Z",
  "payload": {
    "value": "new_value",
    "category": "system",
    "description": "optional description"
  }
}
```

Remove Setting

```
POST /api/settings/{key}/remove
X-CSRF-Token: <csrf_token>
Content-Type: application/json

{
  "public_id": "<uuid7>",
  "session_id": "<client-session>",
  "sequence_id": 2,
  "timestamp": "2026-01-18T12:00:01Z",
  "payload": {}
}
```

Environment Configuration

Development

```
# .env.development
DB_URL=sqlite+aiosqlite:///./data/snapper.db
SERVER_HOST=127.0.0.1
SERVER_PORT=8000
SERVER_RELOAD=true
SERVER_PROXY_HEADERS=true
SERVER_FORWARDED_ALLOW_IPS=127.0.0.1
```

Production

```
# .env.production
DB_URL=postgresql+asyncpg://user:pass@db.example.com/snapper
MASTER_PASSWORD=<strong_random_password>
SERVER_HOST=0.0.0.0
SERVER_PORT=8000
SERVER_RELOAD=false
SERVER_PROXY_HEADERS=true
SERVER_FORWARDED_ALLOW_IPS=127.0.0.1,172.17.0.1
```

Docker

Variables can be passed via docker-compose.yml:

```
services:
  snapper:
    env_file: .env
    environment:
      - DB_URL=postgresql+asyncpg://snapper:password@postgres/snapper
```

Settings Encryption

Sensitive settings are encrypted with Fernet (AES-128-CBC + HMAC-SHA256):

1. Salt is derived from MASTER_PASSWORD via SHA-256
2. Encryption key is derived via PBKDF2-HMAC-SHA256 (100,000 iterations)
3. Values are encrypted with Fernet and stored in database

If MASTER_PASSWORD changes, encrypted settings become inaccessible. Use snapper settings-rotate-encryption to rotate passwords safely.

Configuration Validation

The system validates configuration at startup:

- Database connection check
- ZMQ endpoint verification
- Settings decryption test

Configuration errors are reported through exceptions with clear messages.

Development

Guidelines for developers working on the Snapper project.

Requirements

- Python 3.14+
- Poetry
- Node.js 25+ and pnpm
- TA-Lib (C library)
- Pre-commit hooks

Environment Setup

```
# Install system dependencies (macOS)
make system-deps

# Install Python dependencies
make setup

# Install frontend dependencies
make ui-setup

# Synchronize pre-commit hooks
make pre-refresh

# Refresh dependencies, local tools, and Dockerfile tool pins
make update
```

Quality Gates

Main Command

```
make check-all
```

Executes the complete quality gate:

1. Formatting (ruff, black, isort)
2. Linting (ruff)
3. Type checking (mypy)
4. Docstring compliance

5. No # comments in Python
6. Canonical __main__ guards
7. Empty __init__.py files
8. No forbidden temporal mutations
9. No pragma/noqa/ignore exclusions
10. Tests with 100% coverage
11. Frontend (ESLint, Prettier, dead code, tests with coverage)

Individual Steps

Command	Description
make fmt	Check formatting
make fmt-fix	Fix formatting
make lint	Ruff linting
make lint-fix	Fix linting issues
make typecheck	Mypy type checking
make test	Tests (parallel)
make test-serial	Tests (sequential, debug)
make cov	Tests with coverage

Automatic Fixes

```
make fix          # Backend: fmt-fix + lint-fix + move-imports
make fix-all      # Backend + frontend
```

Code Standards

Types (MANDATORY)

All functions must have type annotations:

```
def calculate_rsi(prices: pd.Series, period: int = 14) -> pd.Series:
    """Calculate RSI indicator."""
    ...
```



```
async def fetch_data(symbol: str) -> list[Candle]:
    """Fetch candle data."""
    ...
```

Test functions must have -> None:

```
async def test_rsi_calculation() -> None:
    """Test RSI calculation returns expected values."""
    ...
```

Docstrings (Google style)

```
def process_signal(signal: Signal, config: dict[str, Any]) -> Order | None:
    """Process trading signal and generate order.

    Takes a trading signal and configuration, validates the signal,
    and generates an appropriate order if conditions are met.

    Args:
        signal: Trading signal to process.
        config: Strategy configuration dictionary.

    Returns:
        Generated order if valid, None otherwise.

    Raises:
        ValueError: If signal data is invalid.
    """
    ...
```

No # Comments

comments are forbidden. Use docstrings:

```
# WRONG
def foo():
    # This calculates something
    x = 1 + 1

# CORRECT
def foo():
    """Calculate the sum."""
    x = 1 + 1
```

Imports

Imports at the top of file, single lines:

```

from datetime import UTC
from datetime import datetime
from typing import Any

import pandas as pd
from sqlalchemy import select

from snapper.config.settings import get_settings
from snapper.data.models import Candle

```

Forbidden:

- from __future__ import annotations
- if TYPE_CHECKING: blocks
- Re-exports in __init__.py

__init__.py

__init__.py files must be empty (or docstring only):

```

"""Snapper strategies module."""

```

Tests

Structure

```

tests/
├── conftest.py           # Shared fixtures
├── api/                  # API tests
├── application/          # Business logic tests
├── cli/                  # CLI tests
├── data/                 # Data layer tests
├── indicators/           # Indicator tests
├── messaging/            # ZMQ tests
├── strategies/           # Strategy tests
└── ...

```

Writing Tests

```

import pytest
from snapper.indicators.rsi import rsi

class TestRSI:
    """Test suite for RSI indicator."""

    def test_rsi_returns_series_with_correct_length(self) -> None:
        """

```

```

    """
    Given a price series of length N
    When RSI is calculated with period P
    Then the result has length N.
    """
    prices = pd.Series([100, 102, 101, 103, 105])
    result = rsi(prices, period=14)
    assert len(result) == len(prices)

def test_rsi_values_in_valid_range(self) -> None:
    """
    Given any price series
    When RSI is calculated
    Then all values are between 0 and 100.
    """
    prices = pd.Series([100, 102, 101, 103, 105, 104, 106])
    result = rsi(prices, period=3)
    assert (result >= 0).all()
    assert (result <= 100).all()

```

Async Tests

```

import pytest

@pytest.mark.asyncio
async def test_fetch_candles() -> None:
    """Test fetching candles from repository."""
    repo = get_repository(db_url)
    candles = await repo.get_candles("BTC-USD", limit=10)
    assert len(candles) <= 10

```

Fixtures

```

@pytest.fixture
def sample_candles() -> list[Candle]:
    """Provide sample candle data for tests."""
    return [
        Candle(open=100, high=105, low=95, close=102, volume=1000),
        Candle(open=102, high=110, low=100, close=108, volume=1200),
    ]

```

100% Coverage

The project requires 100% code coverage (TDD). Check:

```
make cov
```

Configuration in `.coveragerc`.

Frontend

Setup

```
make ui-setup
```

Development

```
make ui-dev
```

Frontend at <http://localhost:3000/> with backend proxy.

Quality Checks

Command	Description
make ui-lint	ESLint
make ui-format	Prettier check
make ui-dead-code	Knip dead code
make ui-test	Vitest tests
make ui-cov	Tests with coverage

Fixes

```
make ui-fix    # lint-fix + format-fix + dead-code-fix
```

Type Generation

```
make ui-gen-types    # OpenAPI types, WebSocket types, Zod schemas, entity  
aliases, permissions
```

Database Migrations

Dev DB Location

The development SQLite database lives at `./data/snapper.db`.

Destructive Migration Strategy (Development)

The project uses a single-file migration (0001_init.py) that is rewritten in place when the schema changes. After editing the migration, reset and reseed:

```
rm -f data/snapper.db && make migrate-dev
```

make migrate-dev applies Alembic migrations (via snapper db-init) then seeds development data (via snapper db-seed --profile dev). There is no standalone make migrate target -- always use make migrate-dev or make migrate-prod.

Applying (Production)

```
make migrate-prod
```

Rollback

```
snapper db-downgrade
```

Docker

Build

```
make docker-build-dev    # Development image  
make docker-build-prod   # Production image
```

Run

```
make docker-run
```

Docker Compose

```
docker compose up -d
```

CI/CD

Pipeline executes the same consolidated gate used locally:

1. make check-all — Backend checks, frontend checks, exclusion scan, and tests with coverage

For debugging a failing pipeline locally, the equivalent steps are:

1. make check — Backend quality checks
2. make ui-check — Frontend lint, format, and dead-code checks
3. make check-exclusions — No pragma/noqa/ignore bypasses
4. make cov — Backend tests with coverage
5. make ui-cov — Frontend tests with coverage

Workflow

Before Commit

```
make check-all
```

Before PR

1. Ensure make check-all passes
2. Add/update tests
3. Update documentation if needed
4. Verify 100% coverage

Code Review Checklist

- [] Types for all functions
- [] Docstrings (Google style)
- [] No # comments
- [] Tests for new functionality
- [] 100% coverage
- [] make check-all passes

Project Structure

```
snapper/
├── src/snapper/           # Source code
│   ├── api/              # API schemas
│   ├── application/      # Business logic
│   ├── auth/             # Authentication
│   ├── cli/              # CLI interface
│   ├── config/           # Configuration
│   ├── core/             # Core types
│   ├── data/             # Data layer
│   ├── indicators/       # Technical indicators
│   ├── infrastructure/   # External integrations
│   ├── interface/        # WebSocket
│   ├── messaging/        # ZeroMQ
│   ├── server/           # FastAPI
│   ├── strategies/       # Trading strategies
│   └── utils/            # Utilities
├── tests/                # Tests
├── frontend/             # React dashboard
├── docs/                 # Documentation
├── scripts/              # Helper scripts
└── data/                 # Local data
```

Tools

Ruff

Lint and formatter. Configuration in pyproject.toml:

```
[tool.ruff.lint]
select = ["E", "F", "W", "I", "B", "UP", "C4", "SIM", "N", "PERF", "ISC",
"PLW", "PLC", "A", "TID", "D"]
```

Mypy

Type checker. Strict mode enabled:

```
[tool.mypy]
python_version = "3.14"
strict = true
```

Black

Formatter. Line length 100:

```
[tool.black]
line-length = 100
```

Pre-commit

Hooks run before commit:

```
pre-commit install
pre-commit run --all-files
```

Debugging

Logging

```
from loguru import logger

logger.info(f"Processing signal: {signal}")
logger.debug(f"Calculated RSI: {rsi_value}")
logger.error(f"Failed to execute order: {e}")
```

ZMQ Logger

```
snapper zmq-logger --payload
```

Serial Tests

```
make test-serial
```

pdb/ipdb

```
import ipdb; ipdb.set_trace()
```

Documentation

Docstrings

Every module, class, and public function must have a docstring.

docs/

Markdown documentation in docs/ directory.

PDF

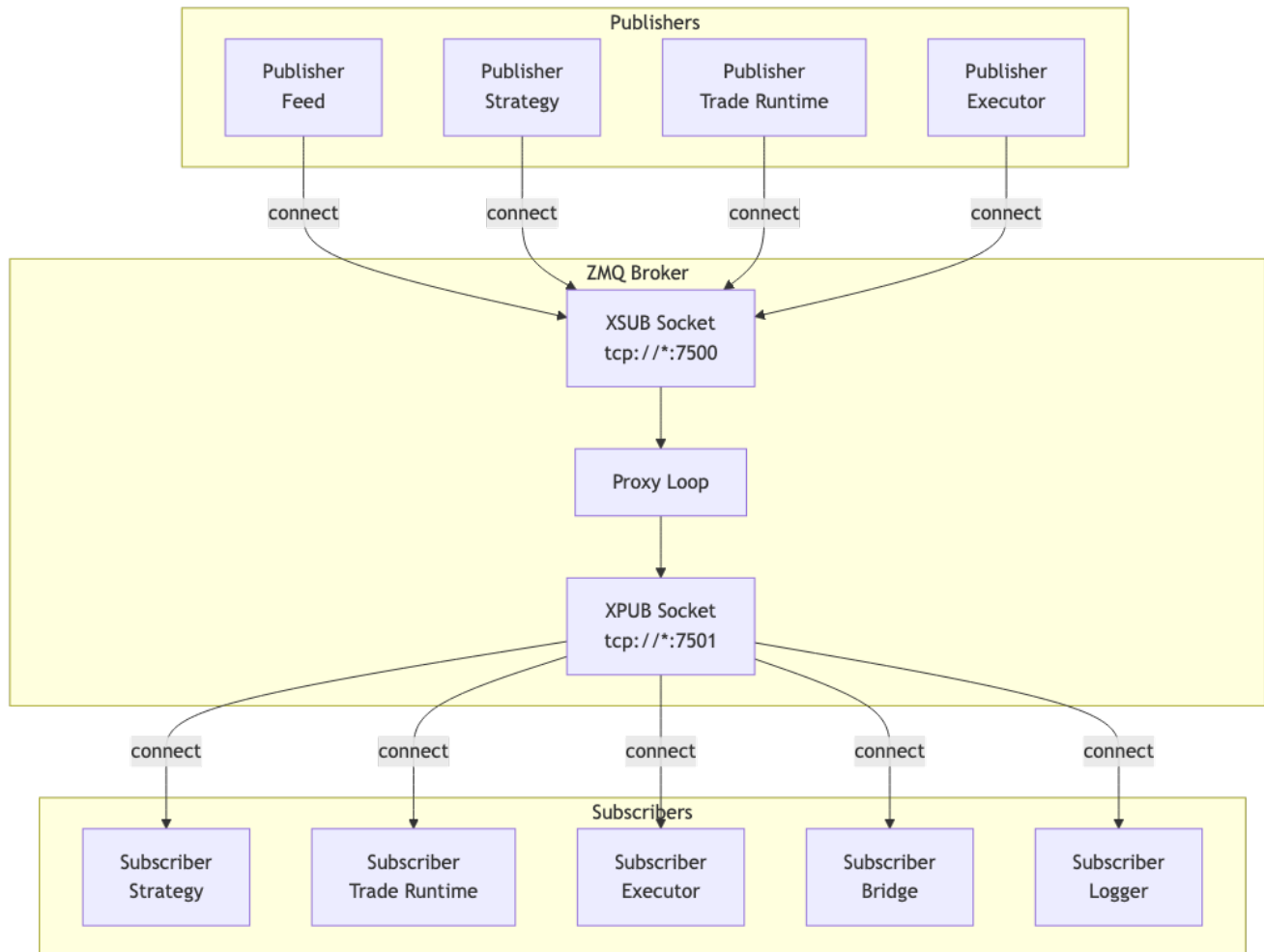
```
make docs-pdf
```

Generates frontend/public/snapper.pdf from README + docs/*.md so the docs ship with the frontend bundle and are linked from the login page.

Messaging (ZeroMQ)

Snapper uses ZeroMQ for inter-process communication in a pub/sub architecture. The central component is an XPUB/XSUB broker that routes messages.

Architecture



XPUB/XSUB Broker

The broker is the central message hub:

- **XSUB** (tcp://*:7500) — Publishers connect here
- **XPUB** (tcp://*:7501) — Subscribers connect here

The broker forwards messages in both directions:

- Data: XSUB → XPUB
- Subscriptions: XPUB → XSUB

Starting the Broker

```
snapper broker
```

Or programmatically:

```
from snapper.messaging.infrastructure.broker import ZmqBrokerThread

broker = ZmqBrokerThread()
broker.start()
# ...
broker.stop()
```

Topics

Topic format varies by category (see per-category tables below).

Market Data

Topic	Description
market.kraken.BTC-USD.candles.1h	Hourly BTC/USD candles from Kraken
market.kraken.BTC-USD.ticks	BTC/USD ticks from Kraken
market.polygon.AAPL.candles.1d	Daily AAPL candles from Polygon

Signals

Topic	Description
signals.paper.BTC-USD.rsi_btc_1h	Paper trading signals
signals.kraken.BTC-USD.live	Live signals from Kraken

Orders & Executions

Topic	Description
orders.commands.kraken.BTC-USD.submit	Order requests for Kraken
orders.events.kraken.BTC-USD.executed	Order executions from Kraken
orders.events.kraken.BTC-USD.submitted	Order submitted events

Topic	Description
orders.events.kraken.BTC-USD.accepted	Order accepted events
orders.events.kraken.BTC-USD.rejected	Order rejected events
orders.events.kraken.BTC-USD.cancelled	Order cancelled events
orders.events.kraken.BTC-USD.expired	Order expired events
orders.events.kraken.BTC-USD.replaced	Order replaced events

TradeCommand, VenueEvent, and TradeProjectionCheckpoint are durable database artifacts used by the trade runtime. They are not additional ZMQ topics.

System

Topic	Description
system.heartbeats.executor.{exchange}	Executor heartbeat for the single-wallet template path (4 segments)
system.heartbeats.executor.{exchange}.{wallet_short}	Per-wallet executor heartbeat (5 segments). wallet_short is the first 12 lowercase hex characters of the wallet UUID
system.heartbeats.strategy.{name}	Strategy heartbeat (e.g. strategy.rsi_btc_1h)
system.heartbeats.feed.{exchange}	Feed heartbeat (e.g. feed.kraken, feed.paper.kraken)
system.settings	Configuration change notifications
system.symbol_aliases	Symbol cache invalidation
system.replay.start	Data replay start
system.replay.end	Replay end

Heartbeat component values use dot notation matching the topic path after system.heartbeats.: executor.kraken, executor.kraken.019d6ca45f2e, strategy.rsi_btc_1h, feed.kraken, feed.paper.kraken. The per-wallet executor heartbeat envelope additionally carries meta.wallet_public_id so subscribers that prefix-match the 4-segment parent topic can still disambiguate by reading the payload.

Message Data Classes

All messages use typed Data classes (Pydantic) from `snapper.messaging.schemas.data`. Every Data class inherits from `StrictDataSchema` and carries:

Field	Type	Description
public_id	string	UUID7 external identifier, stable across REST and WS
type	string	Literal type discriminator
timestamp	datetime	Message creation time
session_id	string	UUID7 of the producer process session (required at construction)
sequence_id	int	Monotonic counter per topic within the session (required at construction)

session_id and sequence_id are allocated by producers from a shared SequenceTracker before the payload is constructed. MessagePublisher.send() serializes and forwards the already-complete payload without modifying provenance. Consumers can use these fields to detect message loss and producer restarts.

The old messaging.schemas.messages module still exists but only contains parse_message() and MessageParseError.

TickData

```
from snapper.messaging.schemas.data import TickData

tick = TickData(
    instrument="BTC-USD",
    exchange="kraken",
    bid=41990.0,
    ask=42010.0,
    last=42000.0,
    volume=0.5,
    timestamp=datetime.now(UTC),
)
```

Fields:

Field	Type	Description
type	string	"tick"
public_id	string	UUID7 external identifier
instrument	string	Instrument symbol
exchange	string	Exchange name
bid	float None	Best bid price
ask	float None	Best ask price
last	float None	Last traded price

Field	Type	Description
volume	float	Volume
timestamp	datetime	Timestamp
session_id	string	Producer session (inherited)
sequence_id	int	Monotonic counter per topic or logical stream (inherited)

CandleData

```
from snapper.messaging.schemas.data import CandleData

candle = CandleData(
    instrument="BTC-USD",
    exchange="kraken",
    timeframe="1h",
    open_at=datetime(2026, 1, 18, 12, 0, tzinfo=UTC),
    open=42000.0,
    high=42500.0,
    low=41800.0,
    close=42300.0,
    volume=1234.56,
    timestamp=datetime.now(UTC),
)
```

Fields:

Field	Type	Description
type	string	"candle"
public_id	string	UUID7 external identifier
instrument	string	Symbol
exchange	string	Exchange
timeframe	string	Timeframe (1m, 5m, 1h, etc.)
open_at	datetime	Exchange-provided candle interval start time
open	float	Open price
high	float	High
low	float	Low
close	float	Close price
volume	float	Volume
vwap	float null	Volume-weighted average price (optional)

Field	Type	Description
trades	int null	Number of trades in the candle (optional)
timestamp	datetime	Timestamp

SignalData

```
from snapper.messaging.schemas.data import SignalData

signal = SignalData(
    instrument="BTC-USD",
    exchange="paper",
    side="buy",
    strength=0.85,
    reason="RSI <= 30",
    strategy_name="rsi_btc_1h",
    price=42000.0,
    fired_at=datetime.now(UTC),
)
```

Fields:

Field	Type	Description
type	string	"signal"
public_id	string	UUID7 external identifier
instrument	string	Symbol
exchange	string	Target exchange
side	string	"buy" or "sell"
strength	float	Signal strength 0.0-1.0
reason	string	Reason
strategy_name	string	Strategy name (required for paper exchange signals)
price	float	Price
fired_at	datetime	Domain time when signal was generated
timestamp	datetime	System timestamp

Paper signals (exchange == "paper") require strategy_name to be set. The schema enforces this invariant at construction time so invalid paper signals cannot be created.

OrderRequestData

```
from snapper.messaging.schemas.data import OrderRequestData

order = OrderRequestData(
    instrument="BTC-USD",
    exchange="kraken",
    client_order_id="ord_123",
    side="buy",
    order_type="limit",
    price=42000.0,
    quantity=0.1,
    strategy_id="rsi_btc_1h",
    mode="live",
)
```

ExecutionData

```
from snapper.messaging.schemas.data import ExecutionData

execution = ExecutionData(
    instrument="BTC-USD",
    exchange="kraken",
    client_order_id="ord_123",
    exchange_order_id="KRAKEN-456",
    side="buy",
    price=42000.0,
    size=0.1,
    last_size=0.1,
    last_price=42000.0,
    fee=0.001,
    fee_asset="USD",
    status="filled",
)
```

OrderData

OrderData is the canonical event for order state. It carries the request-time margin metadata (leverage, reduce_only) all the way to subscribers and the REST /orders endpoint, so the frontend can render shorts and reduce-only intent.

```
from snapper.messaging.schemas.data import OrderData

order_status = OrderData(
    instrument="BTC-USD",
    exchange="kraken",
    client_order_id="ord_123",
    exchange_order_id="KRAKEN-456",
    side="sell",
    order_type="limit",
    size=0.1,
    filled_size=0.0,
```



```
status="accepted",
price=42000.0,
leverage=3,
reduce_only=False,
)
```

HeartbeatData

```
from snapper.messaging.schemas.data import HeartbeatData

heartbeat = HeartbeatData(
    session_id="019e1a2b-0000-7000-8000-000000000001",
    sequence_id=1,
    component="zmq_broker",
    status="healthy",
    sequence=1,
    lag_ms=5,
)
```

Publisher

MessagePublisher (recommended)

MessagePublisher sends fully constructed payloads to an explicit stream_key. Producers allocate session_id and sequence_id from a shared SequenceTracker before constructing the payload, then pass both the topic and payload to MessagePublisher.send().

```
import zmq.asyncio
from snapper.messaging.infrastructure.validated_socket import
ValidatedPublisher, apply_hwm, HWM_MARKET_DATA
from snapper.messaging.infrastructure.publisher import MessagePublisher,
SequenceTracker
from snapper.messaging.topics.builders import topic_for_message

ctx = zmq.asyncio.Context()
raw_socket = ctx.socket(zmq.PUB)
apply_hwm(raw_socket, sndhwm=HWM_MARKET_DATA)
raw_socket.connect("tcp://127.0.0.1:7500")

tracker = SequenceTracker()
publisher = MessagePublisher(ValidatedPublisher(raw_socket), tracker)

topic = topic_for_message(candle_data)

await publisher.send(topic, candle_data)
publisher.close()
```

For brevity, the example assumes candle_data was already constructed with session_id and sequence_id allocated from tracker.

One SequenceTracker per component process; all MessagePublisher instances within that component share it. Counters survive socket reconnects — only a full component restart creates a new session.

Per-Topic Counters

SequenceTracker maintains monotonic counters keyed by **ZMQ topic** (e.g. market.kraken.BTC-USD.candles.1m, orders.events.kraken.BTC-USD.executed). Each topic has its own independent counter. Producers allocate provenance using the final ZMQ topic string and then pass that same topic to MessagePublisher.send().

For DB-only writes that do not flow over ZMQ (instruments, users, settings seed data), the counter key is the destination table name as a logical identifier.

For WS control and REST middleware paths, logical channel names are used: server.control, server.telemetry, rest.control, rest.telemetry.

For paper market data or other cases where the topic cannot be derived from the payload alone, pass an explicit topic override:

```
topic = "market.paper.kraken.BTC-USD.ticks"
await publisher.send(topic, tick_data)
```

ValidatedPublisher (low-level)

Direct access to the socket wrapper without StrictDataSchema serialization helpers:

```
import zmq.asyncio
from snapper.messaging.infrastructure.validated_socket import
ValidatedPublisher, apply_hwm, HWM_MARKET_DATA

ctx = zmq.asyncio.Context()
raw_socket = ctx.socket(zmq.PUB)
apply_hwm(raw_socket, sndhwm=HWM_MARKET_DATA)
raw_socket.connect("tcp://127.0.0.1:7500")
publisher = ValidatedPublisher(raw_socket)

topic = "market.kraken.BTC-USD.candles.1h"
payload = candle_data.to_json().encode()
await publisher.send_multipart(topic, payload)
publisher.close()
```

Subscriber

Subscribing to messages via validated socket wrapper around a raw ZMQ SUB socket:

```
import zmq.asyncio
from snapper.messaging.infrastructure.validated_socket import
ValidatedSubscriber, apply_hwm, HWM_MARKET_DATA
```

```

from snapper.messaging.schemas.messages import parse_message

ctx = zmq.asyncio.Context()
raw_socket = ctx.socket(zmq.SUB)
apply_hwm(raw_socket, rcv_hwm=HWM_MARKET_DATA)
raw_socket.connect("tcp://127.0.0.1:7501") # broker X PUB
subscriber = ValidatedSubscriber(raw_socket)
subscriber.subscribe("market.kraken.BTC-USD.")

while True:
    topic, payload = await subscriber.recv_multipart()
    envelope = parse_message(payload.decode())
    print(f"Received {envelope.type} on {topic}")

```

Market Data Publisher

Built-in publisher for exchange data:

```
snapper feed --symbols "BTC-USD,ETH-USD"
```

Programmatically:

```

from snapper.messaging.publishers.kraken import KrakenMarketDataPublisher

async def run_feed():
    publisher = KrakenMarketDataPublisher(symbols=["BTC-USD", "ETH-USD"])
    await publisher.start()

```

Candle Identity Cache

On startup the publisher loads the latest candle public_id per (instrument, timeframe) pair from the database into an in-memory cache. When a new tick arrives for an existing candle interval (open_at matches), the cached public_id is reused so the same logical candle keeps a stable identity across ZMQ, WebSocket, and the database. When open_at advances to a new interval, a fresh UUID7 is minted and the cache entry is replaced. This avoids any DB reads on the hot path.

Micro-Batch DB Persistence

DB writes are decoupled from the ZMQ publish path via per-loop micro-batching. Each producer loop (candles, ticks, trades) accumulates rows in a local batch and flushes to the database on a size or age threshold:

- **Size trigger:** candles flush at 100 rows, ticks/trades at 500 rows
- **Age trigger:** all streams flush after 50 ms since the first batch item
- **Publish-first:** ZMQ delivery happens before DB persistence; subscribers receive data without waiting for the database

Thresholds are configurable via `write_buffer_flush_ms`, `write_buffer_candle_max_rows`, `write_buffer_tick_max_rows`, and `write_buffer_trade_max_rows` settings (cached at publisher start, require process restart to change).

On shutdown (launcher cancellation or direct stop), each loop flushes its remaining batch in a finally block. The `asyncio.wait` age trigger uses a persistent future that is never cancelled on timeout, preserving the underlying exchange subscription iterator.

Candle flushes that hit an `IntegrityError` (e.g. out-of-order timestamps) fall back to row-by-row retry, isolating the bad row without losing the rest of the batch. The repository `upsert_candles()` contract is unchanged.

Per-stream flush error counters (`_flush_errors`) drive the heartbeat status field: "warning" if any stream has errors, "healthy" otherwise. Counters reset to zero on the next successful flush of that same stream.

Order Executor

Service executing orders:

```
snapper executor -e kraken
```

Executor:

1. Subscribes to `orders.commands.{exchange}.*` topics
2. Receives `OrderRequestData` from the trade runtime via the durable outbox dispatcher
3. Executes order via exchange API
4. Persists `VenueEvent` rows for accepted, fill, and terminal observations
5. Publishes `ExecutionData` or `OrderData`

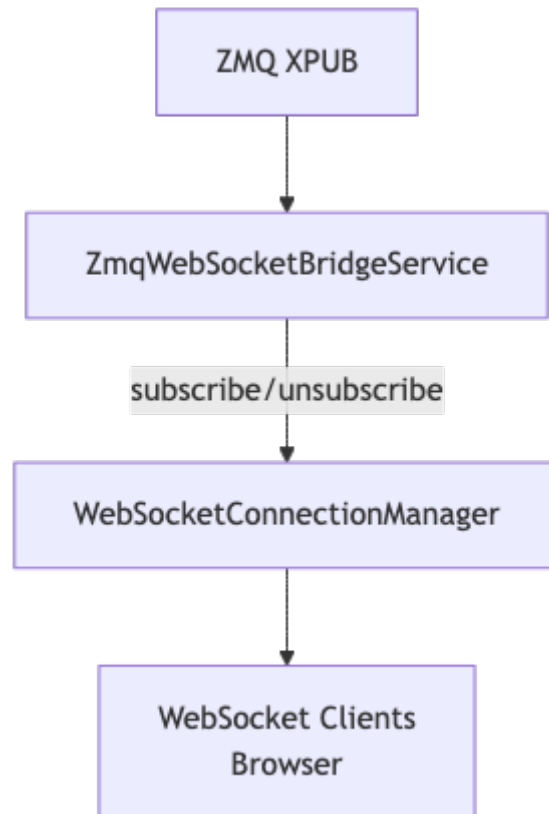
Failed `VenueEvent` persistence is treated as fail-closed on the accepted/fill paths and aborts downstream publish from the executor.

The trade runtime subscribes to `orders.events.*` to keep `TradeService` and `BalanceService` in sync during normal operation. `VenueEvent` rows are used as the durable recovery and reconciliation backbone.

`PlanExecutorService` (see [docs/architecture.md](#) → Execution Plans) also subscribes to `orders.events.` and `market.` on the broker `XPUB`, routes `ExecutionData` → fill propagation, `OrderData` → plan status transitions, and `TickData` → evaluator dispatch. Plan cancels flow through the same `TradeCommand` outbox as submits, with `command_type="cancel"` branching `_outbox_publish` to emit `OrderCancelData` on `orders.commands.{exchange}.{instrument}.cancel`.

ZMQ-WebSocket Bridge

Bridge between ZMQ and WebSocket for frontend:



Bridge automatically:

- Subscribes to ZMQ topics when WebSocket client subscribes
- Validates each inbound ZMQ message as JSON and runs `GapDetector.check()` before forwarding
- Drops messages with malformed JSON (logged as WARNING; counted in `invalid_messages` per topic)
- Forwards messages via `_forward_to_clients` with per-subscription backpressure
- Throttles market data per subscriber (configurable `throttle_ms`)
- Drops market data when a client exceeds `MAX_PENDING_MESSAGES_MARKET` (100)
- Disconnects slow clients on trade topics when exceeding `MAX_PENDING_MESSAGES_TRADE` (1000)
- Unsubscribes when last client disconnects
- Records control events (subscribe errors, client disconnects) to the control table

Gap Detection

`GapDetector` tracks `session_id` and `sequence_id` per received ZMQ topic and emits log messages when it finds sequence gaps or producer session resets.

```
from snapper.messaging.infrastructure.gap_detector import GapDetector

detector = GapDetector()
detector.check(received_topic, session_id, sequence_id)
```

Rules:

- Messages without provenance (`session_id == ""` or `sequence_id == 0`) are **rejected** with a WARNING log and a `rejected_unstamped` counter increment. This makes unstamped traffic observable rather than silently passing through.
- First message on a topic: sets baseline. If `sequence_id > 1` the subscriber joined mid-stream (logged INFO).
- Same topic, new `session_id`: producer restarted (logged INFO, counter resets).
- In-order message (`sequence_id == expected`): accepted silently.
- Gap (`sequence_id > expected`): logs WARNING with the missing range, then advances.
- Duplicate / reorder (`sequence_id < expected`): logs DEBUG, state unchanged.

`GapDetector.check()` returns `True` when the message carries valid provenance and was processed, `False` when rejected as unstamped.

`GapDetector` is wired into the ZMQ-WebSocket bridge and into subscriber loops inside the executor, trade runtime coordinator, and `strategy _listen_loop()`.

Message Logger

Debug tool for monitoring ZMQ traffic:

```
snapper zmq-logger --payload --audit-file logs/zmq.jsonl
```

Logs all messages passing through the broker.

Configuration

Endpoints in `.env`:

```
ZMQ_BROKER_XSUB=tcp://127.0.0.1:7500
ZMQ_BROKER_XPUB=tcp://127.0.0.1:7501
```

Throttling

`TopicSchema` defines throttling per topic:

```
TopicSchema(  
    name="market",  
    pattern="market.",  
    throttle_ms=100, # Max 10 msg/s  
    ...  
)
```

Message Validation

ValidatedPublisher and ValidatedSubscriber validate **topic strings**, not message payloads. ValidatedPublisher.send_multipart() calls validate_topic() before sending; ValidatedSubscriber.subscribe() calls validate_subscription_pattern() before subscribing. Both raise TopicValidationError on invalid topics.

High Water Mark (HWM) Policy

ZMQ sockets use explicit high water marks via apply_hwm() from validated_socket.py to bound queue depth and prevent silent message loss:

Tier	Constant	Value	Used by
Order flow	HWM_ORDER_FLOW	0 (unlimited)	Executors, trade runtime coordinator
Broker	HWM_BROKER	10 000	XSUB (rcvhwm), XPUB (sndhwm)
Market data	HWM_MARKET_DATA	5 000	Publishers, strategies, bridge, settings, symbol updaters
Audit	HWM_AUDIT	10 000	Message logger

apply_hwm() must be called **before** connect() or bind().

Retry and Resilience

Sockets automatically:

- Reconnect on connection loss
- Buffer messages when broker unavailable
- LINGER=0 on close (discard pending)
- HWM applied before connect/bind (see above)

Topic Derivation

`topic_for_message()` in `snapper.messaging.topics.builders` derives the canonical ZMQ topic from a typed Data class. Producers typically call it before `MessagePublisher.send()`.

```
from snapper.messaging.topics.builders import topic_for_message

topic = topic_for_message(candle_data)
await publisher.send(topic, candle_data)
```

Paper market data topics include a `source_exchange` segment that is not carried in the payload, so they must always be published with an explicit topic override.

WebSocket Message Provenance

All WebSocket protocol messages (authentication, subscription management, ping/pong, errors) inherit directly from `StrictDataSchema`, so they carry the same provenance fields as ZMQ data payloads: `public_id`, `session_id`, `sequence_id`, and `timestamp`. Every payload item in the system — whether it flows over ZMQ, REST, or WebSocket — has a uniform provenance envelope.

Audit Tables: Control and Telemetry

Two destination tables provide always-available observability for non-domain traffic:

- **control** — Always-on audit for commands, authentication events, subscribe/unsubscribe messages, and REST mutation requests. Recording is wired in three places:
- **WS handlers** — `_record_ws_control()` records auth, subscribe, unsubscribe, and error events with `transport="ws"`. Client causation linkage is extracted from inbound messages via `_extract_client_provenance()` and stored as `client_session_id` and `client_public_id` on the control row.
- **REST middleware** — `ClientProvenanceMiddleware._record_control()` records every mutation (POST) with `transport="rest"`, redacted payload, outcome (ok/error/exception), and server-side provenance.
- **ZMQ bridge** — `_record_bridge_control()` records subscribe errors and client disconnects with `transport="zmq"`.

All control writes use a finally block so the record is persisted regardless of whether the request succeeded or failed. The write is non-blocking: any DB failure is logged and swallowed so the response already sent to the client is never invalidated.

- **telemetry** — Toggleable high-volume table for pings, heartbeats, pongs, and GET read requests. Recording is gated by the `TELEMETRY_RECORDING_ENABLED` environment variable (default false). When disabled, `SequenceTracker` counters still increment — only the DB write is skipped. Recording is wired in:
- **WS handlers** — `_record_ws_telemetry()` records ping/pong/heartbeat events
- **REST middleware** — `_record_telemetry()` records GET reads (health, status, entity endpoints)

The non-blocking audit invariant applies to both tables: audit writes never reject, delay, or invalidate the primary request/message flow.

Best Practices

1. **One broker per system** — All components connect to the same broker
2. **Topic hierarchy** — Use hierarchy for filtering (`market.kraken.*`)
3. **Data types** — Always use typed Data classes from `messaging.schemas.data`
4. **Provenance** — Use `MessagePublisher` (not `ValidatedPublisher` directly) so every message carries `session_id` and `sequence_id` for gap detection
5. **One SequenceTracker per component** — Create it once at `start()` and share across all publisher instances; restart creates a new session
6. **Heartbeats** — Keep component-specific heartbeat cadences small and regular
7. **Graceful shutdown** — Close sockets with `LINGER=0`

Observability

Snapper exposes operator-facing health metrics so a dashboard can see the gradient toward exhaustion before a crash. The current observability surface is **Cluster A** of the three-cluster monitoring initiative: process-level counters sampled into an in-memory ring buffer with a REST query API.

Cluster B (per-table SCD2 DB stats) and Cluster C (retention policy framework) are sequenced after this. C lands before B because C defines what "archivable" means.

Endpoints

All four routes are gated by `Permission.READ_SYSTEM_STATUS` (held by `AI_DELEGATE`, `VIEWER`, `OPERATOR`, `ADMIN`). The two POST routes also require CSRF (cookie-auth path); Authorization: Bearer requests bypass CSRF per the project-wide auth contract.

Method	Path	Purpose
GET	/api/metrics/system	Latest snapshot
GET	/api/metrics/system/history?since&until&limit	Windowed slice of the ring buffer
POST	/api/metrics/system/tracemalloc/start?duration_s	Arm Python tracemalloc with auto-stop deadline
POST	/api/metrics/system/tracemalloc/stop	Disarm tracemalloc + cancel pending deadline

Failure contract

If the snapshotter singleton failed to start at lifespan time, the `app.state.system_metrics_snapshotter` attribute stays `None` and every metrics route returns HTTP 503 with body `{"detail": "system metrics snapshotter not available"}`. The rest of the application continues to serve other endpoints.

Cold-start contract

`SystemMetricsSnapshotter.start()` takes one eager synchronous sample BEFORE returning, so the first request after lifespan startup completes hits a populated buffer.

Snapshot fields

Each sample captures eight nested groups plus two top-level flags. All field names are stable; downstream consumers (frontend, iOS) regenerate from the backend OpenAPI / JSON-Schema export and pin the wire shape.

process

Field	Type	Description
pid	int	Process ID.
uptime_seconds	float	Monotonic-clock seconds since the snapshotter was constructed.
status	str	psutil process status (running, sleeping, ...).
num_threads	int	Live thread count.
num_fds	int	Open file descriptor count.
num_connections	int	Open TCP/UDP socket count.

cpu

Field	Type	Description
process_percent	float	psutil.Process.cpu_percent reading at sample time.
user_time_seconds	float	Cumulative user-mode CPU.
system_time_seconds	float	Cumulative system-mode CPU.
cgroup_quota_microseconds	int null	cgroup CPU quota microseconds per period; null when absent or unlimited.
cgroup_throttled_count	int null	Cumulative throttle event count from cpu.stat nr_throttled; null when cgroup absent.

memory

Field	Type	Description
rss_bytes	int	Resident set size (RSS).
rss_peak_bytes	int	Peak RSS from /proc/self/status VmHWM; falls back to rss_bytes on non-Linux.

Field	Type	Description
vms_bytes	int	Virtual memory size.
python_traced_bytes	int null	Bytes tracked by tracemalloc; null when tracemalloc is OFF.
native_bytes	int null	max(0, rss - python_traced) when tracemalloc is active; null otherwise — exposes "native dark matter" (numpy / pandas / zmq / aiosqlite native cache / pydantic-core Rust).
cgroup_limit_bytes	int null	cgroup memory cap; null when absent or unlimited.
cgroup_current_bytes	int null	Current cgroup memory consumption.
saturation_pct	float null	cgroup_current / cgroup_limit when both available; null otherwise.

asyncio

Field	Type	Description
active_tasks	int	len(asyncio.all_tasks()).
pending_tasks	int	Subset of all_tasks whose done() is False.

gc

Field	Type	Description
collections_gen0	int	Cumulative GC collection count for generation 0.
collections_gen1	int	Generation 1.
collections_gen2	int	Generation 2.
uncollectable	int	Cumulative count of objects GC could not free.
current_objects	int	Sum of gc.get_count() across all generations at sample time.

limits

Field	Type	Description
rlimit_nproc	int	Soft RLIMIT_NPROC.

Field	Type	Description
rlimit_nofile	int	Soft RLIMIT_NOFILE.
rlimit_as_bytes	int	Soft RLIMIT_AS (address space).

saturation

Percentage toward exhaustion (0.0 - 1.0). The gradient operators actually need to see (vs raw counts) — the rate of increase here is the leading indicator of a thread leak or fd leak.

Field	Type	Description
threads_pct	float null	num_threads / rlimit_nproc; null when rlimit_nproc is RLIM_INFINITY or zero.
fds_pct	float null	num_fds / rlimit_nofile; null under the same conditions.

db_internal

Field	Type	Description
aiosqlite_live_connections	int	len(_live_aiosqlite_connections) — atomic read; each entry corresponds to one OS thread under NullPool.
pool_size	int null	Reserved for future queue-pool instrumentation. Currently always null in Cluster A; will populate once the SQLAlchemy pool reflection helper lands alongside the DB_POOL_MODE=queue rollout.
pool_checked_out	int null	Reserved for future queue-pool instrumentation. Currently always null in Cluster A.

Top-level flags

Field	Type	Description
bus_time	datetime (UTC)	Sample timestamp; window queries match against this field.
tracemalloc_active	bool	True iff tracemalloc.is_tracing() was True at sample time.

Field	Type	Description
cgroup_version	"v1" "v2" null	cgroup detection result; null on hosts without cgroup.

Configuration

The snapshotter reads two environment variables directly (no AppSettings extension this iteration). Defaults match the in-code constants in [snapper.application.system_metrics.snapshotter](#).

Variable	Default	Effect
SYSTEM_METRICS_INTERVAL_SECONDS	5	Seconds between sampler ticks. Empty / unparseable / non-positive falls back to the default.
SYSTEM_METRICS_HISTORY_CAP	17280	Ring buffer cap (snapshot count). 17280 $\approx$ 24h at the default 5s interval. Empty / unparseable / non-positive falls back to the default.

Operators can drop the ring buffer cap to reduce resident memory ($720 \approx 1h \approx 360$ KB).

Sampling cadence + ring buffer

- Sample interval: configurable, default **5 seconds**.
- Per-snapshot byte budget: ~ 500 bytes (≈ 30 metrics, ~ 15 bytes each).
- Default cap 17280 snapshots $\approx$ **8.6 MB resident**.
- Eviction: oldest-first via collections.deque(maxlen=N) semantics.
- All buffer access goes through an asyncio.Lock so concurrent route reads see a consistent view (single-writer / multi-reader).

cgroup detection

Runs at every sample and degrades silently when paths are absent:

- v2 default in modern Docker (kernel 5.0+, Docker Engine 20.10+): /sys/fs/cgroup/memory.max, cpu.max, cpu.stat, memory.current.
- v1 fallback: memory/memory.limit_in_bytes, memory/memory.usage_in_bytes, cpu,cpuacct/cpu.cfs_quota_us (or the legacy cpu/cpu.cfs_quota_us mount), and the matching cpu.stat file for nr_throttled.

- `cgroup_version` reports the detected layout; null on dev hosts without cgroup (macOS, non-containerised Linux without unified hierarchy).

Tracemalloc

Off by default — enabling tracemalloc costs **5-10% CPU** while active and holds extra metadata in process memory. Operators arm it briefly to capture the `python_traced_bytes` byte counter for the `native_bytes` diagnostic, then auto-stop fires after a bounded window.

Knob	Default	Cap
<code>duration_s</code> (POST query)	600s	3600s (clamped)

Calling `start` while already armed REPLACES the deadline (cancels the previous timer, starts a fresh one with the new duration). The route response carries the clamped `requested_duration_seconds`, so the operator sees what was actually applied.

Example invocations

Latest snapshot:

```
curl -fsS \
  -H "Authorization: Bearer ${SNAPPER_ACCESS_TOKEN}" \
  https://snapper.example.com/api/metrics/system \
  | jq '.payload | {threads_pct: .saturation.threads_pct,
  rss: .memory.rss_bytes, aiosqlite: .db_internal.aiosqlite_live_connections}'
```

Last hour, capped at 60 samples:

```
SINCE=$(date -u -d '-1 hour' +%FT%TZ)
curl -fsS \
  -H "Authorization: Bearer ${SNAPPER_ACCESS_TOKEN}" \
  "https://snapper.example.com/api/metrics/system/history?since=${SINCE}
  &limit=60" \
  | jq '.count, [.payload[].memory.rss_bytes]'
```

Arm tracemalloc for 5 minutes to inspect the native gap:

```
curl -fsS -X POST \
  -H "Authorization: Bearer ${SNAPPER_ACCESS_TOKEN}" \
  "https://snapper.example.com/api/metrics/system/tracemalloc/start?
  duration_s=300" \
  | jq
sleep 30
curl -fsS \
  -H "Authorization: Bearer ${SNAPPER_ACCESS_TOKEN}" \
```

```
https://snapper.example.com/api/metrics/system \
| jq '.payload.memory | {rss: .rss_bytes, traced: .python_traced_bytes,
native: .native_bytes}'
```

Multi-instance note

The ring buffer is per-process. With $N > 1$ instances each hosts its own buffer; aggregation across instances is out of scope for Cluster A. Operators who need cross-instance views should poll each instance directly until a downstream collector lands (Cluster B/C territory).

Retention

Cluster C ships a **policy-driven retention loop** that periodically archives + purges old rows from event tables, complementing the existing manual snapper archive CLI. Policies are declarative (`RetentionPolicy(table, retain_days, backlog_lookback_days)`) and evaluated by `RetentionService` on every scheduler tick.

Endpoint

Method	Path	Purpose
GET	/api/metrics/retention	Most recent scheduler tick's per-policy summary

The route is gated by `Permission.READ_SYSTEM_STATUS` (same gate as the system metrics surface).

Failure / cold-start contract

The route returns HTTP 503 with one of three details:

- "retention scheduler not available" — singleton failed to start at lifespan time (B22 attribute-absent contract).
- "retention scheduler disabled" — the operator set `RETENTION_DISABLED=true`; the scheduler is parked.
- "retention scheduler not yet run" — eager run did not populate the summary before the first request (defensive; should not happen in production because `start()` awaits the eager `run_once`).

Policy semantics

For each scheduler tick, with `today_utc = datetime.now(UTC).date()`:

```
oldest_kept_day      = today_utc - retain_days      # rows on this day stay in
DB
last_eligible_day    = oldest_kept_day - 1d         # day_end (inclusive)
earliest_scanned_day = last_eligible_day - backlog_lookback_days # day_start
(inclusive)
```

The archiver call covers the inclusive day-range `[day_start, day_end]`, matching `EventArchiver.export(...)` semantics. Steady state after the backlog drains processes one new day per tick.

Concrete example with `today_utc = 2026-05-01`, `retain_days = 1`,
`backlog_lookback_days = 30`:

- `oldest_kept_day = 2026-04-30` (rows with timestamp on 2026-04-30 stay in DB).
- `last_eligible_day = 2026-04-29` (last day archived + purged this tick).
- `earliest_scanned_day = 2026-03-30` (oldest day touched this tick).
- The archiver query covers `[2026-03-30 00:00 UTC, 2026-04-30 00:00 UTC)` per the repository semantics.

The shipped default policy:

Table	retain_days	backlog_lookback_days
telemetry	1	30

Configuration

Variable	Default	Effect
RETENTION_INTERVAL_SECONDS	3600	Scheduler loop period. Empty / unparseable / $\leq 0 \rightarrow$ default.
RETENTION_DISABLED	false	Truthy ("1"/"true"/"yes", case-insensitive) parks the scheduler entirely.
RETENTION_DRY_RUN	false	Truthy forces <code>purge=False</code> on every archiver call regardless of policy. Operators flip to true for one cycle when adding a NEW high-volume policy to verify the window before any DB row is deleted.
RETENTION_OUTPUT_DIR	data	Filesystem root passed to EventArchiver; matches the snapper archive CLI default.

Roll-out checklist for a new high-volume policy

1. Add the new RetentionPolicy entry to RETENTION_POLICIES in src/snapper/application/retention/policies.py.
2. Set RETENTION_DRY_RUN=true for one full interval and inspect GET /api/metrics/retention — check that the day_start / day_end window matches expectations and archived_rows is non-zero.
3. If the new policy targets a deeply backlogged table, run a one-time snapper archive --table=<...> --from=<old> --to=<recent> --purge to drain backlog before flipping RETENTION_DRY_RUN=false.
4. Flip RETENTION_DRY_RUN=false. Subsequent ticks will both archive AND purge.

Multi-instance note

The retention scheduler is single-instance only for v1. With $N > 1$ instances, RETENTION_DISABLED=true on $N-1$ instances avoids double-archive / double-purge races. A coordinator-based extension is a future iteration.

Cluster B — per-table SCD2 stats (DB metrics)

GET /api/metrics/db/tables returns the latest sampled per-table row counters for every registered event + state SCD2 table. The endpoint is operator telemetry: at a glance it answers "is telemetry growing as expected", "are there orders accumulating closed versions", "how many rows are eligible for the next retention cycle".

Cluster B endpoint

GET /api/metrics/db/tables — READ_SYSTEM_STATUS permission gate. Returns a DbStatsResponse envelope wrapping a DbStatsData payload with tables: list[TableStatsItem] in the sampler's deterministic order (STATE-table block first alphabetical, EVENT-table block second alphabetical).

Cluster B failure / cold-start contract

State	Status	Detail	Retry-After
Snapshotter helper failed before assigning	503	DB metrics snapshotter not initialized	(none)
DB_METRICS_DISABLED=true (operator opt-out)	503	DB metrics snapshotter disabled via DB_METRICS_DISABLED	(none)

State	Status	Detail	Retry-After
Started but no sample yet (cold-start)	503	DB metrics snapshotter has not completed a sample yet	<interval_seconds>
Healthy	200	—	—

The cold-start window lasts up to `DB_METRICS_INTERVAL_SECONDS` (default 60s) — by design, the sampler does NOT block lifespan startup on the first sample. Frontend dashboards must poll-with-backoff using the Retry-After header.

Snapshot fields

TableStatsItem

Field	Type	Notes
table	str	Table name (key in <code>EVENT_TABLES</code> or <code>STATE_TABLES</code>).
table_kind	"event" "state"	Discriminates wire semantics.
total	int \ null	Total row count. null only on per-table query failure with no prior sample to clone.
current	int \ null	Active SCD2 versions (rows whose <code>known_to</code> equals the SCD2 sentinel) for state tables; null for event tables (no SCD2 lifecycle — reporting 0 would imply the dimension exists).
closed	int \ null	Superseded SCD2 versions for state tables; null for event tables.
archivable	int \ null	Row count in the policy retention window when a <code>RETENTION_POLICIES</code> entry applies; null when no policy applies (semantically distinct from 0).
is_stale	bool	True when the row was reused from a prior sample after a per-table query timeout or exception.
last_sampled_at	datetime	UTC timestamp of the row's source sample. On a stale clone this is the original timestamp, NOT the current cycle's clock.

DbStatsData

Field	Type	Notes
snapshot_started_at	datetime	When the sampler began the cycle.
snapshot_completed_at	datetime	When the sampler finished the cycle.
interval_seconds	float	Echo of the configured cadence.
tables	list[TableStatsItem]	One entry per registered table.

Per-kind semantics

- **EVENT tables** (append-only — ticks, trades, signals, executions, telemetry, control):
total = COUNT(*). The current / closed axes are null because event tables have no SCD2 lifecycle. archivable is non-null only for tables with a registered policy (currently only telemetry).
- **STATE tables** (SCD2-versioned — orders, positions, instruments, etc.): current counts rows whose known_to equals the SCD2 sentinel; closed counts superseded versions; total = current + closed (Python addition trusted on the SCD2 invariant). archivable is null until a policy is registered for the table.

Cluster B/C alignment

The archivable counter computes the same window as Cluster C's `compute_retention_window(today_utc, policy)`: the half-open timestamp $\geq$ day_start midnight UTC AND timestamp $<$ day_end + 1d midnight UTC predicate, with the policy's (retain_days, backlog_lookback_days) knobs. This means dashboard archivable and the next retention cycle's archived_rows MUST agree exactly — if they drift, one of the two has a boundary bug. The tests/application/db_stats/test_cluster_alignment.py integration test pins the equality contract.

Cluster B configuration

Variable	Default	Effect
DB_METRICS_INTERVAL_SECONDS	60	Sampler loop period. Empty / unparseable / $\leq 0 \rightarrow$ default.
DB_METRICS_DISABLED	false	

Variable	Default	Effect
		Truthy ("1"/"true"/"yes", case-insensitive) parks the snapshotter entirely. PG operators with deployments lacking psycpg2 in deps boot cleanly with this flag (the underlying repo factory is never called).

PER_TABLE_TIMEOUT_SECONDS is a module constant (30s, not env- configurable for v1). On per-table timeout, the snapshotter clones the prior TableStats with `is_stale=True`; if no prior sample exists the row carries all-null counters. Per-table failures NEVER abort the sampler tick.

Sampling order + cold-start

STATE tables sample first (alphabetical by name), EVENT tables second. Atomic swap of `_latest_snapshot` happens at the END of the cycle — readers wait one full `interval_seconds` before the first 200, regardless of order. STATE-first ordering is for resilience to EVENT timeouts: if a heavy EVENT table (e.g. ticks at 50M rows) times out, STATE counters were already computed and the snapshot still publishes with EVENT entries marked `is_stale=True` from a prior run.

Migration 0002_telemetry_timestamp_index

Cluster B's archivable query on telemetry filters on `Telemetry.timestamp` (the bus-time column inherited from `TemporalMixin`). Without an index on that column, the query is a full table scan over a high-volume audit table. The `0002_telemetry_timestamp_index` migration adds `Index("ix_telemetry_timestamp", "timestamp")` so both Cluster B's counter and Cluster C's existing retention scan run in $O(\log n)$.

The PG path uses `op.get_context().autocommit_block() + postgresql_concurrently=True` so a live deploy can apply the migration without blocking writes. SQLite path runs the standard create/drop without the block.

Operations runbook — multi-instance trade coordinator

The trade runtime supports static-hash shard partitioning. At `SNAPPER_COORDINATOR_INSTANCE_COUNT=1` (the default) a single `TraderCoordinator` owns every shard. Scaling to $N \geq 2$ deploys multiple coordinators against the same DB and ZMQ broker, each owning $\sim 1/N$ of the shards deterministically via SHA-256 of the shard key.

This document covers operating the N-instance trade runtime: scale-up, scale-down, and crash recovery. It is intentionally opinionated about `systemd` because `systemd` template units are the stable recipe. A contract for alternative orchestrators (Docker Compose, Kubernetes, Nomad) is also specified so operators can adopt them with empirical verification.

Prerequisites: systemd template unit

All subsequent recipes assume a template unit `snapper-trade-zmq@.service` that consumes `%i` as the instance id:

```
# /etc/systemd/system/snapper-trade-zmq@.service
[Unit]
Description=Snapper Trade Coordinator (instance %i)
After=network.target snapper-broker.service

[Service]
Type=simple
User=snapper
EnvironmentFile=/etc/snapper/coordinator.env
Environment=SNAPPER_COORDINATOR_INSTANCE_ID=%i
ExecStart=/opt/snapper/bin/snapper trade-zmq
Restart=always
RestartSec=2

[Install]
WantedBy=multi-user.target
```

With this template, `snapper-trade-zmq@0.service` and `snapper-trade-zmq@1.service` are distinct `systemd` units with separate cgroups and separate journal streams on the same host. Per-instance PID is queried via `systemctl show -p MainPID snapper-trade-zmq@0.service`.

`/etc/snapper/coordinator.env` holds the shared bootstrap env, including `SNAPPER_COORDINATOR_INSTANCE_COUNT=2` during $N=2$ operation:

```
DB_URL=postgresql+asyncpg://snapper:...@db.internal/snapper
ZMQ_BROKER_XSUB=tcp://broker.internal:7500
ZMQ_BROKER_XPUB=tcp://broker.internal:7501
MASTER_PASSWORD=...
SNAPPER_COORDINATOR_INSTANCE_COUNT=2
```

Scale up N=1 → N=2 (systemd)

```
# 1. Edit /etc/snapper/coordinator.env – set
SNAPPER_COORDINATOR_INSTANCE_COUNT=2.

# 2. Stop the currently-running single-instance coordinator.
sudo systemctl stop snapper-trade-zmq@0.service

# 3. Verify graceful shutdown.
sudo systemctl status snapper-trade-zmq@0.service | grep -E "Active:|Main PID:"

# 4. Start both instances.
sudo systemctl start snapper-trade-zmq@0.service
sudo systemctl start snapper-trade-zmq@1.service

# 5. Verify both are live.
sudo systemctl is-active snapper-trade-zmq@0.service # expect: active
sudo systemctl is-active snapper-trade-zmq@1.service # expect: active
sudo journalctl -u snapper-trade-zmq@0.service -n 20 | grep "instance 0/2"
sudo journalctl -u snapper-trade-zmq@1.service -n 20 | grep "instance 1/2"
```

Scale down N=2 → N=1 (systemd)

```
# 1. Stop BOTH coordinator instances first – full cutover is required.
sudo systemctl stop snapper-trade-zmq@0.service
sudo systemctl stop snapper-trade-zmq@1.service

# 2. Verify nothing remains.
sudo systemctl list-units 'snapper-trade-zmq@*' --state=active --no-legend
# (expect empty output)

# 3. Edit /etc/snapper/coordinator.env – set
SNAPPER_COORDINATOR_INSTANCE_COUNT=1.

# 4. Start instance 0 only.
sudo systemctl start snapper-trade-zmq@0.service
```

Why full cutover is required

Scale up and scale down both require stopping ALL old-N coordinators BEFORE starting any new-N coordinator. Overlapping old-N and new-N instances is the failure mode to prevent.

Scenario: improper restart with overlap. If an operator skipped the cutover step and left instance 0 running as count=1 while starting instance 1 as count=2:

- Instance 0 believes it owns ALL shards (count=1 → every owns() returns True).
- Instance 1 believes it owns shards where hash(shard_key) % 2 == 1.
- Both coordinators dispatch the same TradeCommand for shards where hash() % 2 == 1 → duplicate venue orders OR a race on status='created' → 'dispatched'.

The ~10s dispatch pause for owned shards during the cutover window is the explicit no-HA trade-off. HA was excluded from scope by design.

Crash recovery (systemd)

If instance K crashes:

```
# 1. Diagnose.
sudo journalctl -u snapper-trade-zmq@1.service -n 100 --no-pager

# 2. Check whether Restart=always brought it back.
sudo systemctl is-active snapper-trade-zmq@1.service

# 3a. If active: confirm the coordinator resumed with its original ownership.
sudo journalctl -u snapper-trade-zmq@1.service -n 200 | grep "instance 1/2"
sudo journalctl -u snapper-trade-zmq@1.service -n 200 | grep "recovery complete"

# 3b. If inactive (Restart=always gave up): manually restart.
sudo systemctl restart snapper-trade-zmq@1.service
```

During the outage window, shards owned by the crashed instance pause until the coordinator comes back. Typical outage is under 10s if Restart=always is configured.

Recovery rebuilds engines from:

1. Checkpoints (carry shard_key directly, filtered by ownership).
2. Live executions (filtered by recovered shard_key; paper rows are EXCLUDED under N>1 because ExecutionRow has no strategy_tag).
3. Active orders (same treatment as executions).

Under N>1 paper mode, state that never produced a checkpoint is not recovered by the restarted coordinator. Paper strategies that need to survive restart MUST persist checkpoints.

Known limitation — REST orders with wallet_public_id under N>1

The REST order endpoints at `src/snapper/server/order_routes.py` build the `TradeCommand.shard_key` without the wallet segment, while the signal-driven engine's `shard_key` appends `.w{wallet_short}` via `_compute_shard_key` when `wallet_public_id` is non-empty.

At $N=1$ this is dormant because every shard is owned by the single coordinator, the outbox has no ownership filter, and the CID guard is gated on `instance_count > 1`.

Under $N>1$, a REST order with a non-empty `wallet_public_id` writes a `TradeCommand` whose `shard_key` hashes to a different instance than the engine for the same (exchange, instrument, wallet). The owning coordinator dispatches the command to the venue correctly, but the venue ACK is dropped by the CID guard on every coordinator because the CID was never registered in `_order_shard_keys` (the REST path does not populate it).

Consequence at $N>1$: REST orders with `wallet_public_id` reach the venue, but the coordinator's `TradeService` projection does not reflect fills/cancellations. Monitoring via the orders and executions tables still works — only the in-memory coordinator state drifts.

Mitigation until a forward fix lands: for $N>1$ deployments that need REST orders, either (a) set `wallet_public_id=""` on the REST request, or (b) stay on $N=1$ for workflows that mix signal-driven + REST-driven orders. Signal-only workflows are unaffected.

Non-systemd deployments (contract)

The operational recipes are anchored on the systemd template above. For non-systemd platforms, the deployment author writes a recipe that satisfies the same contract:

1. Two distinct supervised processes on the same host (or across hosts) with `SNAPPER_COORDINATOR_INSTANCE_ID=0` and `=1` respectively, and `SNAPPER_COORDINATOR_INSTANCE_COUNT=2` in both.
2. Per-instance lifecycle (stop / start / restart / logs) via the orchestrator's native mechanism — never by process pattern match (`pkill -f` is too broad for multi-instance).
3. Scale-up and scale-down follow the same stop-all → verify-empty → start-new cutover flow as the systemd recipes above.
4. At least one concrete recipe per chosen orchestrator, verified empirically, with a provenance line:

Verified on YYYY-MM-DD against <platform-version>

The regex enforced by tests/meta/test_operations_runbook.py is `^Verified on \d{4}-\d{2}-\d{2} against \S+$` — exact casing, anchored to the start and end of the line, placed inside the non-systemd recipe section (not anywhere in the file).

Acceptable orchestrators for the non-systemd recipe are docker compose, kubectl (Kubernetes), or nomad — the meta-test rejects bare docker because docker run alone does not provide the per-instance lifecycle the contract requires.

Verified environments

Verified on 2026-04-18 against docker-compose-v2.29.7

Kraken Equities (TradFi) market data

TradFi index futures (Kraken FCM: MNQ/ES/YM/RTY/NKD/M2K) are shipped in market-data-only mode. The symbol updater + publisher processes are disabled at code level; runtime enable lives in the DB Setting rows for the relevant processes so rollout + rollback happen without code deploys.

Contracts used by this flow:

- REST: `iapi.kraken.com/api/internal/markets/all/futures-contracts` for instrument metadata; `.../{ws_symbol}/ticker/history` for historical candles. Requires Origin: `https://pro.kraken.com` + Referer: `https://pro.kraken.com/` headers.
- WS: `wss://ws-equities.kraken.com` for live delayed (~10 min) ticks + trades. The outer envelope's delayed flag propagates into every TickData message as `is_delayed`.
- No order API — every submit route (POST `/api/orders`, POST `/api/executions-plans`, POST `/api/trailing-stops`) calls `snapper.server._capability_guard.require_tradable` and rejects TradFi instruments with HTTP 422 error_code=`instrument_market_data_only`.

Enable the feed

```
# 1. Populate Symbol + SymbolExchangeCapability rows from iapi.
#   (kraken_equities entry must be present for is_tradeable
#   cache hits; default deny on missing row.)
snapper update-kraken-equities-symbols --force

# 2. Verify the rows landed.
sqlite3 snapper.db \
  "SELECT COUNT(*) FROM symbol_exchange_capabilities
  WHERE exchange='kraken_equities' AND valid_to='9999-12-31T23:59:59';"
# expect >= 38 (indices only; full catalog up to ~180 contracts)

# 3. Enable the symbol-updater + feed-publisher processes via DB
#   Setting rows (NOT env vars — see feedback_no_asking_continue.md
#   for the rollout preference). ``category='process'`` matches the
```

```
# existing ``registry_syncer`` writes; the column is NOT NULL on
# the settings table (see ``src/snapper/data/models.py::Setting``).
sqlite3 snapper.db <<SQL
INSERT OR REPLACE INTO settings
    (key, value, category, timestamp, session_id, sequence_id)
VALUES
    ('process_kraken_equities_symbol_updater',
     '{"enabled":true}', 'process', datetime('now'), 'ops', 0),
    ('process_kraken_equities_feed_publisher',
     '{"enabled":true}', 'process', datetime('now'), 'ops', 0);
SQL

# 4. Restart the runtime OR trigger a live registry sync
# (registry_syncer picks up the Setting rows automatically
# on the next interval).
```

Backfill historical candles

```
# 30-day 1-hour backfill of the four default TradFi symbols
snapper kraken-equities-backfill-candles -t 1h -d 30

# Or the Makefile wrapper (equivalent)
make backfill-kraken-equities-candles

# Single symbol, daily candles, 90 days
snapper kraken-equities-backfill-candles -s MNQM6-CME -t 1d -d 90 --no-resume
```

The endpoint is undocumented/internal — upstream application-layer failures (HTTP 200 with result=null or non-empty errors) raise RuntimeError so they are distinguishable from legitimately-empty windows. See `src/snapper/infrastructure/exchanges/implementations/kraken_equities.py:get_ohlcv` for the error contract.

Quarterly rotation

Default TradFi symbols are quarterly expiry contracts (MNQM6-CME = Jun 26, etc.) and must be rotated before the `SymbolExchangeCapability.maturity` timestamp on any default symbol drops below 14 days. Rotation cadence:

1. Pull the current maturity list: `sqlite3 snapper.db \ "SELECT native_symbol, datetime(maturity,'unixepoch') FROM symbols JOIN symbol_exchange_capabilities ON symbols.public_id = symbol_exchange_capabilities.symbol_public_id WHERE exchange='kraken_equities' AND valid_to='9999-12-31T23:59:59';"`
2. Identify the next quarterly (e.g. MNQU6-CME Sep 26 when MNQM6-CME Jun 26 drops below 14 days).
3. Update `AppSettings.instruments[KRAKEN_EQUITIES]` in code AND/OR the instruments DB Setting row to include the new nearest contract. Deploy + roll back via git revert on the code path; DB Setting path is immediate.

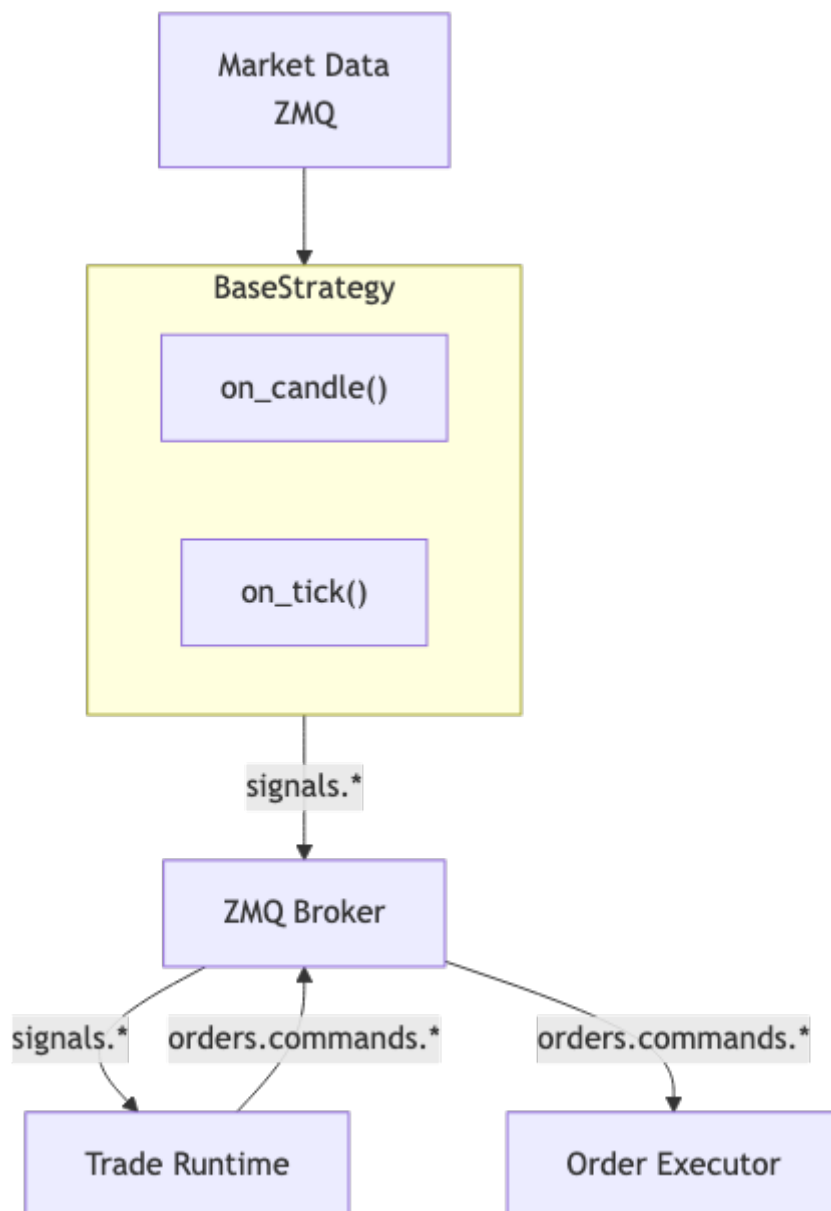
Troubleshooting

- **Zero instruments after update-kraken-equities-symbols --force:** iapi reachability / Origin header mismatch. Verify `curl -H 'Origin: https://pro.kraken.com' -H 'Referer: https://pro.kraken.com/' \ 'https://iapi.kraken.com/api/internal/markets/all/futures-contracts?delayed=true' | head -c 200` returns JSON with `result.data[...]`, not `result:null,errors:[...]`.
- **WS disconnect loops on ws-equities.kraken.com:** inspect Kraken status page; no operator action required — the WS client backs off on reconnect. If the `symbol_updater` keeps returning zero rows across two consecutive runs, fire a monitoring alert.
- **Submit returns 422 instrument_market_data_only:** expected. TradFi is observation-only. Point the strategy at a `can_trade=True` instrument (crypto/xStocks) for execution.

Trading Strategies

Snapper provides a framework for creating trading strategies based on ZeroMQ messaging. Strategies subscribe to market data and publish signals. Strategies produce intent only; the trade runtime owns order lifecycle, positions, and balance projections.

Strategy Architecture



Creating Strategies

Basic Structure

```
from snapper.strategies.base import BaseStrategy, StrategySignal, StrategyConfig
from snapper.strategies.decorators import register_strategy,
create_strategy_process
from snapper.messaging.schemas.data import CandleData

@register_strategy("MyStrategy")
@create_strategy_process(
    process_name="my_strategy_btc",
    default_config={
        "name": "my_strategy_btc",
        "inputs": ["market.kraken.BTC-USD.candles.1h"],
        "outputs": ["BTC-USD"],
        "exchange": "paper",
        "params": {
            "threshold": 0.5,
            "period": 14,
        },
    },
)
class MyStrategy(BaseStrategy):
    """Strategy description in docstring."""

    def __init__(self, config: StrategyConfig) -> None:
        """Initialize strategy."""
        super().__init__(config)
        self.threshold = self.params.get("threshold", 0.5)
        self.period = self.params.get("period", 14)

    async def on_candle(self, instrument: str, candle: CandleData) ->
StrategySignal | None:
        """Process candle and generate signal."""
        candles = self.candle_buffer.get(instrument, [])
        if len(candles) < self.period:
            return None

        closes = [c.close for c in candles]
        current_price = closes[-1]

        if self._should_buy(closes):
            return StrategySignal(
                instrument=instrument,
                side="buy",
                strength=1.0,
                price=current_price,
                reason="Buy condition met",
            )

        if self._should_sell(closes):
            return StrategySignal(
                instrument=instrument,
                side="sell",
                strength=1.0,
                price=current_price,
                reason="Sell condition met",
            )
```

```

        return None

    def _should_buy(self, closes: list[float]) -> bool:
        """Buy condition logic."""
        return False

    def _should_sell(self, closes: list[float]) -> bool:
        """Sell condition logic."""
        return False

    async def reset(self) -> None:
        """Reset strategy state (for replay)."""
        pass

```

Decorators

@register_strategy(name)

Registers strategy class in factory under given name.

```

@register_strategy("RSIReversion")
class RSIReversion(BaseStrategy):
    ...

```

@create_strategy_process(process_name, default_config)

Creates process wrapper for strategy and registers in process manager.

```

@create_strategy_process(
    process_name="strategy_rsi_eth",
    default_config={
        "name": "rsi_eth_1h",
        "inputs": ["market.kraken.ETH-USD.candles.1h"],
        "outputs": ["ETH-USD"],
        "exchange": "paper",
        "params": {"period": 14, "upper": 70, "lower": 30},
    },
)
class RSIReversion(BaseStrategy):
    ...

```

Strategy Configuration

StrategyConfig

Field	Type	Description
name	string	Unique strategy instance name
strategy_class	string	Strategy class name
inputs	list[str]	List of ZMQ topics to subscribe
outputs	list[str]	List of instruments for signals
exchange	string	Target exchange (paper, kraken, walutomat)
params	dict	Strategy-specific parameters
wallet_public_id	string	Wallet that will execute orders for this strategy. Empty default for backwards compatibility; REQUIRED (non-empty) for any strategy that uses create_ai_review_and_await() — see "AI delegate consultation" below.
operator_public_id	string	Trading-identity operator that owns this strategy instance. Empty default; validated against the launching principal's operator_public_ids when populated.

AI delegate consultation

Strategies that consult an AI delegate via `create_ai_review_and_await()` and then forward the approved outcome to `emit_signal(outcome=...)` MUST set `StrategyConfig.wallet_public_id` to a non-empty value. The trader-coordinator's caps gate (`TradingCapsEnforcer.guard_with_ai_review_attribution`) fail-closes early when the engine submission's `wallet_public_id` is empty:

```
CapsViolationError: missing_wallet_for_ai_review_attribution
  submission.wallet_public_id required for the strategy AI gate;
  engine state must populate wallet on the submission before
  invoking this guard
```

The error is loud and operator-actionable: it identifies the exact missing config field. Without `wallet_public_id`, the engine cannot map the AI delegate's approved trade to a wallet for cap evaluation (per-user notional / open-orders / quantity caps key on `wallet+user`). Pure non-AI strategies that never invoke `create_ai_review_and_await()` continue to run without a wallet; they bypass caps via `guard_service_principal()` per the existing audit-bypass contract.

The `ai_review_public_id` carried on the published `SignalData` is transport-only end-to-end through the ZMQ wire. The companion `ai_review_dispatch_version` (dedup key) is also transport-only — the strategy citation validator does not compare it; the bus publisher reads `dispatch_version` from the cited row at publish time so downstream caps-violation fanout events are always tagged with the row-of-record version.

Input Topics

Common formats:

- `market.{exchange}.{instrument}.candles.{timeframe}` — Candle streams
- `market.{exchange}.{instrument}.ticks` — Tick streams
- `market.{exchange}.{instrument}.trades` — Trade streams

Examples:

- `market.kraken.BTC-USD.candles.1h` — Hourly BTC/USD candles from Kraken
- `market.kraken.ETH-USD.ticks` — ETH/USD ticks from Kraken
- `market.polygon.AAPL.trades` — AAPL trade tape from Polygon

Output Topics (Signals)

Generated automatically:

- Paper: `signals.paper.{instrument}.{strategy_name}`
- Live: `signals.{exchange}.{instrument}.live`

StrategySignal Structure

```
@dataclass
class StrategySignal:
    """Trading signal."""

    instrument: str
    side: TradeSide
    strength: float
    reason: str
    price: float
    timestamp: datetime | None
```

Built-in Strategies

RSIReversion

Mean-reversion strategy based on RSI.

Parameters:

Parameter	Default	Description
period	14	RSI period
upper	70.0	Overbought threshold (sell)
lower	30.0	Oversold threshold (buy)
cooldown	2	Candles between signals

Logic:

- Buy when RSI $\leq$ lower
- Sell when RSI $\geq$ upper

```
from snapper.strategies.rsi import RSIReversion
```

MACDCrossover

Trend-following strategy based on MACD.

Parameters:

Parameter	Default	Description
fast	12	Fast EMA
slow	26	Slow EMA
signal_period	9	StrategySignal line period

Logic:

- Buy when MACD histogram crosses from negative to positive
- Sell when histogram crosses from positive to negative

```
from snapper.strategies.macd import MACDCrossover
```

Cointegration

Pairs trading strategy based on cointegration.

```
from snapper.strategies.cointegration import CointegrationPairs
```

Data Access

Candle Buffer

```
async def on_candle(self, instrument: str, candle: CandleData) ->
StrategySignal | None:
    candles = self.candle_buffer.get(instrument, [])

    recent = candles[-20:]

    import pandas as pd
    closes = pd.Series([c.close for c in candles])
```

CandleData

```
from snapper.messaging.schemas.data import CandleData
```

Fields:

Field	Type	Description
type	string	"candle"
public_id	string	UUID7 external identifier
open	float	Open price
high	float	High
low	float	Low
close	float	Close price
volume	float	Volume
timestamp	datetime	Timestamp
timeframe	string	Timeframe
instrument	string	Symbol
exchange	string	Exchange

Technical Indicators

RSI

```
from snapper.indicators.rsi import rsi
import pandas as pd

closes = pd.Series([c.close for c in candles])
rsi_values = rsi(closes, period=14)
current_rsi = rsi_values.iloc[-1]
```

MACD

```
from snapper.indicators.ta_lib_adapter import macd

macd_line, signal_line, histogram = macd(closes, fast=12, slow=26, signal=9)
```

TA-Lib (via adapter)

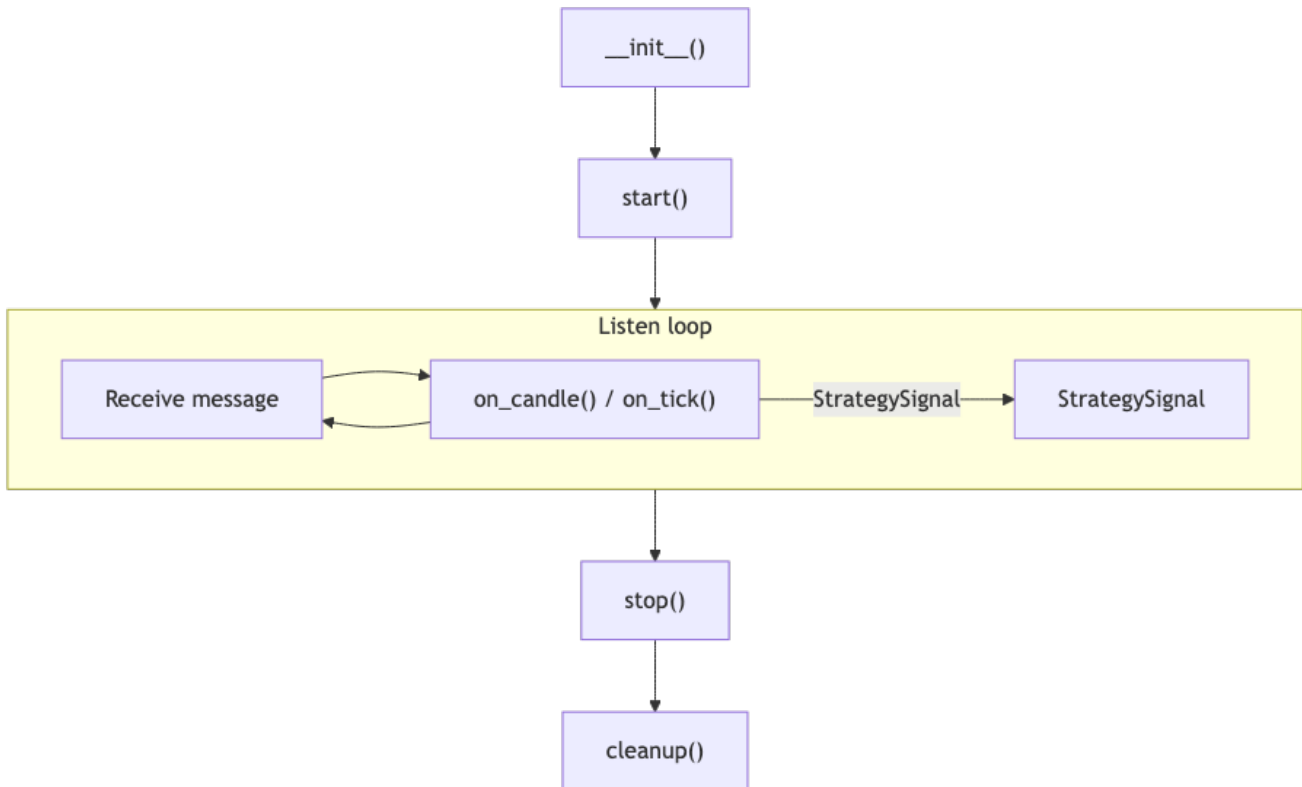
```
from snapper.indicators.ta_lib_adapter import rsi, macd
```

Configuration Validation

Framework automatically validates:

1. **Strategy name** — cannot be empty
2. **Inputs** — at least one required
3. **Outputs** — at least one instrument must be defined
4. **Exchange** — must be one of: paper, kraken, walutomat
5. **Instruments** — must be available on selected exchange
6. **Paper/live mixing** — mixing paper inputs with live exchange not allowed

Strategy Lifecycle



Running Strategies

Via Process Manager (recommended)

Strategies are automatically managed by the server:

```
snapper server
```

Strategies can be enabled/disabled in the dashboard.

Standalone (for testing)

```
import asyncio
from snapper.strategies.factory import StrategyFactory
from snapper.strategies.base import StrategyConfig

config = StrategyConfig(
    name="test_rsi",
    strategy_class="RSIReversion",
    inputs=["market.kraken.BTC-USD.candles.1h"],
    outputs=["BTC-USD"],
    exchange="paper",
    params={"period": 14, "upper": 70, "lower": 30},
)
```

```
factory = StrategyFactory()
strategy = factory.create_strategy(config)

async def main():
    await strategy.start()

asyncio.run(main())
```

Testing Strategies

Unit Tests

```
import pytest
from snapper.strategies.rsi import RSIReverision
from snapper.strategies.base import StrategyConfig
from snapper.messaging.schemas.data import CandleData

@pytest.fixture
def rsi_strategy() -> RSIReverision:
    config = StrategyConfig(
        name="test_rsi",
        strategy_class="RSIReverision",
        inputs=["market.kraken.BTC-USD.candles.1h"],
        outputs=["BTC-USD"],
        exchange="paper",
        params={"period": 14, "upper": 70, "lower": 30},
    )
    return RSIReverision(config)

async def test_buy_signal_on_low_rsi(rsi_strategy: RSIReverision) -> None:
    # Prepare data with low RSI
    ...
    signal = await rsi_strategy.on_candle("BTC-USD", candle)
    assert signal is not None
    assert signal.side == "buy"
```

Backtesting

Use replay data from paper topics:

```
default_config={
    "inputs": ["replay.BTC-USD.candles.1h"],
    "exchange": "paper",
    ...
}
```

Trailing Stop Plans

Trailing stops are server-side evaluated execution plans that ratchet a stop price as the market moves favorably and fire a market close when the stop is breached. Attach a trailing stop to an open position_cycle via POST /api/trailing-stops (see docs/api.md).

Parameters

Field	Type	Bounds	Meaning
trailing_pct	float	$0 < x < 100$	Distance from peak price, as percentage. 5.0 means "trigger 5 % below the highest favorable price seen".
min_lock_pct	float	$0 \leq x < 100$ (default 0)	Profit threshold the market must cross before the trailing stop arms. 0 means "arm immediately on attach"; 5.0 means "wait until price has moved 5 % in your favor before tracking a stop".

Behavior

- **Peak ratchet:** every tick where the last price moves favorably updates the peak; the stop level recomputes as $\text{peak} * (1 - \text{trailing_pct}/100)$ for longs or $\text{peak} * (1 + \text{trailing_pct}/100)$ for shorts.
- **Stop monotonicity:** for longs, the stop only moves UP (never down); for shorts the stop only moves DOWN. An adverse tick never loosens the stop.
- **min_lock dead zone:** while $\text{min_lock_pct} > 0$ and the market has not yet crossed that threshold, the trailing stop is attached and armed but NOT tracking — a close is NOT emitted no matter how far the price moves against you. A decision warning is logged on the trailing_stop_created row at create time.
- **Single trailing stop per cycle:** enforced by the partial unique index `uq_ep_active_trailing_stop_per_cycle`. A second create on the same open cycle returns HTTP 409.
- **Cycle close sweeps the plan:** if the cycle closes externally (reconciliation flat, manual close, bracket fill), `_sweep_cycle_closures()` cancels the trailing stop on the next clock tick.
- **Checkpoint persistence:** `peak_price` and `current_stop` are persisted every 10 s via the plan-executor checkpoint loop. On restart, state is restored and `peak_price` is floored at `entry_price` (long: $\max(\text{peak}, \text{entry})$; short: $\min(\text{peak}, \text{entry})$).
- **Downtime semantics:** if the service is down when the true market price crosses the stop, the stop fires on the first tick received after restart. There is no placed order on the exchange — the evaluator is purely server-side.

Interaction with brackets

A position cycle may carry both a bracket and a trailing_stop plan at the same time (the two partial unique indexes are disjoint per plan_type). Whichever fires first closes the cycle; the other is swept by the cycle-close handler.

Limitations

- entry_price and total_quantity are frozen at attach time. Dollar-cost averaging (scaling into a position) requires POST /cancel + re-attach after the new average is known.
- No exchange-side native trailing stops — every decision is made by PlanExecutorService against live ticks.
- Futures only (same capability gate as brackets: supports_reduce_only). Not supported on Kraken spot.
- No breakeven-move feature (ratchet stop to entry after N % profit).
- No time-based activation (arm after N minutes regardless of price).
- Live rollout gated on the same follow-ups (wallet-safe routing + orphan-cycle admin) that brackets depend on.

Cross-asset market-data pattern

Snapper enables strategies to subscribe to a market-data-only feed (SymbolExchangeCapability.can_trade=False) and emit signals whose target is an execution-capable instrument on a different venue. Kraken FCM index futures (kraken_equities) are the primary driver: they publish ~10-minute-delayed candles + ticks but have no order API.

Rules

- Strategies MAY subscribe to any combination of can_market_data=True topics regardless of the can_trade state of the source instrument.
- Every emitted StrategySignal.instrument MUST point at an can_trade=True instrument on the strategy's configured exchange. Snapper's order-entry capability guard (snapper.server._capability_guard.require_tradable) rejects submits against can_trade=False pairs with HTTP 422 error_code='instrument_market_data_only' — the signal would be dropped before reaching the venue.
- Tick-driven strategies that consume delayed feeds MUST gate on TickData.is_delayed=True before treating the price as current. Candle-driven cross-asset strategies inherit the source-feed latency implicitly; document the lag in the live runbook.

- Output instruments listed in `StrategyConfig.outputs` are validated at startup against the launching operator's scope grants. The execution target must fall within the operator's scope.

Reference implementation

See `src/snapper/strategies/examples/tradfi_observe_crypto_execute.py` for a minimal EMA-crossover strategy that observes MNQM6-CME on `kraken_equities` and targets BTC-USD on `kraken`. The file is NOT registered with the process registry — it ships as a copy-paste starting point. Activation steps live in the module docstring; `tests/meta/test_reference_strategy_not_registered.py` enforces the non-registration invariant so future maintainers cannot accidentally promote the illustration into a live process.